

Практика 5. Типы данных

осень 2026

Андрей Стоян, Илья Колегов

осень 2026

Летучка 5

Летучка 5 · 7 минут · Ваше ФИ: _____

1. Расставьте скобки в типе `String -> Int -> Bool`: _____
2. Напишите наиболее общий тип `\x -> (x, x)`: _____
3. Замените скобки на `$`, где это возможно: `f (g x) (h y) ≡` _____
4. Дано `fac 0 = 1; fac n = n * fac (n - 1)`. Один шаг без арифметики: `fac 3` $\rightsquigarrow$ _____
5. Список `[10, 20, 30]` сопоставили с образцом `x : xs`. Чему равно `xs`? _____

Конструкторы и сопоставление с образцом

Тип или конструктор?

✎ Напишите типы `Dog` и `Parrot`.

```
data CatT = CatD String Int
data Pet  = Cat CatT | Dog String Int | Parrot
```

⚡ **Решение.**

```
ghci> :t Dog
Dog :: String -> Int -> Pet
ghci> :t Parrot
Parrot :: Pet
```

От декларации к типам конструкторов

`data` объявляет новый тип `Pet`. Справа от `=` перечислены способы построить его значение.

```
data Pet = Cat CatT           -- новый тип Pet
        | Dog String Int
        | Parrot
```

От декларации к типам конструкторов

`Cat`, `Dog` и `Parrot` — новые конструкторы данных. `|` разделяет варианты.

```
data Pet = Cat CatT           конструктор Cat
         | Dog String Int      конструктор Dog
         | Parrot              конструктор Parrot
```

От декларации к типам конструкторов

После имени конструктора записаны типы его аргументов. У `Parrot` аргументов нет.

data	Pet	=	Cat	CatT	один аргумент: CatT
			Dog	String Int	два аргумента: String, Int
			Parrot		без аргументов

От декларации к типам конструкторов

Типы аргументов становятся входами конструктора. Тип результата у всех трёх — **Pet**.

Cat :: CatT -> Pet получаем Pet

Dog :: String -> Int -> Pet получаем Pet

Parrot :: Pet уже значение Pet

Разобрать то, что сконструировали

 **Устно:** Какую ветку выберет `showPet (Cat (CatD "murka" 2))`? Какие имена свяжет образец?

```
1 showPet :: Pet -> String
2 showPet pet = case pet of
3   Parrot -> "Parrot"
4   Dog name _ -> "Dog: " ++ name
5   Cat (CatD name _) -> "Cat: " ++ name
```

```
showPet (Cat (CatD "murka" 2))
```

Разобрать то, что сконструировали

`Parrot` и `Dog` не подходят. В последней ветке совпадают оба конструктора: `Cat` и `CatD`.

```
1 showPet :: Pet -> String
2 showPet pet = case pet of
3   Parrot -> "Parrot"
4   Dog name _ -> "Dog: " ++ name
5   Cat (CatD name _) -> "Cat: " ++ name
```

`showPet (Cat (CatD "murka" 2))`

значение `Cat` (`CatD` "murka" 2)

образец `Cat` (`CatD` name _)

Разобрать то, что сконструировали

`name` связывается с `"murka"`. Образец `_` принимает возраст `2`, не связывая имени.

```
1 showPet :: Pet -> String
2 showPet pet = case pet of
3   Parrot -> "Parrot"
4   Dog name _ -> "Dog: " ++ name
5   Cat (CatD name _) -> "Cat: " ++ name
```

`showPet (Cat (CatD "murka" 2))`

значение `Cat ( CatD    "murka"      2    )`

`name =` `"murka"`

образец `Cat ( CatD    name      _    )`

возраст игнорируется

Разобрать то, что сконструировали

Берём правую часть выбранной ветки и подставляем значение `name`.

```
1 showPet :: Pet -> String
2 showPet pet = case pet of
3   Parrot -> "Parrot"
4   Dog name _ -> "Dog: " ++ name
5   Cat (CatD name _) -> "Cat: " ++ name
```

"Cat: " ++ "murka"

name = "murka"

Разобрать то, что сконструировали

Конкатенация строк даёт результат "Cat: murka".

```
1 showPet :: Pet -> String
2 showPet pet = case pet of
3   Parrot -> "Parrot"
4   Dog name _ -> "Dog: " ++ name
5   Cat (CatD name _) -> "Cat: " ++ name
```

"Cat: murka"

Возраст: что делать с попугаем?

✎ Закончите функцию, чтобы она обрабатывала каждый вариант `Pet`. Возраст попугая в нашем типе не хранится.

```
age :: Pet -> Maybe Int
age pet = case pet of
  Dog _ age -> Just age
  Cat (CatD _ age) -> Just age
  -- допишите недостающую ветку
```

⚡Решение.

```
Parrot -> Nothing
```

`Nothing` — нет данных о возрасте; `Just 0` — известный возраст, равный нулю.

- **Δ Полнота сопоставления:** учтены все конструкторы

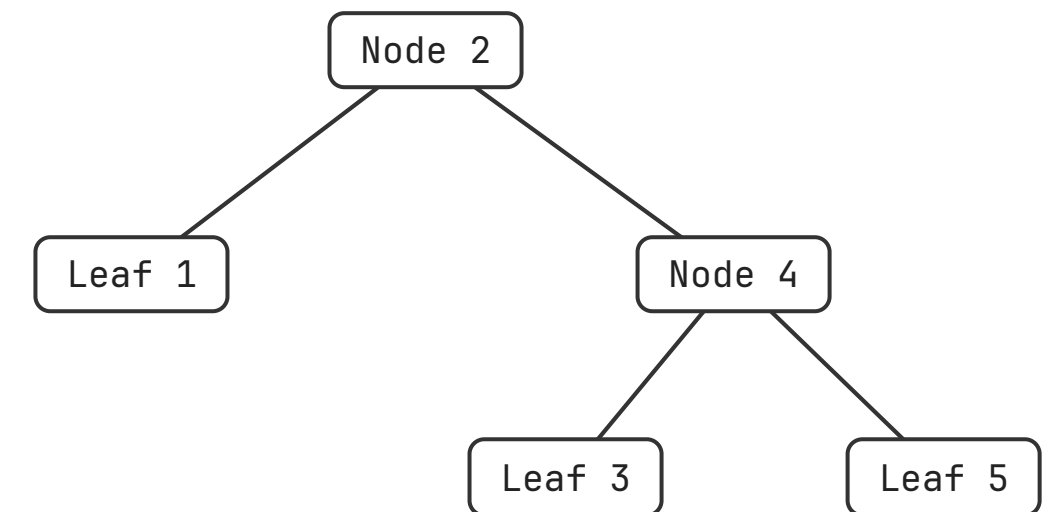
Рекурсивные структуры данных

Как устроено значение дерева

```
data Tree a = Leaf a | Node (Tree a) a (Tree a)
```

```
t :: Tree Int  
t = Node (Leaf 1) 2 (Node (Leaf 3) 4 (Leaf 5))
```

- **Leaf** хранит одно значение
- **Node** хранит значение и два поддерева
- **a** — тип значений: здесь **Tree Int**



Проредуцируйте высоту

✎ Проредуцируйте `height t` до числа. Высоту считаем в вершинах: у листа она равна 1.

```
data Tree a = Leaf a | Node (Tree a) a (Tree a)
```

```
t :: Tree Int  
t = Node (Leaf 1) 2 (Node (Leaf 3) 4 (Leaf 5))
```

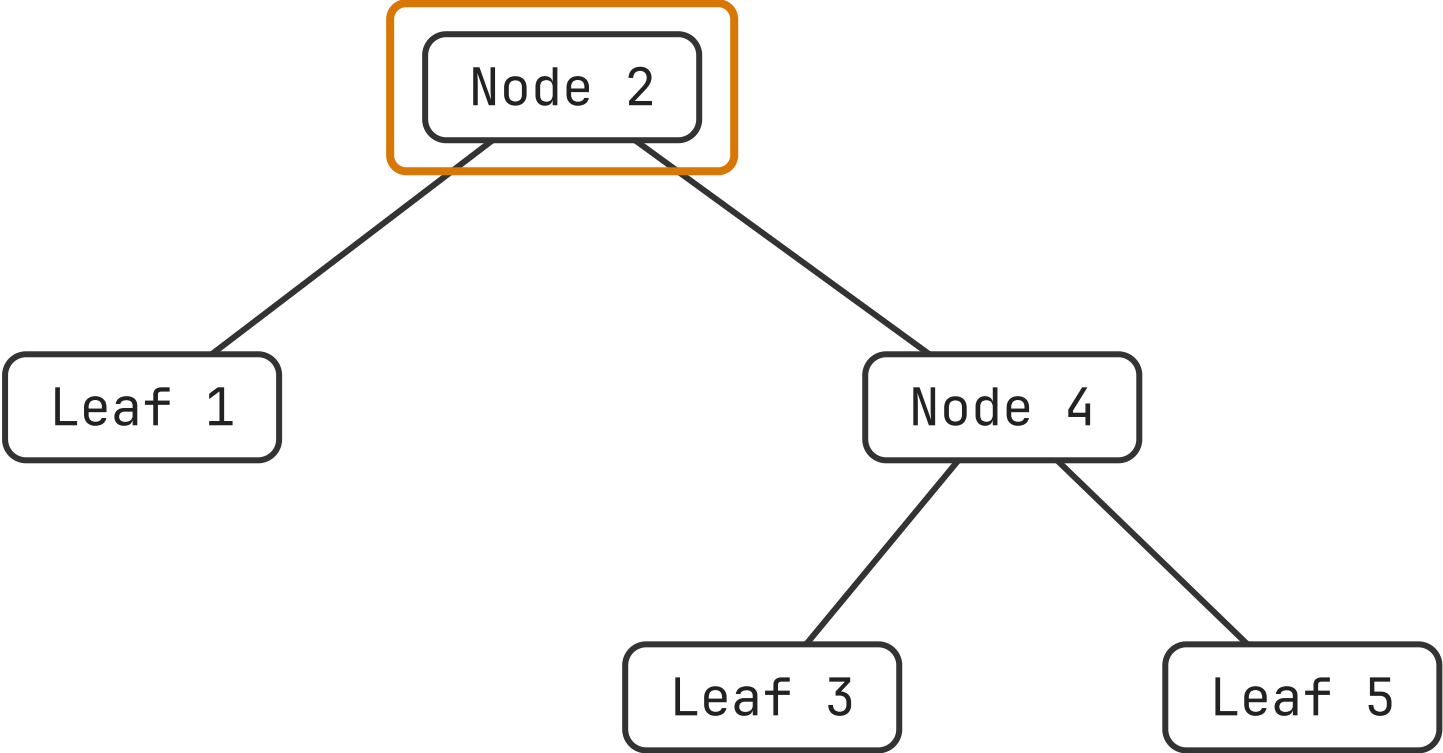
```
height :: Tree a -> Int  
height t = case t of  
  Leaf _ -> 1  
  Node l _ r -> 1 + max (height l) (height r)
```

Высота: полный терм и спуск

Проредуцируем весь терм. Жёлтый фрагмент переписывается следующим кликом.

```
height
(Node
  (Leaf 1)
  2
  (Node
    (Leaf 3)
    4
    (Leaf 5)))
```

ШАГИ РЕДУКЦИИ
height: правило Node



Высота: полный терм и спуск

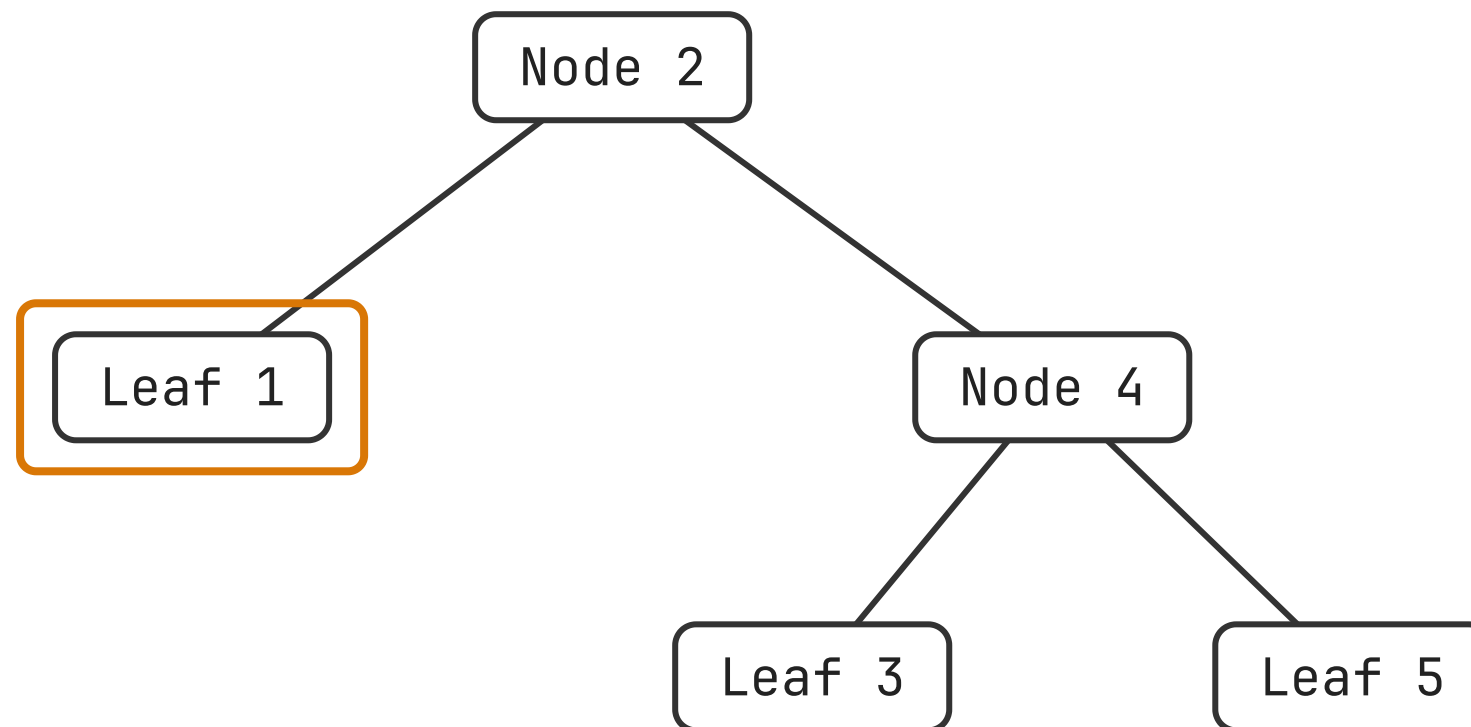
Раскрыли корень; теперь переписываем левый вызов.

```
1 + max
  (height (Leaf 1))
  (height
    (Node
      (Leaf 3)
      4
      (Leaf 5)))
```

ШАГИ РЕДУКЦИИ

height: правило Node

height: база Leaf 1

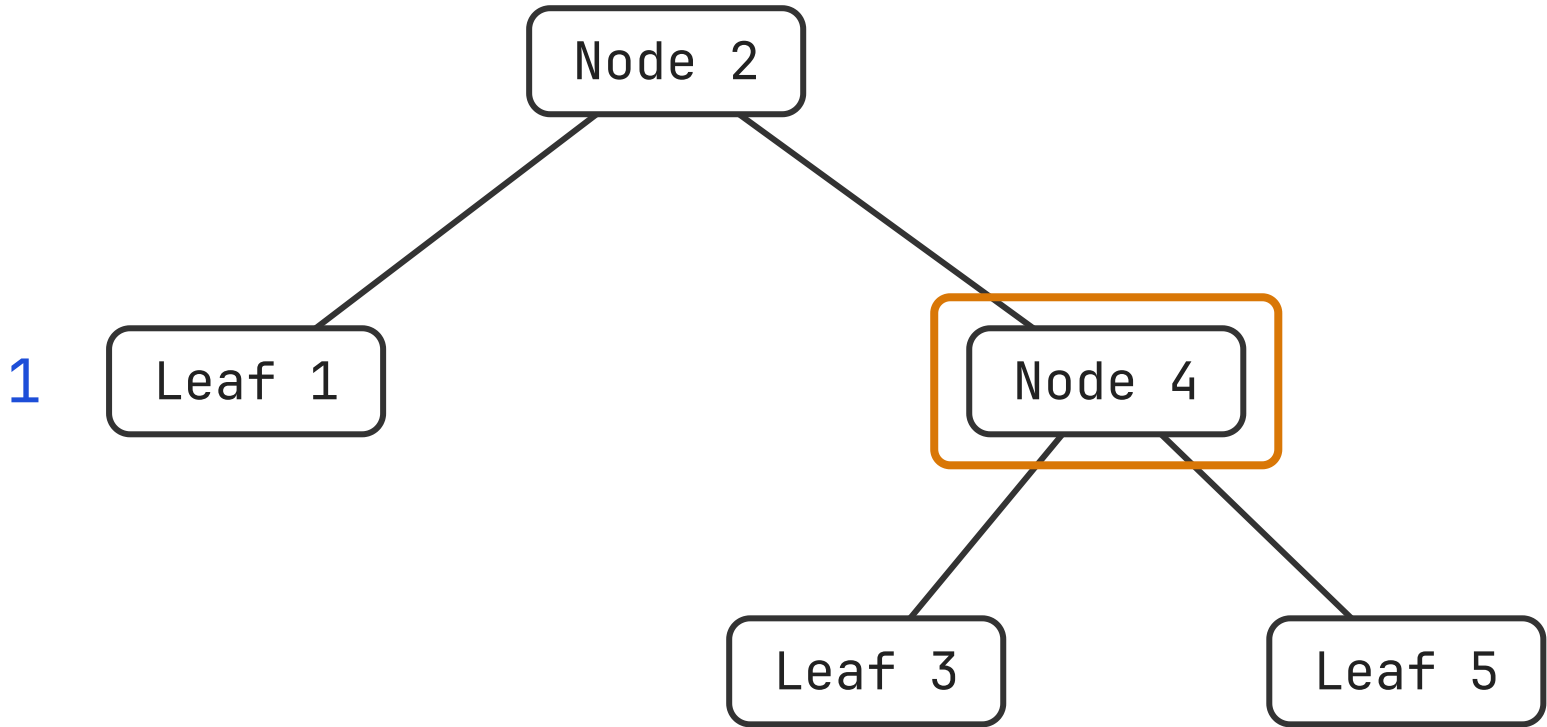


Высота: полный терм и спуск

Высота слева известна. Весь ожидающий контекст остаётся на экране.

```
1 + max
  1
  (height
    (Node
      (Leaf 3)
      4
      (Leaf 5)))
```

- ШАГИ РЕДУКЦИИ
- height: правило Node
 - height: база Leaf 1
 - height: правило Node 4

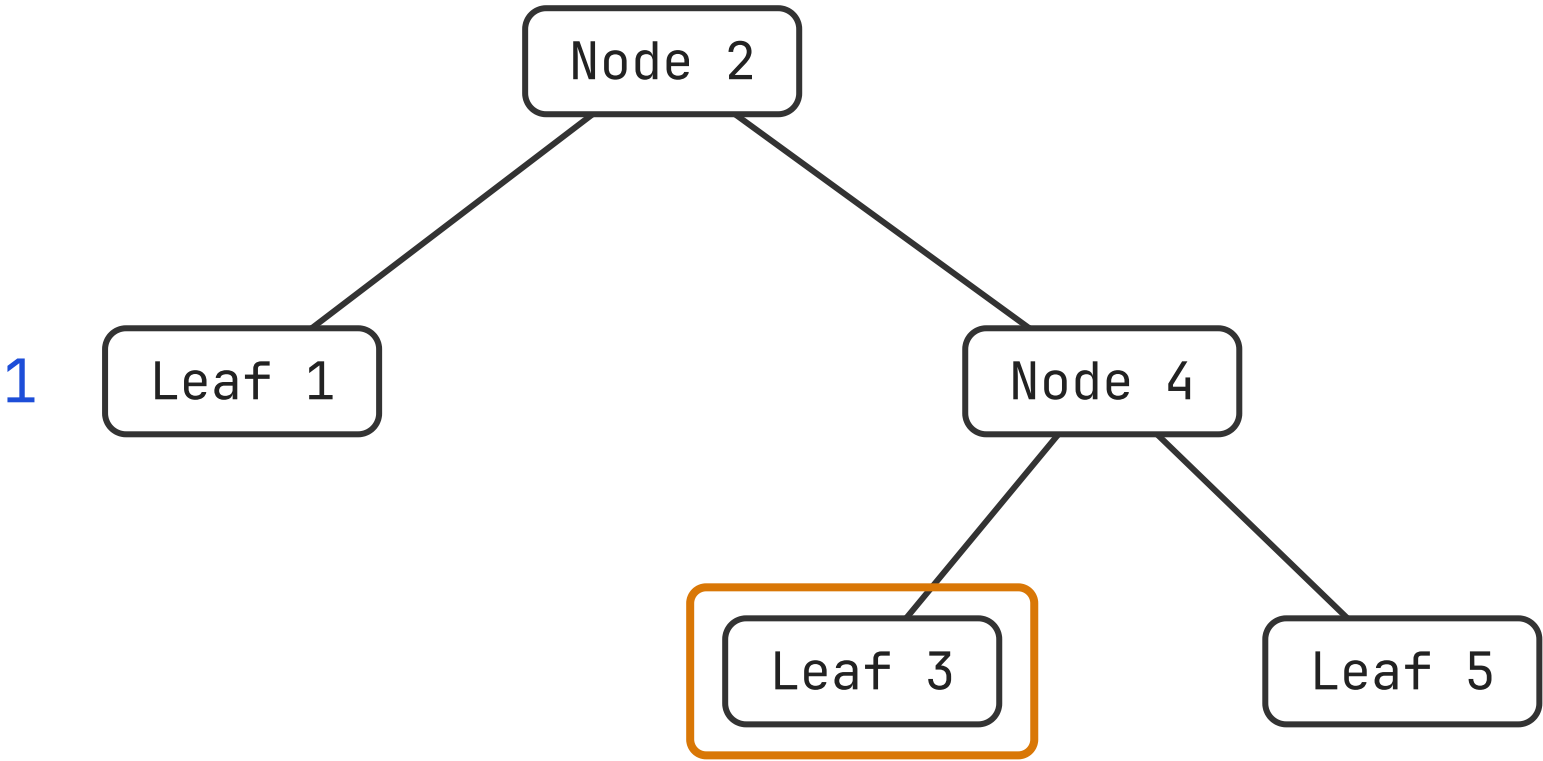


Высота: полный терм и спуск

Правое поддереву тоже Node: ещё два вызова в прежнем контексте.

```
1 + max
  1
  (1 + max
    (height (Leaf 3))
    (height (Leaf 5)))
```

- ШАГИ РЕДУКЦИИ
- height: правило Node
 - height: база Leaf 1
 - height: правило Node 4
 - height: база Leaf 3

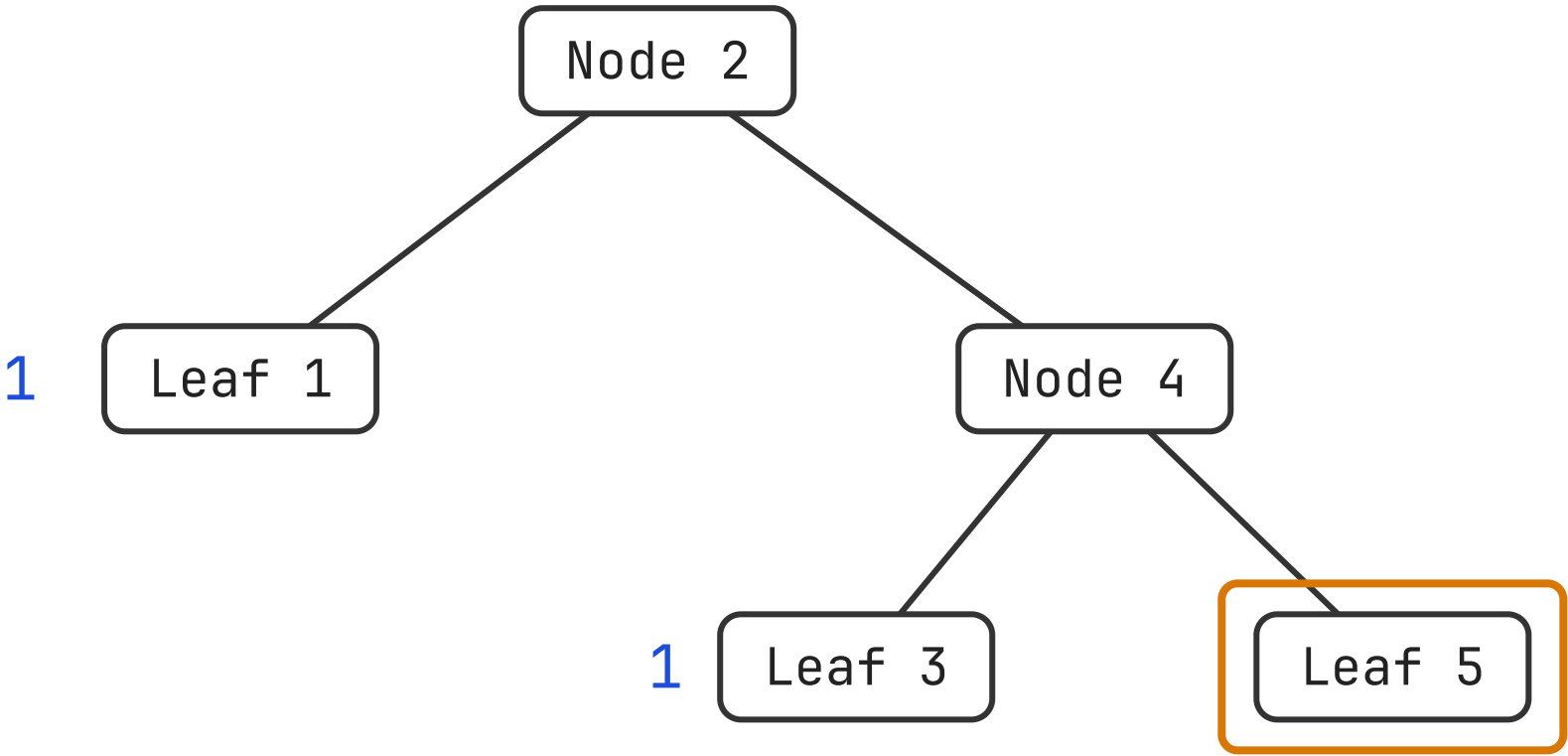


Высота: полный терм и спуск

Левый лист справа дал 1; осталось вычислить последний лист.

```
1 + max
  1
  (1 + max
    1
    (height (Leaf 5)))
```

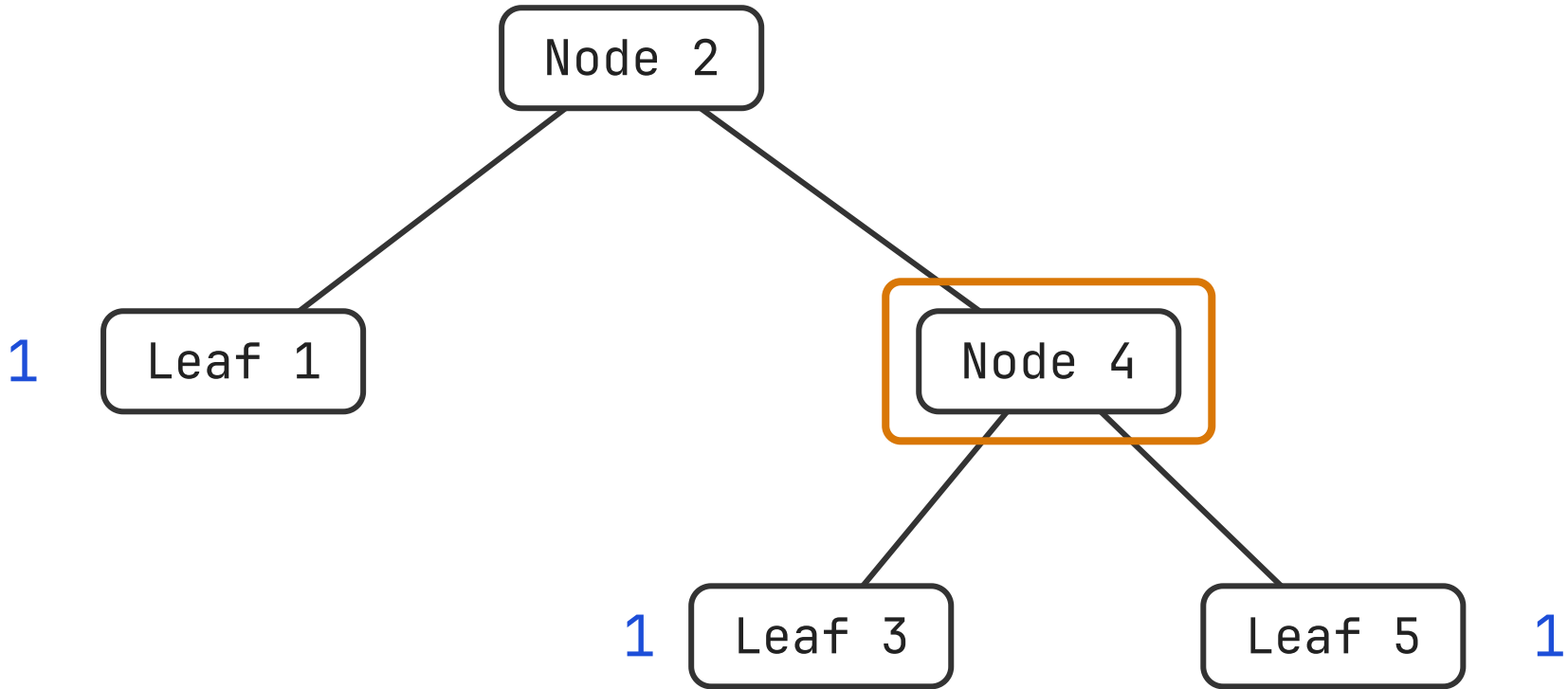
- ШАГИ РЕДУКЦИИ
- height: правило Node
 - height: база Leaf 1
 - height: правило Node 4
 - height: база Leaf 3
 - height: база Leaf 5



Высота: полный терм и спуск

Все листья вычислены. Сначала внутренний max.

$$1 + \max \begin{matrix} 1 \\ (1 + \max \\ \quad 1 \\ \quad 1) \end{matrix}$$



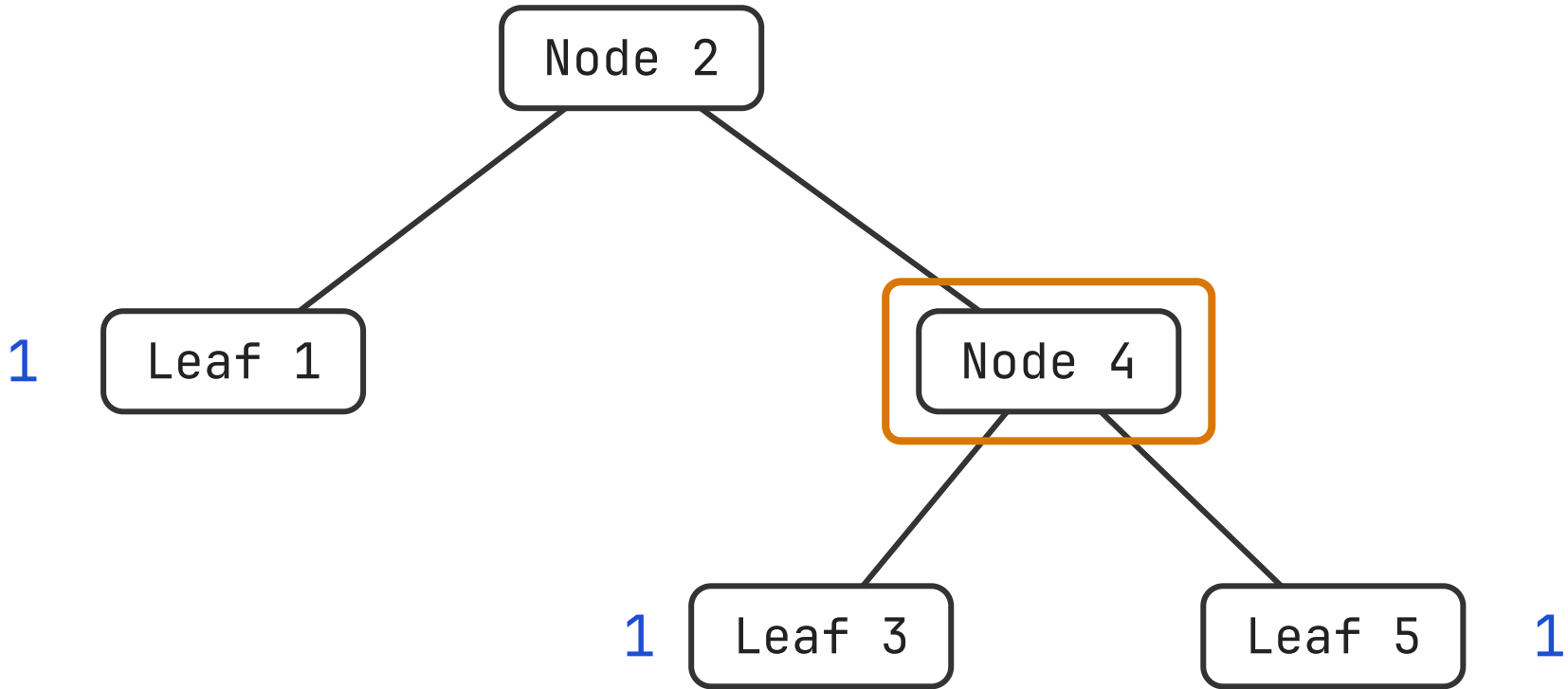
ШАГИ РЕДУКЦИИ

- height: правило Node
- height: база Leaf 1
- height: правило Node 4
- height: база Leaf 3
- height: база Leaf 5
- max 1 1 $\rightsquigarrow$ 1

Высота: полный терм и спуск

В правом поддереве осталось сложение.

$$1 + \max_{1 \atop (1 + 1)}$$



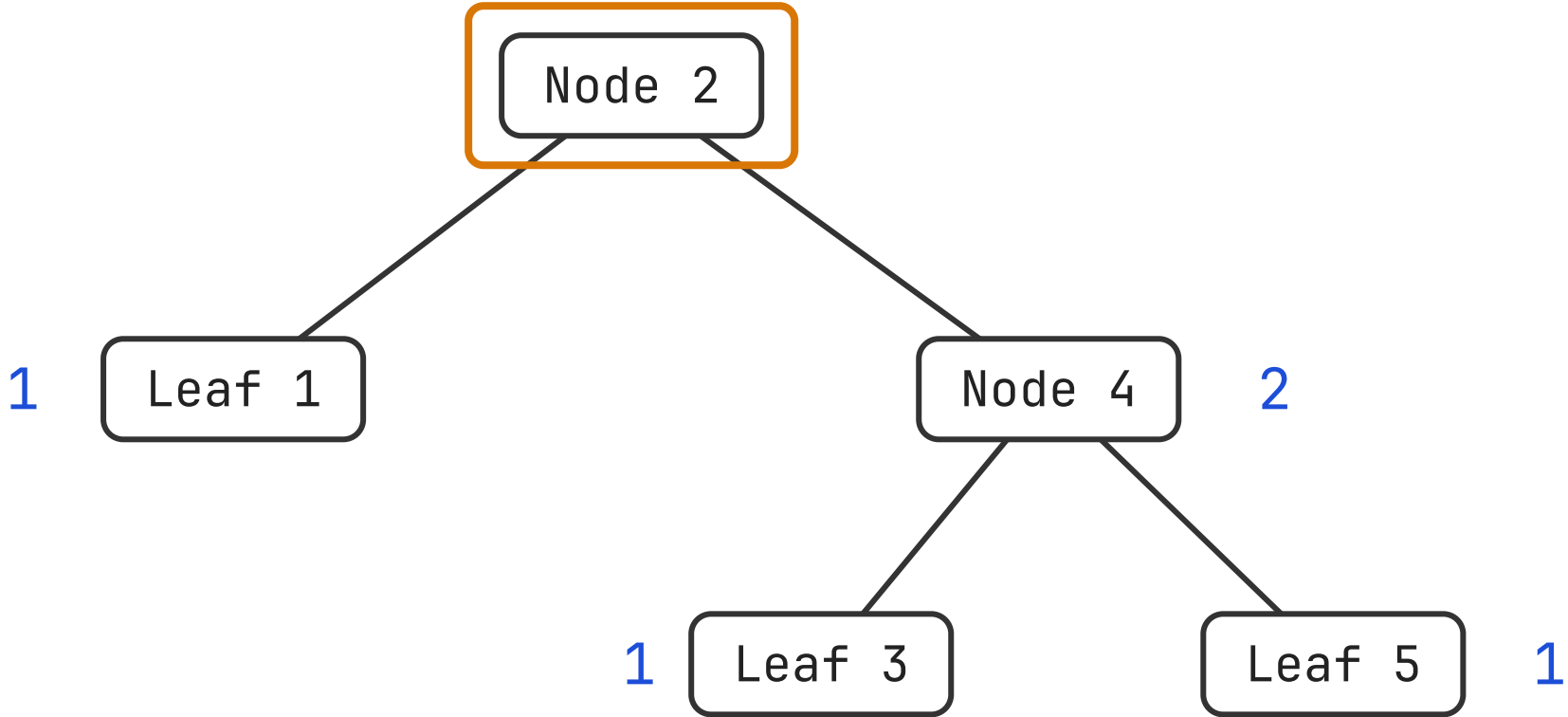
ШАГИ РЕДУКЦИИ

height: правило Node
height: база Leaf 1
height: правило Node 4
height: база Leaf 3
height: база Leaf 5
max 1 1 $\rightsquigarrow$ 1
1 + 1 $\rightsquigarrow$ 2

Высота: полный терм и спуск

Высота справа равна 2. Теперь внешний max.

1 + max
1
2



ШАГИ РЕДУКЦИИ

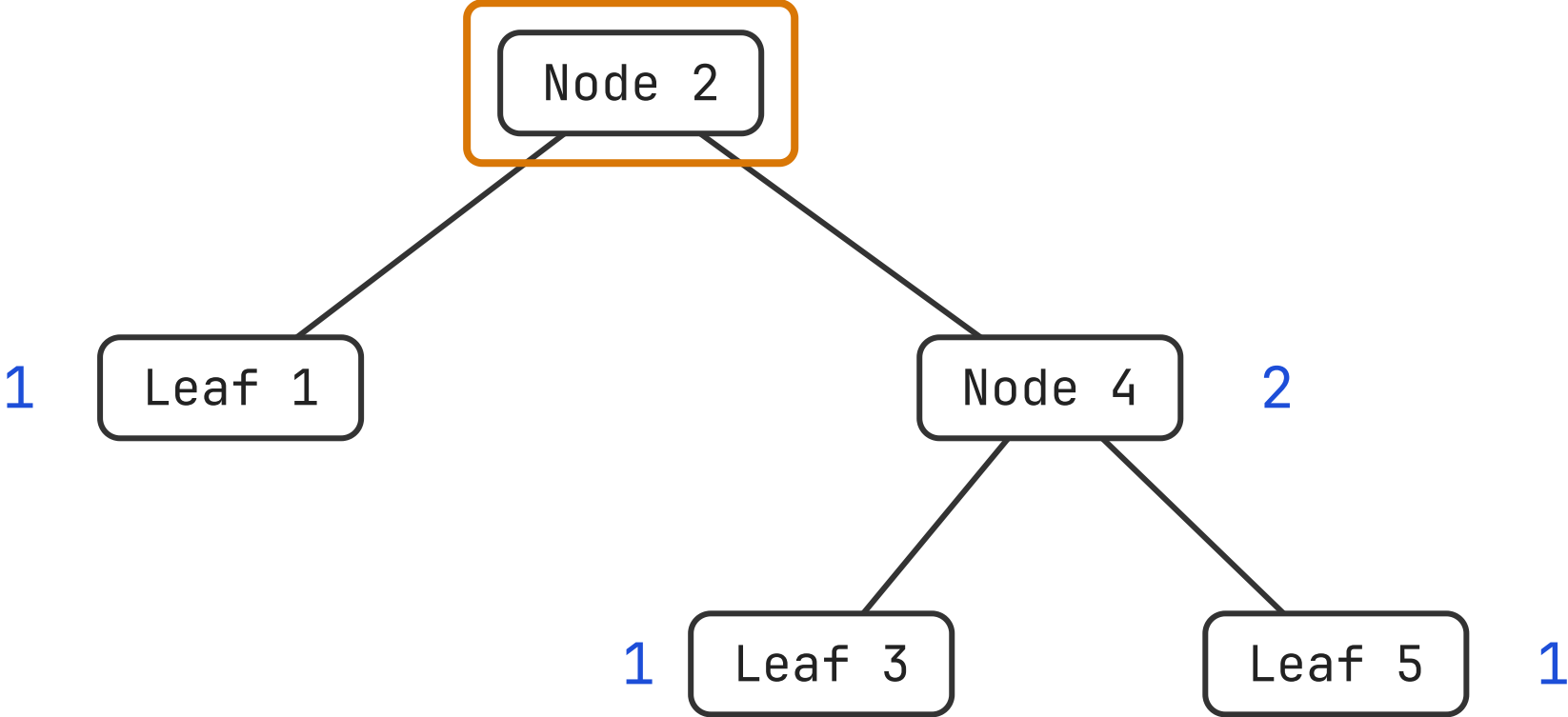
height: правило Node
height: база Leaf 1
height: правило Node 4
height: база Leaf 3
height: база Leaf 5

max 1 1 $\rightsquigarrow$ 1
1 + 1 $\rightsquigarrow$ 2
max 1 2 $\rightsquigarrow$ 2

Высота: полный терм и спуск

Осталось последнее сложение у корня.

1 + 2

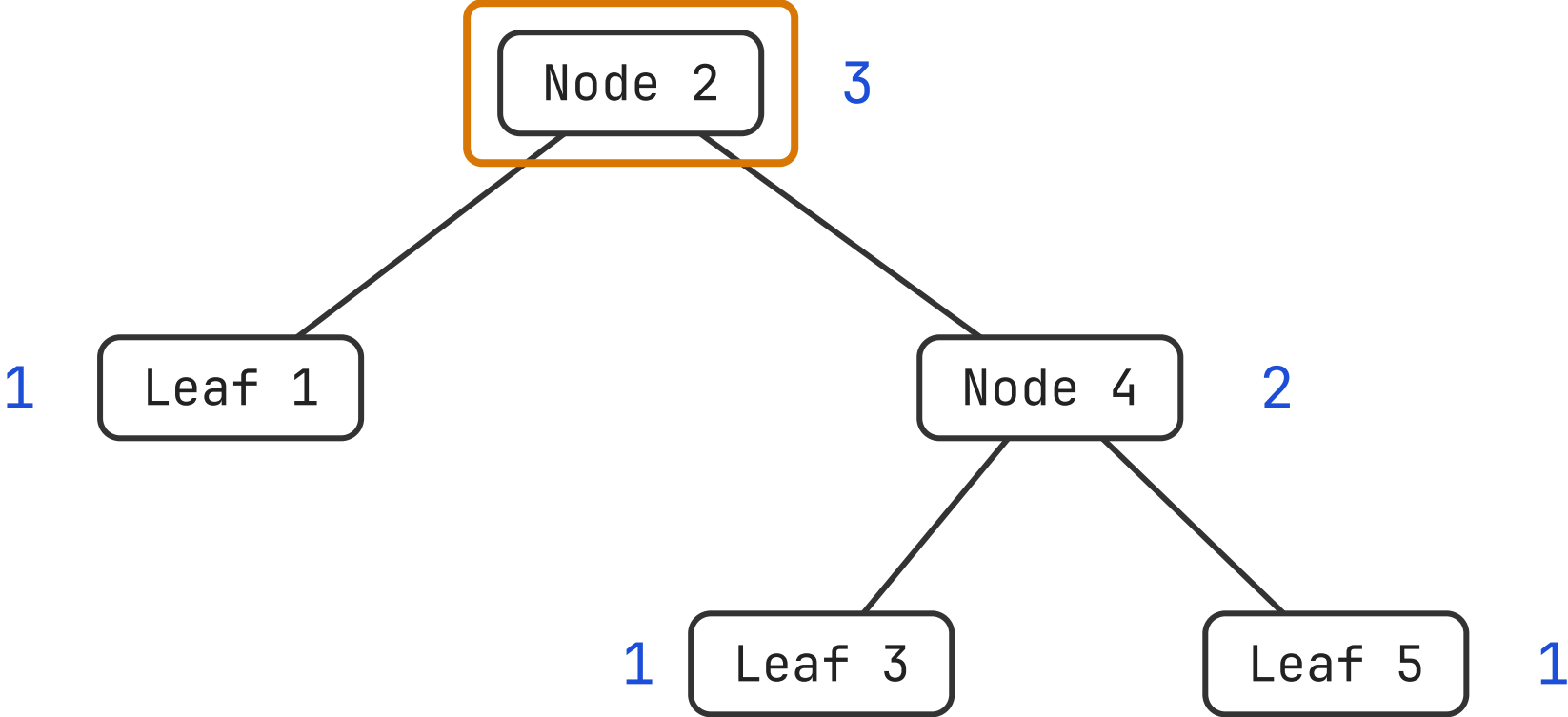


- ШАГИ РЕДУКЦИИ
- height: правило Node
 - height: база Leaf 1
 - height: правило Node 4
 - height: база Leaf 3
 - height: база Leaf 5
 - max 1 1 $\rightsquigarrow$ 1
 - 1 + 1 $\rightsquigarrow$ 2
 - max 1 2 $\rightsquigarrow$ 2
 - 1 + 2 $\rightsquigarrow$ 3**

Высота: полный терм и спуск

Полный терм проредуцирован до 3.

3



ШАГИ РЕДУКЦИИ

height: правило Node
height: база Leaf 1
height: правило Node 4
height: база Leaf 3
height: база Leaf 5
 $\max\ 1\ 1 \rightsquigarrow 1$
 $1 + 1 \rightsquigarrow 2$
 $\max\ 1\ 2 \rightsquigarrow 2$
 $1 + 2 \rightsquigarrow 3$
результат 3

Напишите сумму значений дерева

✎ Напишите `sumTree :: Tree Int -> Int` целиком. Функция должна складывать все значения дерева: и в листьях, и во внутренних узлах.

⚡ *Решение.*

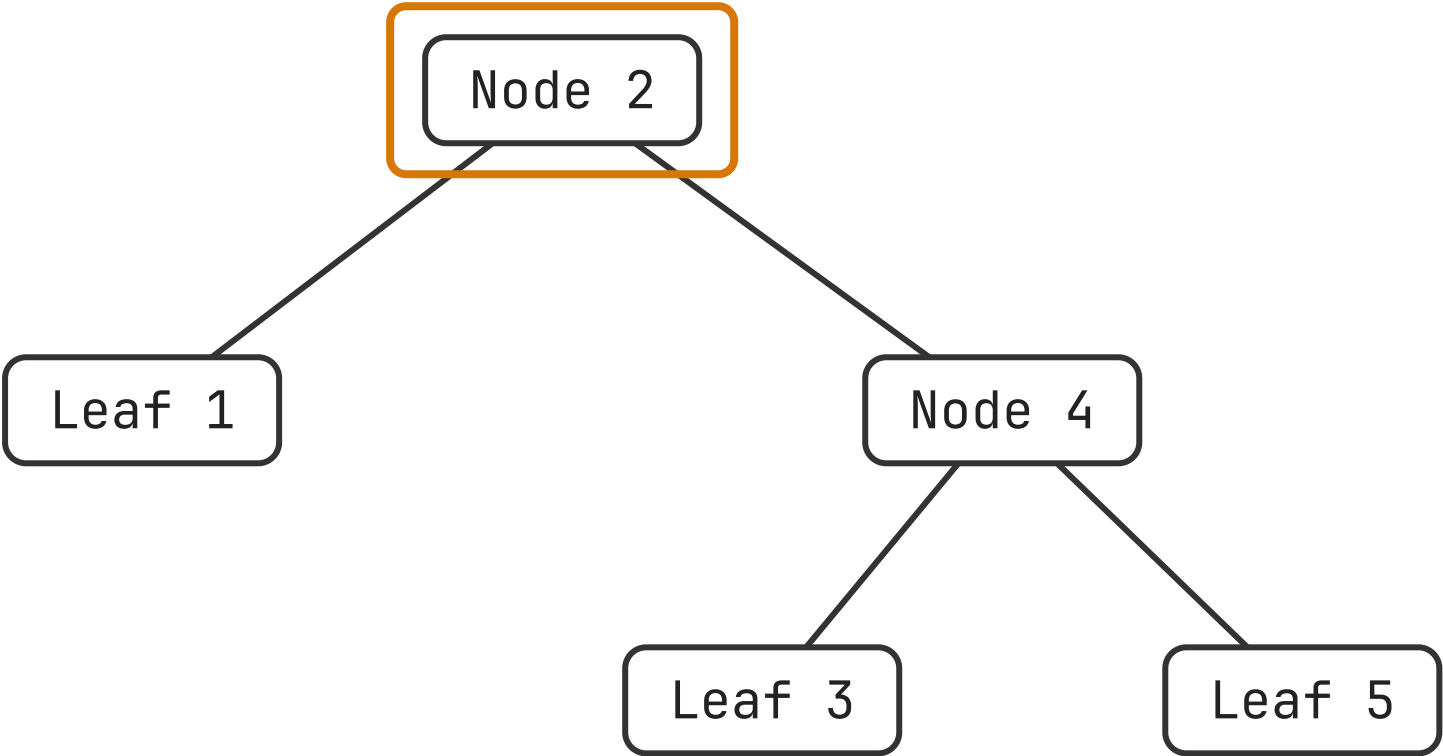
```
sumTree :: Tree Int -> Int
sumTree t = case t of
  Leaf x -> x
  Node l x r -> sumTree l + x + sumTree r
```

Сумма: полный терм и спуск

Проредуцируем sumTree на том же дереве.

```
sumTree
(Node
  (Leaf 1)
  2
  (Node
    (Leaf 3)
    4
    (Leaf 5)))
```

ШАГИ РЕДУКЦИИ
sumTree: правило Node

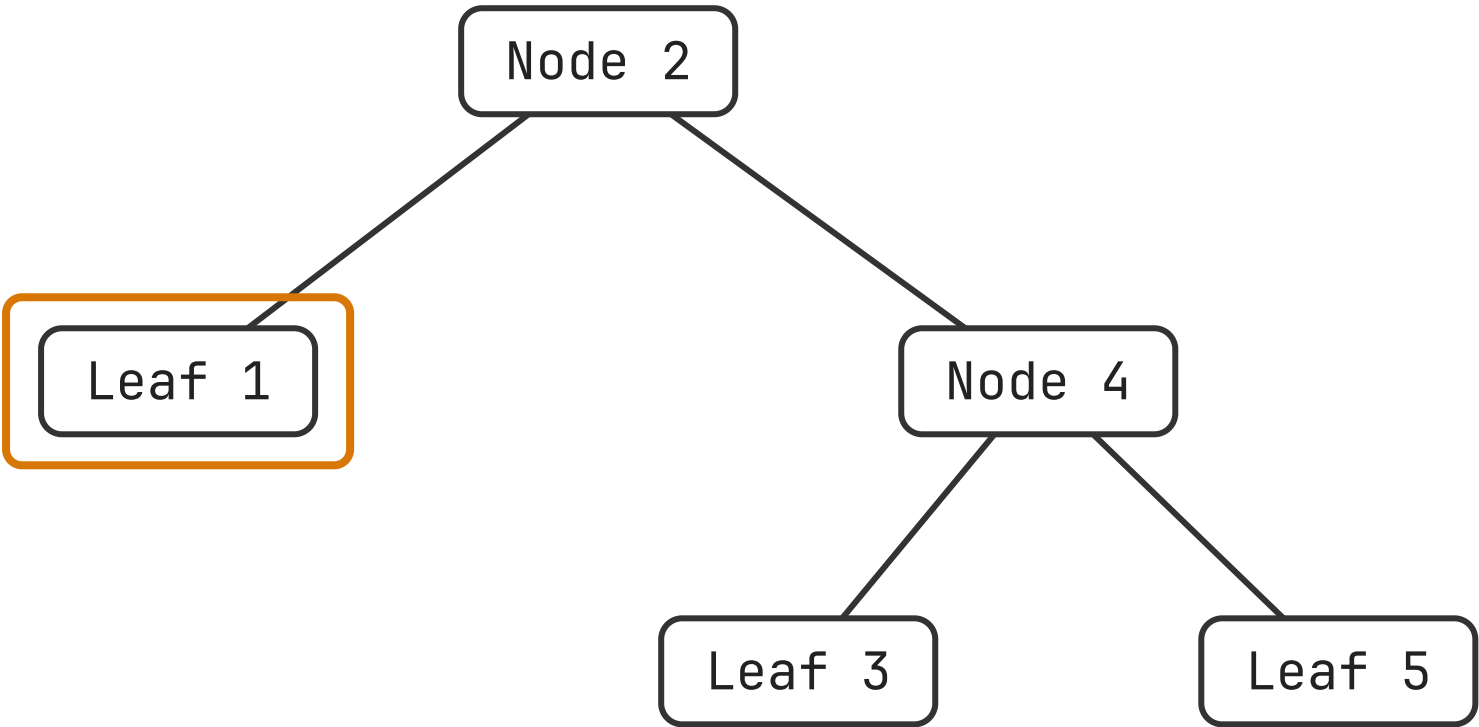


Сумма: полный терм и спуск

У корня остаются значение 2 и вызов справа.

```
sumTree (Leaf 1) + 2
+ sumTree
  (Node
    (Leaf 3)
    4
    (Leaf 5))
```

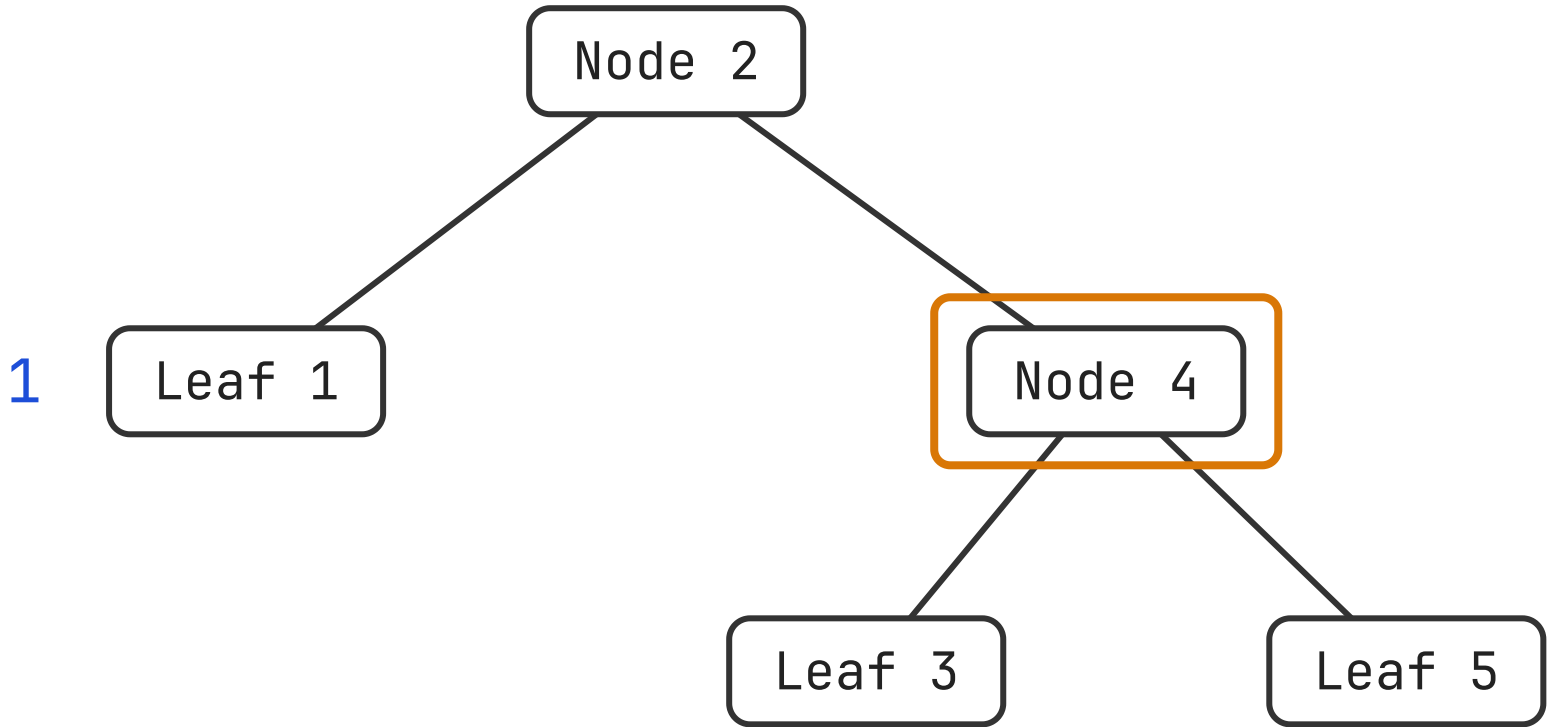
ШАГИ РЕДУКЦИИ
sumTree: правило Node
sumTree: база Leaf 1



Сумма: полный терм и спуск

Левый лист дал 1. Переписываем правый вызов.

```
1 + 2
+ sumTree
  (Node
    (Leaf 3)
    4
    (Leaf 5))
```



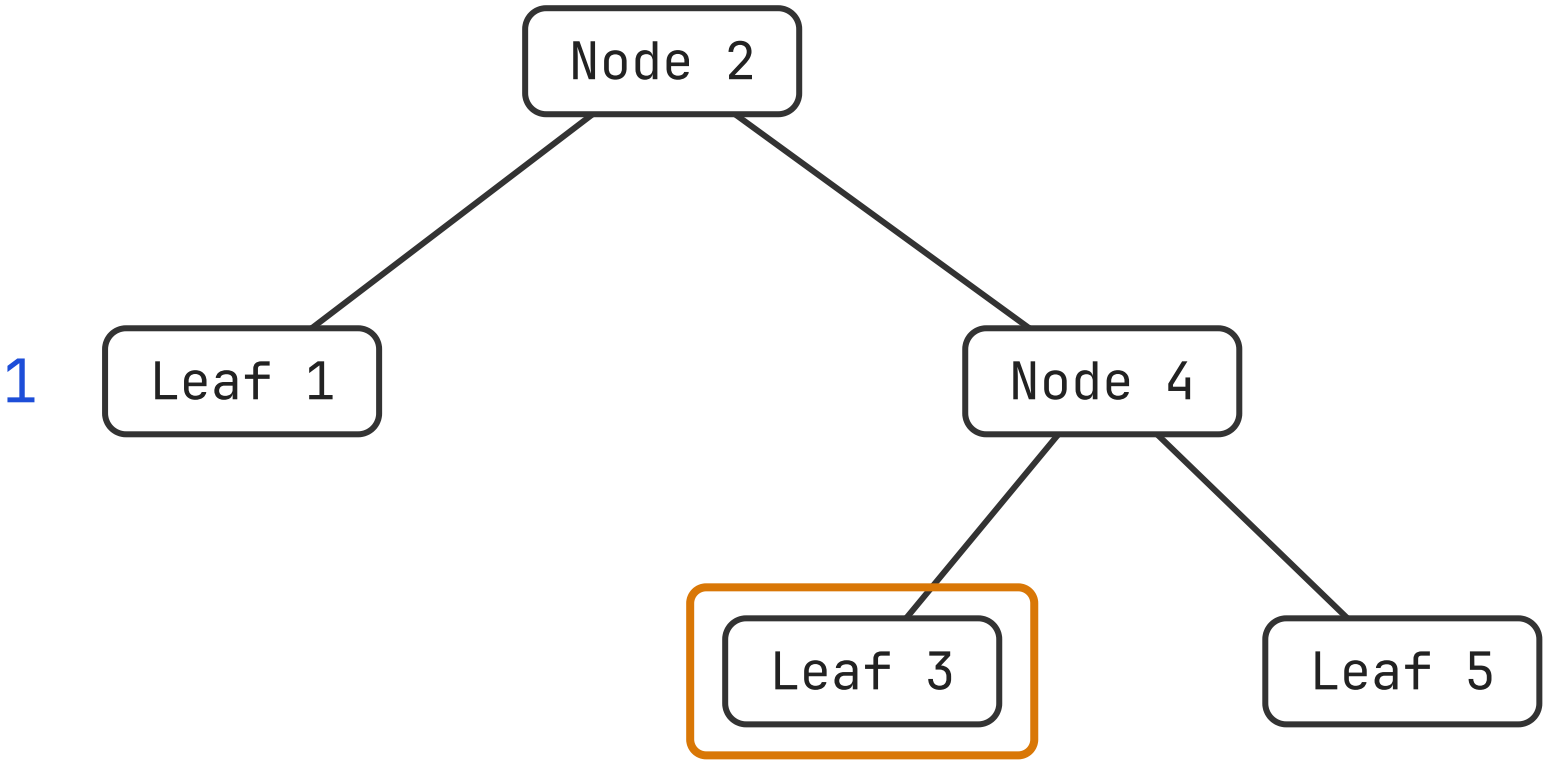
- ШАГИ РЕДУКЦИИ
- sumTree: правило Node
 - sumTree: база Leaf 1
 - sumTree: правило Node 4

Сумма: полный терм и спуск

Спускаемся к Leaf 3; внешняя сумма остаётся видимой.

```
1 + 2
+ sumTree (Leaf 3) + 4
+ sumTree (Leaf 5)
```

- ШАГИ РЕДУКЦИИ
- sumTree: правило Node
 - sumTree: база Leaf 1
 - sumTree: правило Node 4
 - sumTree: база Leaf 3

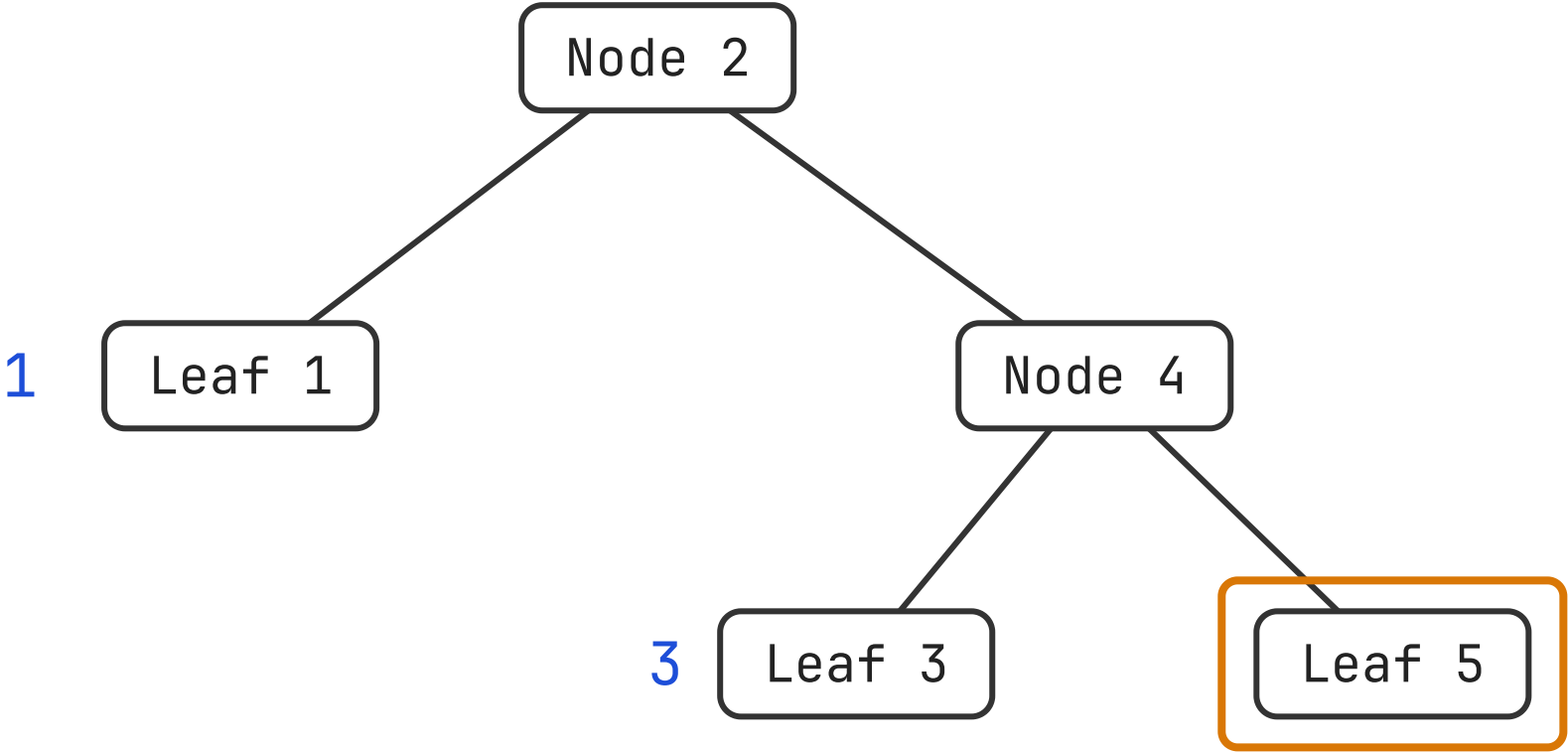


Сумма: полный терм и спуск

Leaf 3 дал 3; осталось вычислить последний лист.

1 + 2
+ 3 + 4
+ sumTree (Leaf 5)

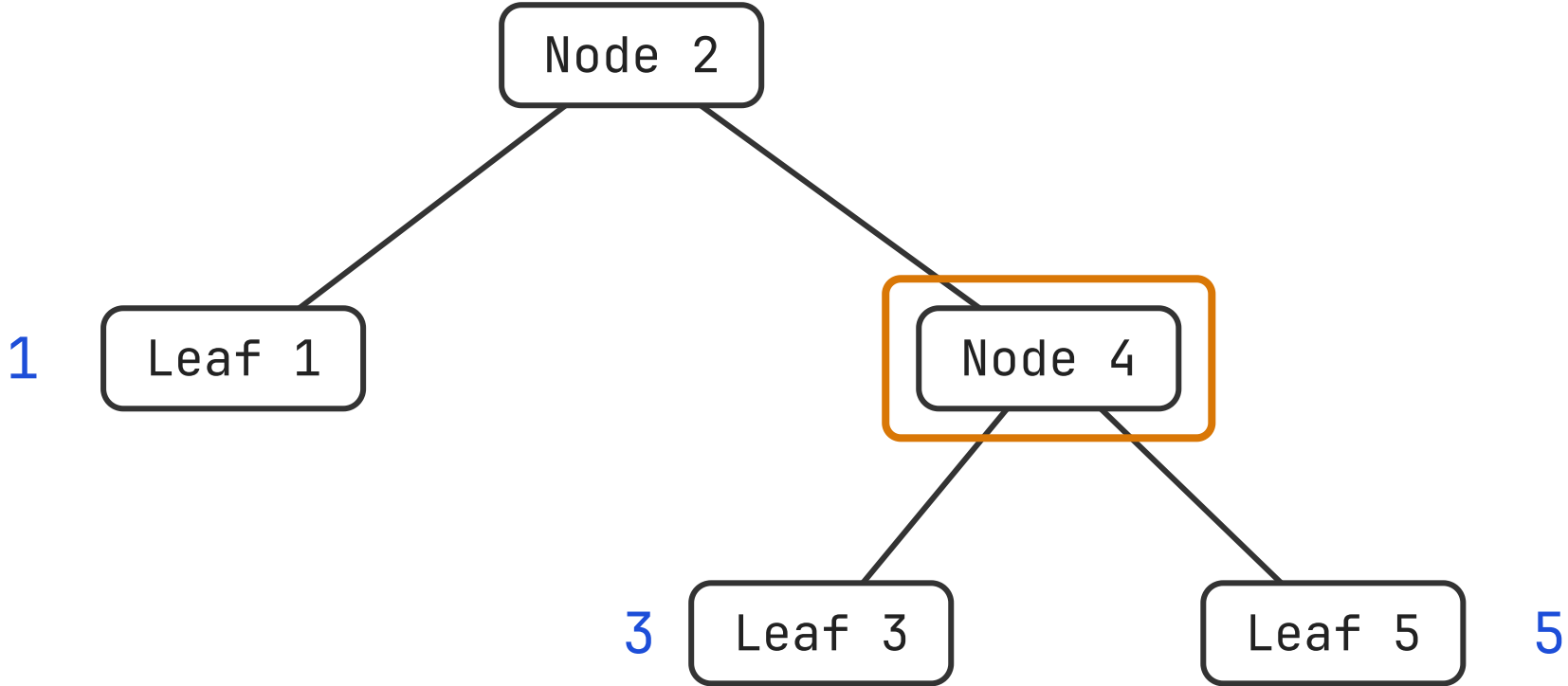
- ШАГИ РЕДУКЦИИ
- sumTree: правило Node
 - sumTree: база Leaf 1
 - sumTree: правило Node 4
 - sumTree: база Leaf 3
 - sumTree: база Leaf 5**



Сумма: полный терм и спуск

Теперь суммируем значения внутри правого поддерева.

$$\begin{array}{r} 1 + 2 \\ + 3 + 4 \\ + 5 \end{array}$$



ШАГИ РЕДУКЦИИ

sumTree: правило Node

sumTree: база Leaf 1

sumTree: правило Node 4

sumTree: база Leaf 3

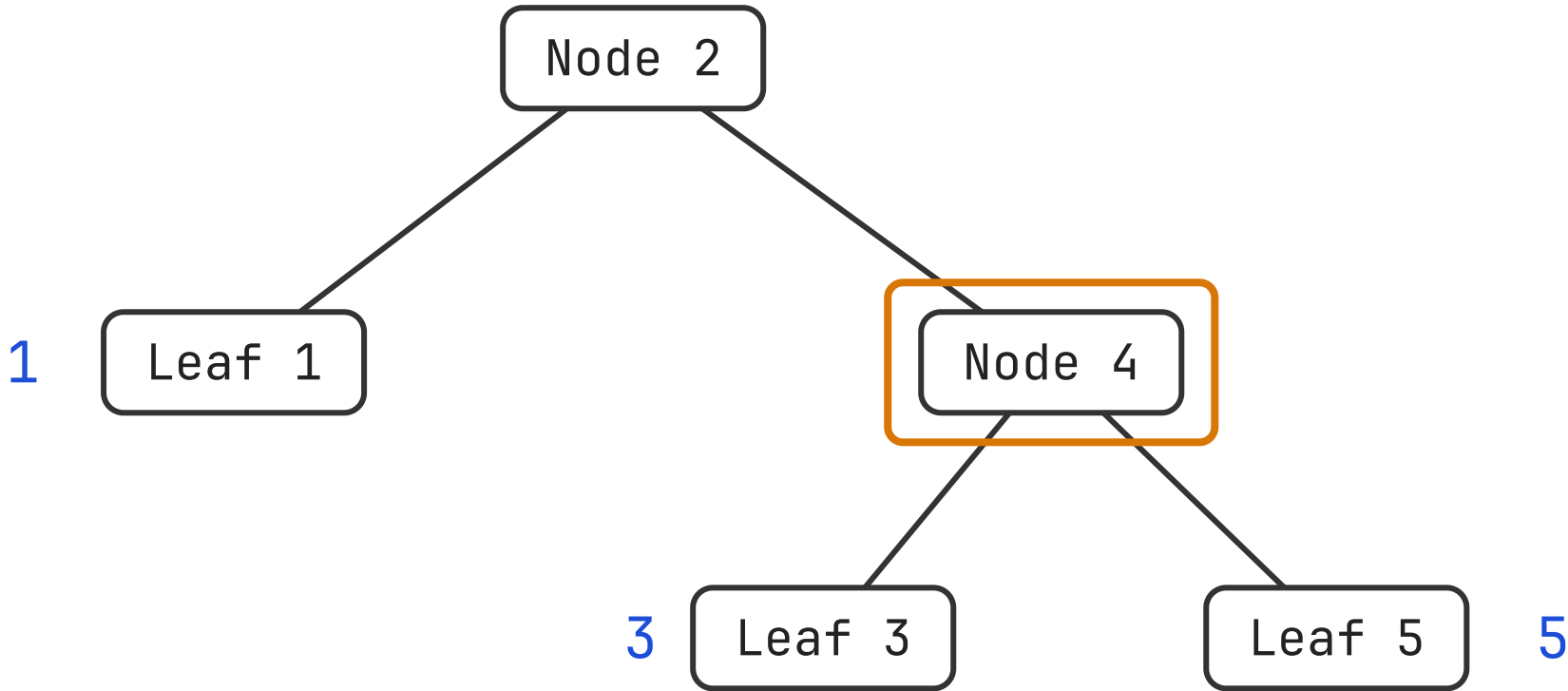
sumTree: база Leaf 5

$$3 + 4 \rightsquigarrow 7$$

Сумма: полный терм и спуск

Правый узел прибавляет последнее значение.

$$\begin{array}{r} 1 + 2 \\ + 7 + 5 \end{array}$$



ШАГИ РЕДУКЦИИ

sumTree: правило Node

sumTree: база Leaf 1

sumTree: правило Node 4

sumTree: база Leaf 3

sumTree: база Leaf 5

$$3 + 4 \rightsquigarrow 7$$

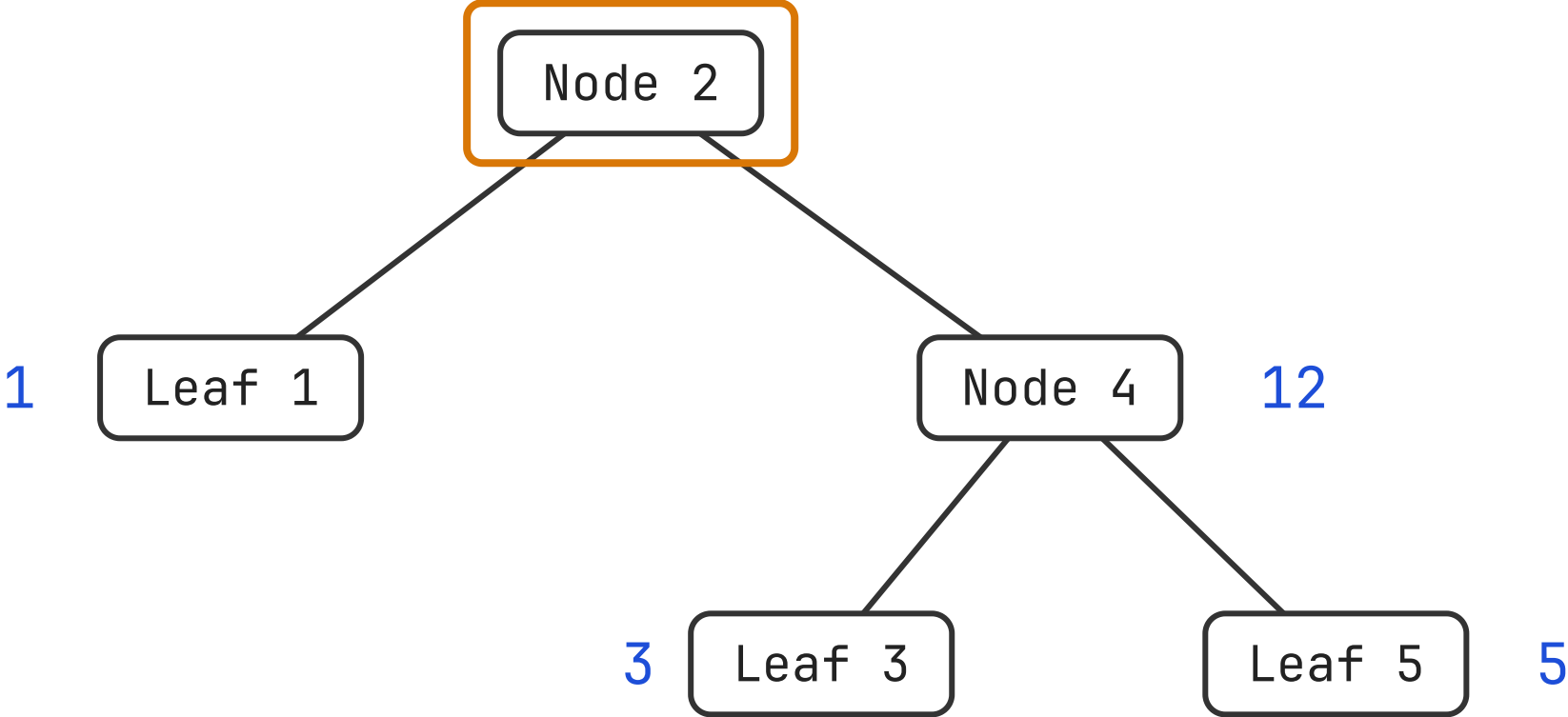
$$7 + 5 \rightsquigarrow 12$$

Сумма: полный терм и спуск

Правая сумма равна 12. Возвращаемся к внешнему контексту.

1 + 2

+ 12



ШАГИ РЕДУКЦИИ

sumTree: правило Node

sumTree: база Leaf 1

sumTree: правило Node 4

sumTree: база Leaf 3

sumTree: база Leaf 5

3 + 4

→ 7

7 + 5

→ 12

1 + 2

→ 3

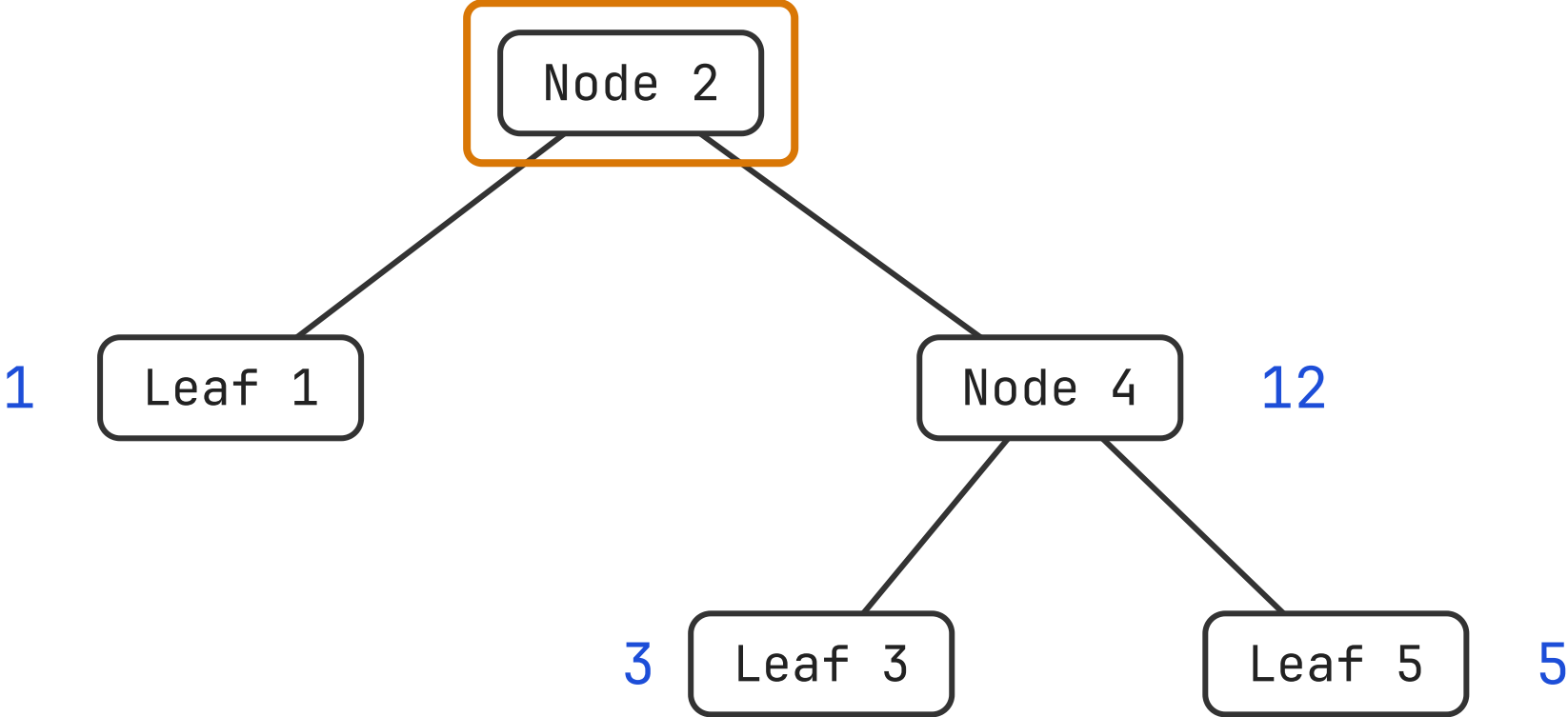
Сумма: полный терм и спуск

Осталось сложить два результата.

3

+

12



ШАГИ РЕДУКЦИИ

sumTree: правило Node

sumTree: база Leaf 1

sumTree: правило Node 4

sumTree: база Leaf 3

sumTree: база Leaf 5

$$3 + 4 \rightsquigarrow 7$$

$$7 + 5 \rightsquigarrow 12$$

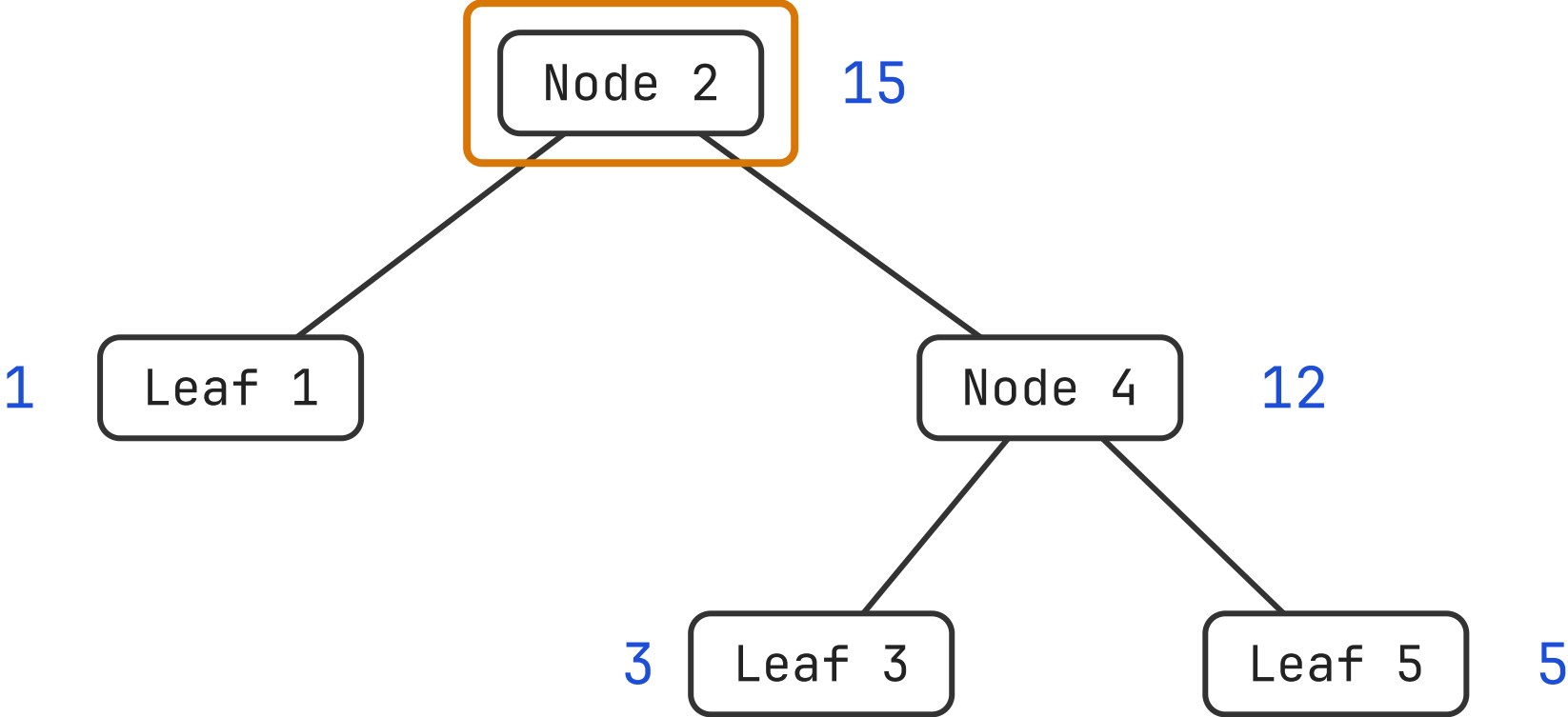
$$1 + 2 \rightsquigarrow 3$$

$$3 + 12 \rightsquigarrow 15$$

Сумма: полный терм и спуск

Полный терм проредуцирован до 15.

15



ШАГИ РЕДУКЦИИ

sumTree: правило Node

sumTree: база Leaf 1

sumTree: правило Node 4

sumTree: база Leaf 3

sumTree: база Leaf 5

$$3 + 4 \rightsquigarrow 7$$

$$7 + 5 \rightsquigarrow 12$$

$$1 + 2 \rightsquigarrow 3$$

$$3 + 12 \rightsquigarrow 15$$

результат 15

Меняем значения, сохраняем дерево

✎ Заполните пропуски. Функция `f` может менять тип содержимого, а форма дерева должна сохраниться.

```
transform :: (a -> b) -> Tree a -> Tree b
transform f t = case t of
  Leaf x -> Leaf _____
  Node l x r -> Node _____
```

⚡Решение.

```
transform f t = case t of
  Leaf x -> Leaf (f x)
  Node l x r -> Node (transform f l) (f x) (transform f r)
```

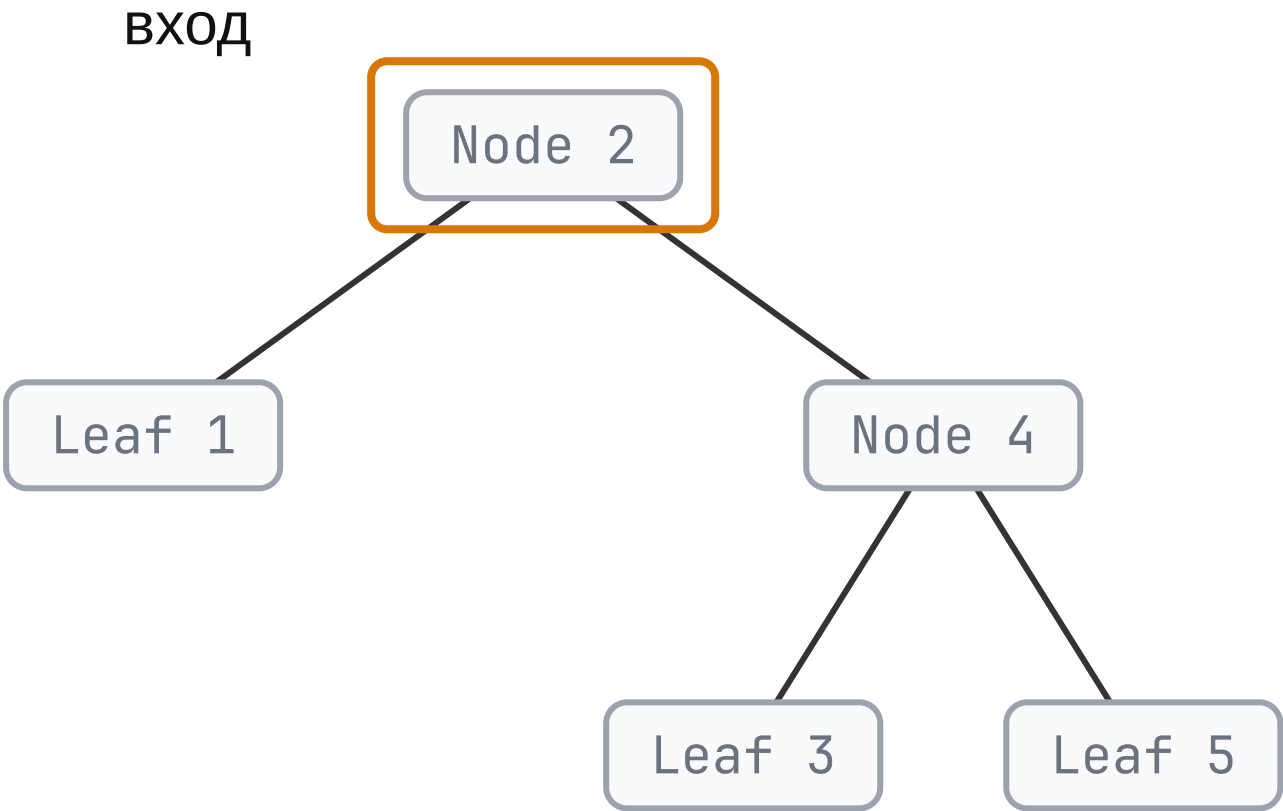
- Схема рекурсии прежняя; результаты теперь собираются обратно в дерево
- Высота не меняется: `height (transform f t) == height t`

transform: собираем новое дерево

Преобразуем (+ 10) каждое значение; исходное дерево слева остаётся прежним.

```
transform (+ 10)
(Node
  (Leaf 1)
  2
  (Node
    (Leaf 3)
    4
    (Leaf 5)))
```

ШАГИ РЕДУКЦИИ
transform: правило Node



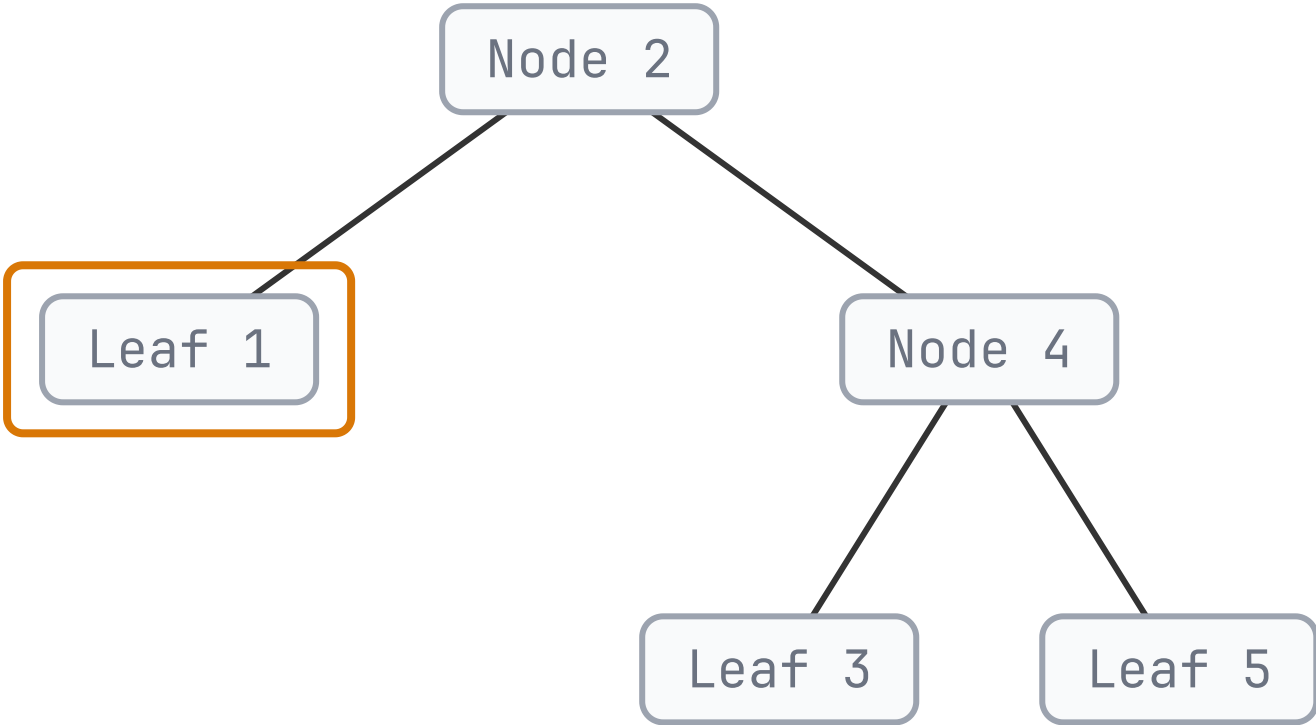
transform: собираем новое дерево

Node строится заново из двух преобразованных поддеревьев и нового значения.

Node

```
(transform (+ 10) (Leaf 1))  
((+ 10) 2)  
(transform (+ 10)  
  (Node  
    (Leaf 3)  
    4  
    (Leaf 5)))
```

ВХОД



ШАГИ РЕДУКЦИИ

transform: правило Node

НОВОЕ ДЕРЕВО

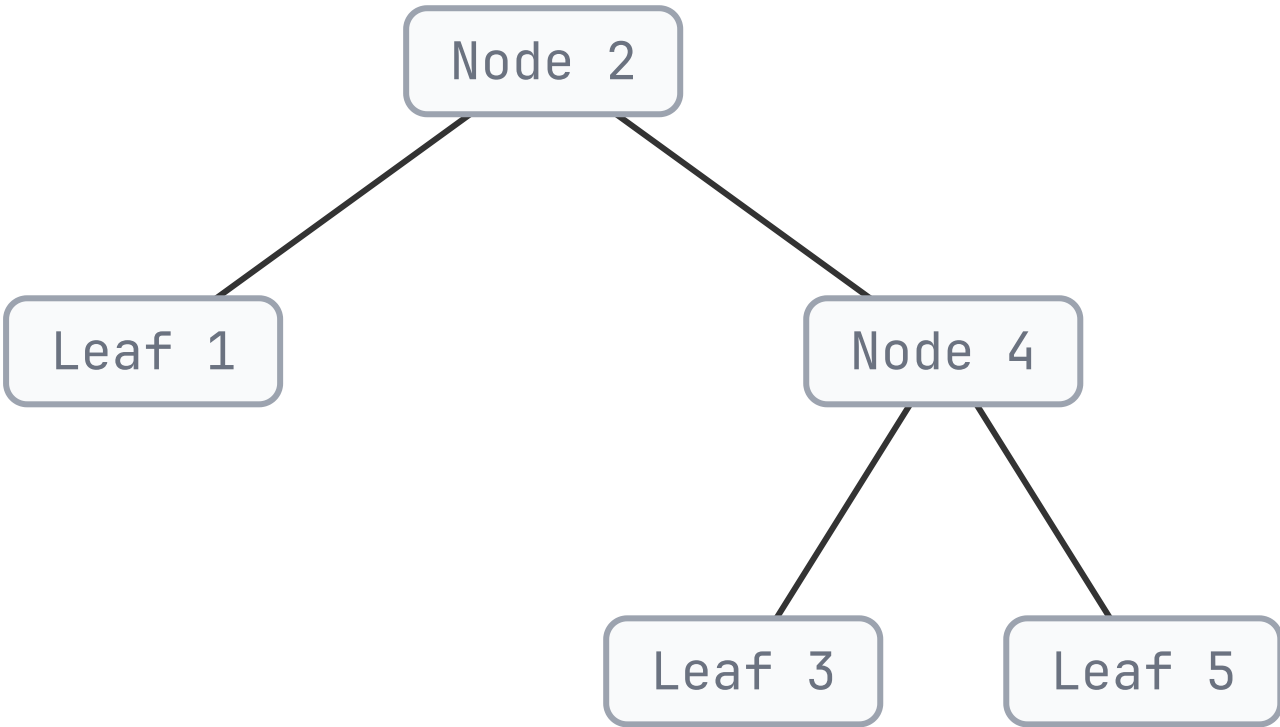


transform: собираем новое дерево

Для листа достаточно обернуть результат f x в Leaf.

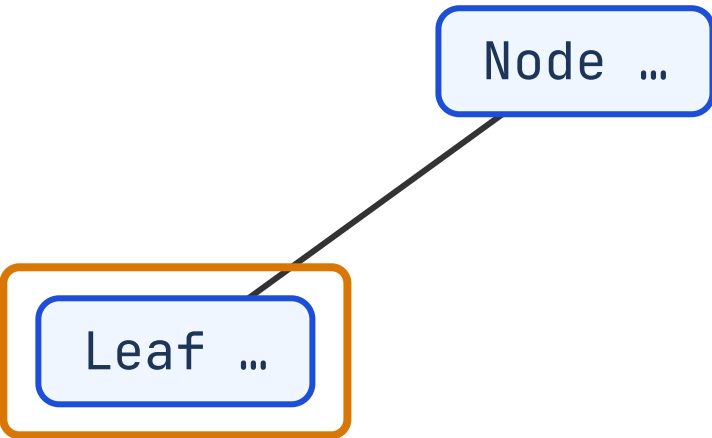
```
Node
  (Leaf ((+ 10) 1))
  ((+ 10) 2)
  (transform (+ 10)
    (Node
      (Leaf 3)
      4
      (Leaf 5)))
```

ВХОД



ШАГИ РЕДУКЦИИ
transform: правило Node

НОВОЕ ДЕРЕВО

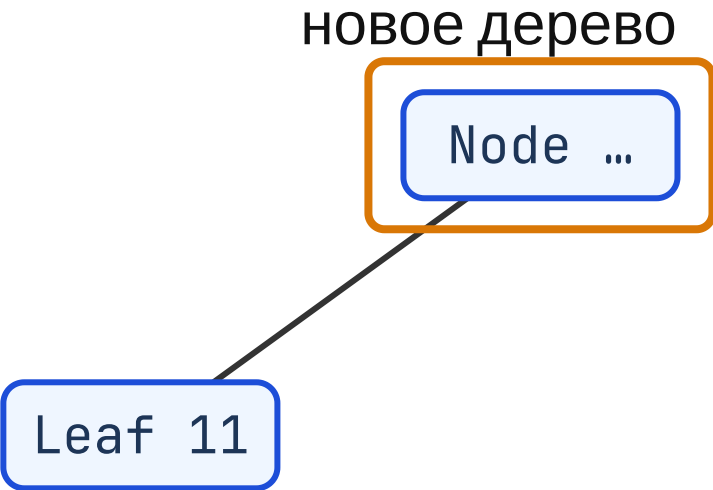
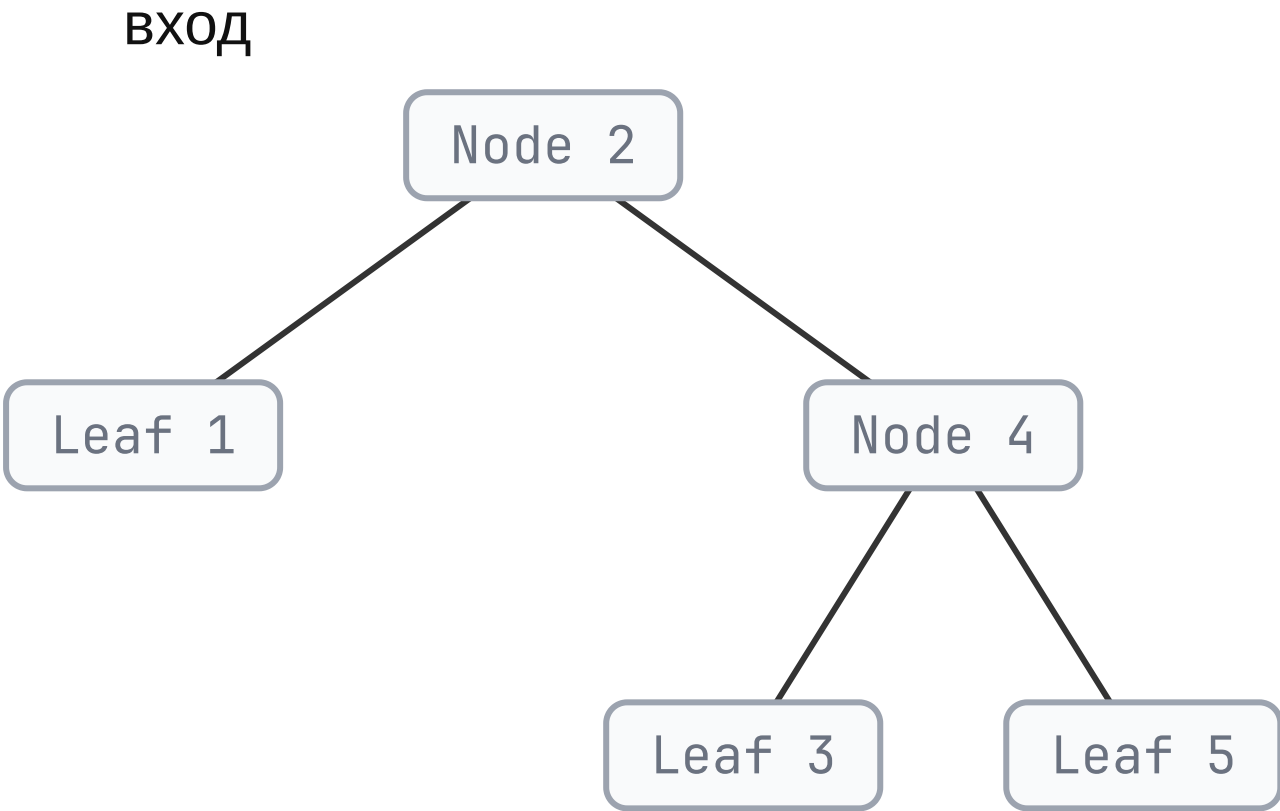


transform: собираем новое дерево

Теперь преобразуем значение в корне; левое поддерево уже готово.

```
Node
(Leaf 11)
((+ 10) 2)
(transform (+ 10)
  (Node
    (Leaf 3)
    4
    (Leaf 5)))
```

ШАГИ РЕДУКЦИИ
transform: правило Node

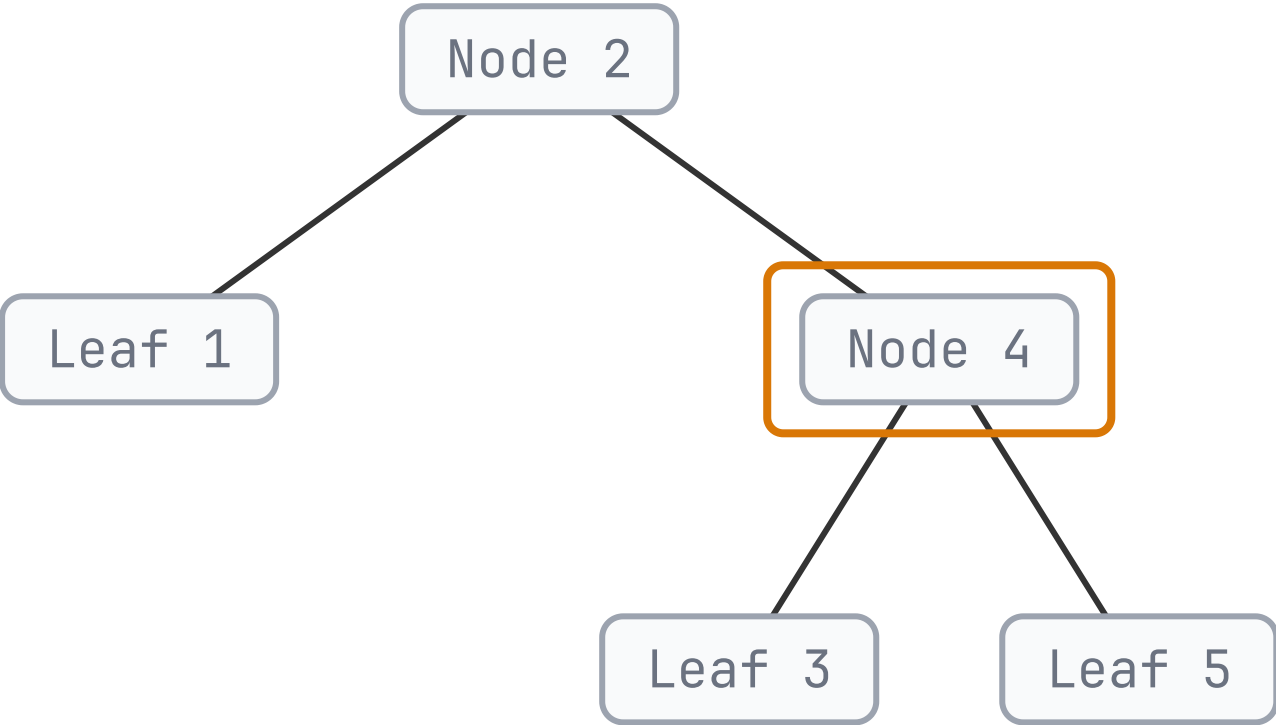


transform: собираем новое дерево

Переходим к правому поддереву; полный контекст Node остаётся видимым.

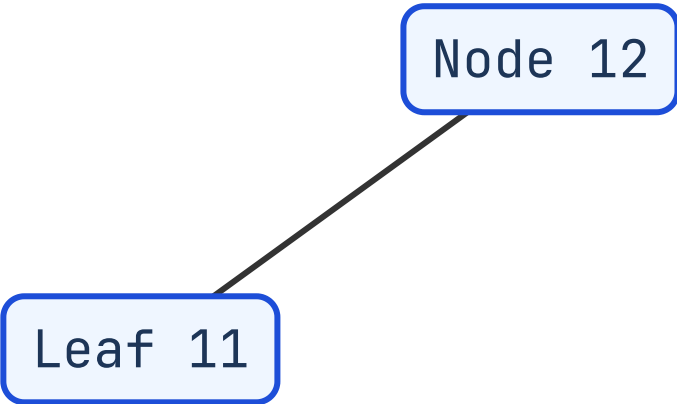
```
Node
(Leaf 11)
12
(transform (+ 10)
(Node
(Leaf 3)
4
(Leaf 5)))
```

ВХОД



ШАГИ РЕДУКЦИИ
transform: правило Node
transform: правило Node 4

НОВОЕ ДЕРЕВО

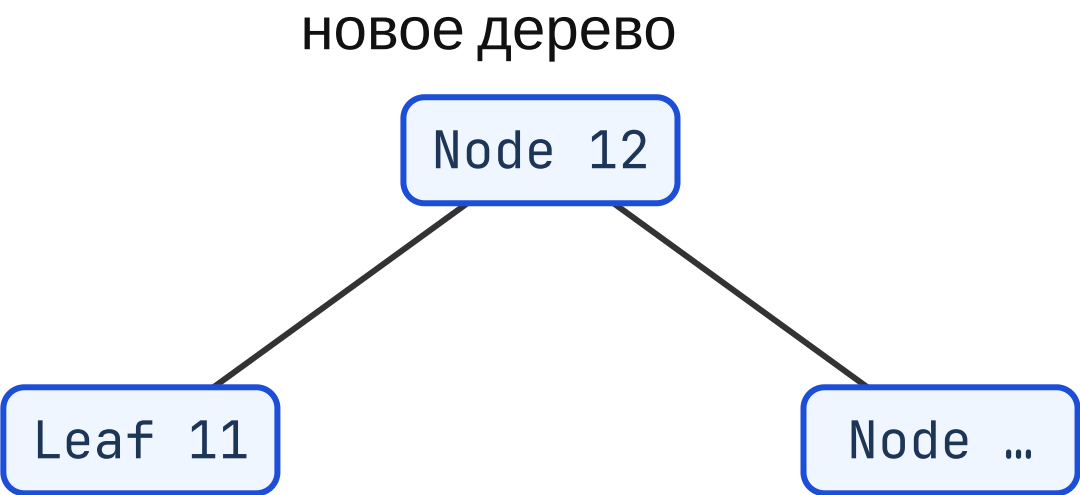
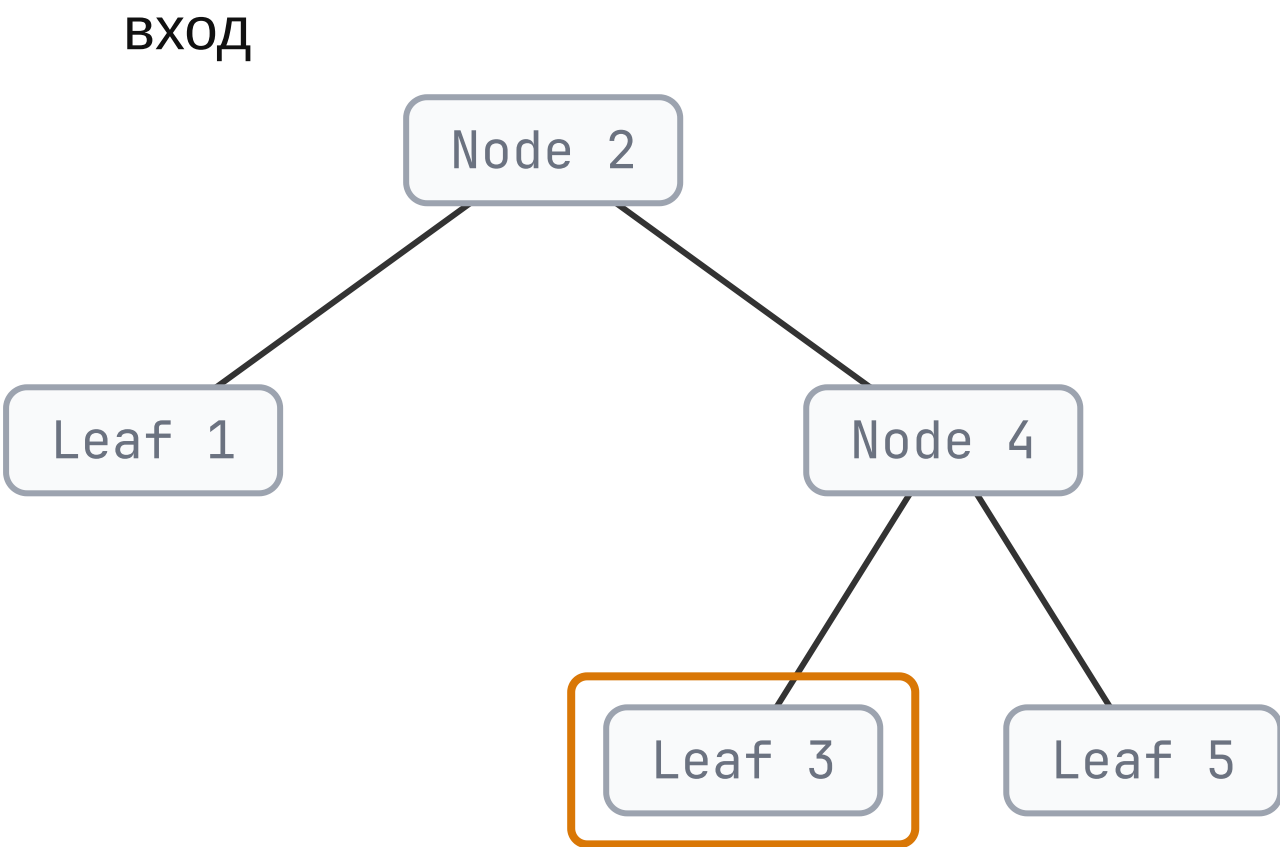


transform: собираем новое дерево

Правый Node собирается по той же схеме.

```
Node
(Leaf 11)
12
(Node
  (transform (+ 10) (Leaf 3))
  ((+ 10) 4)
  (transform (+ 10) (Leaf 5)))
```

- ШАГИ РЕДУКЦИИ
- transform: правило Node
- transform: правило Node 4

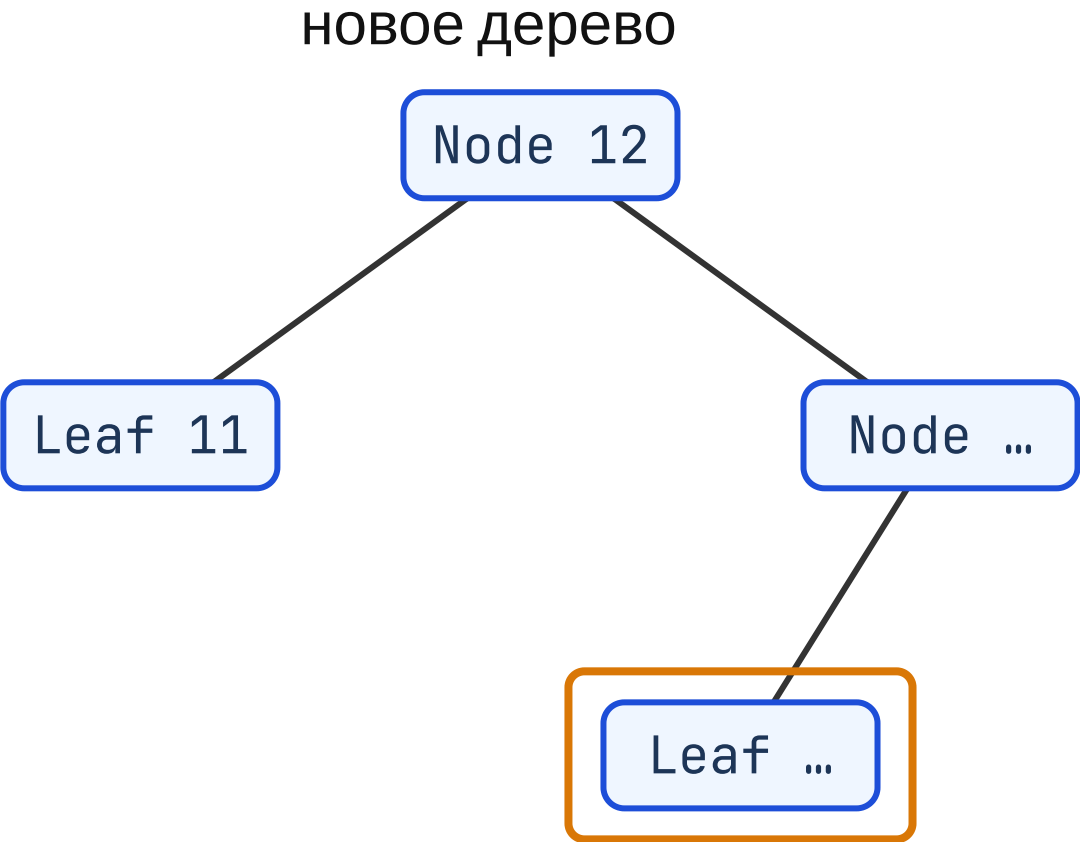
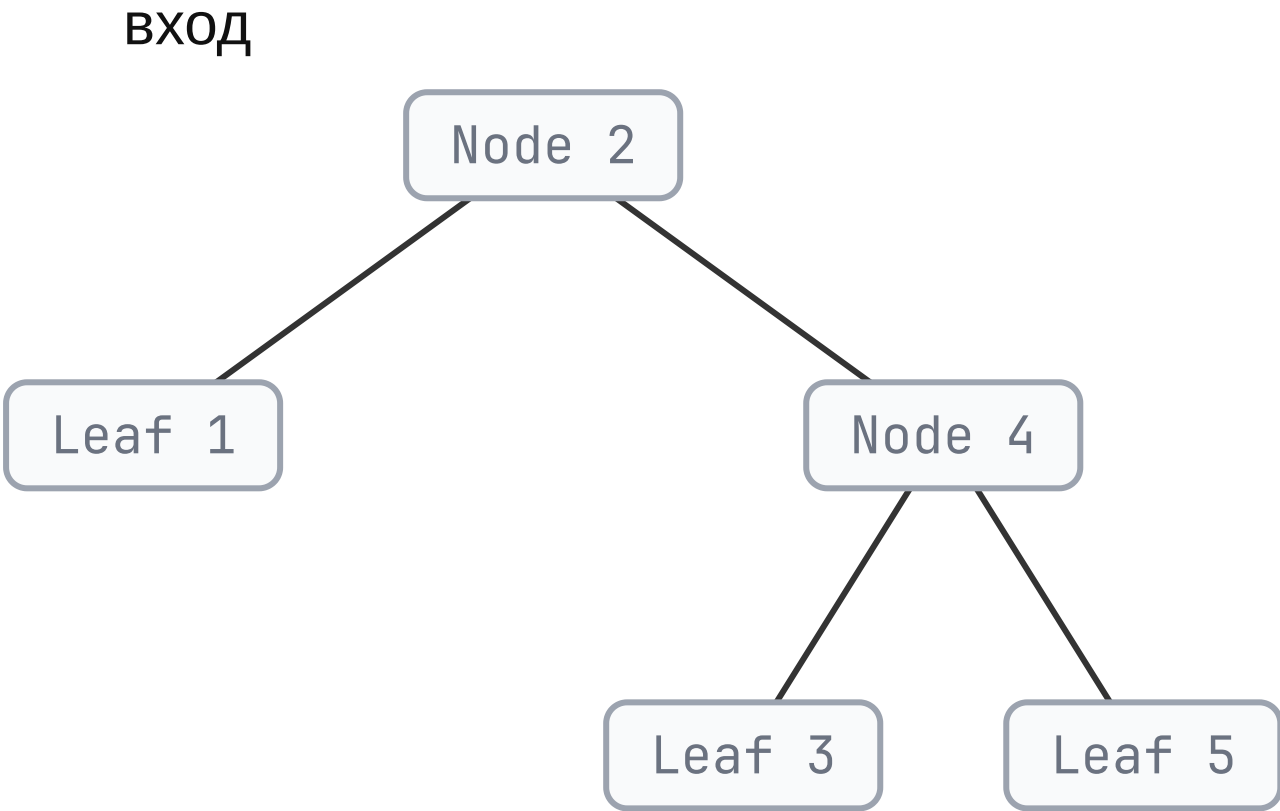


transform: собираем новое дерево

Лист 3 превращается в Leaf 13.

```
Node
(Leaf 11)
12
(Node
  (Leaf ((+ 10) 3))
  ((+ 10) 4)
  (transform (+ 10) (Leaf 5)))
```

ШАГИ РЕДУКЦИИ
transform: правило Node
transform: правило Node 4

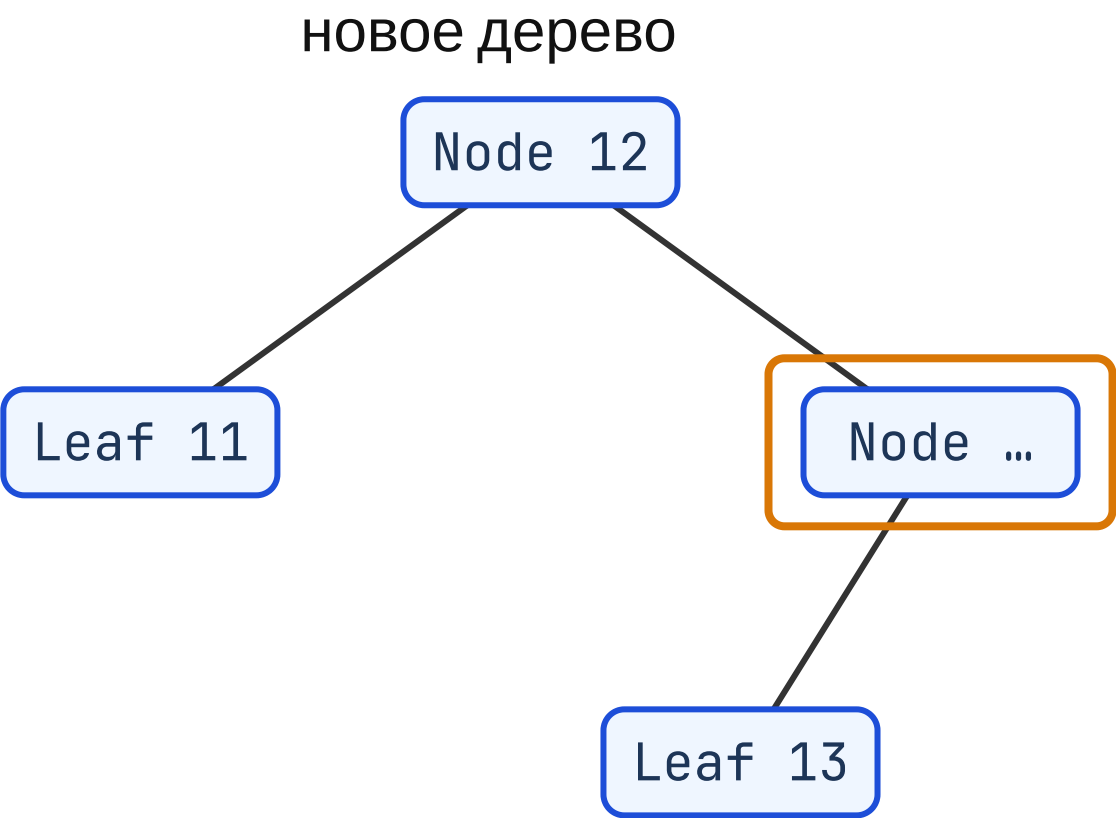
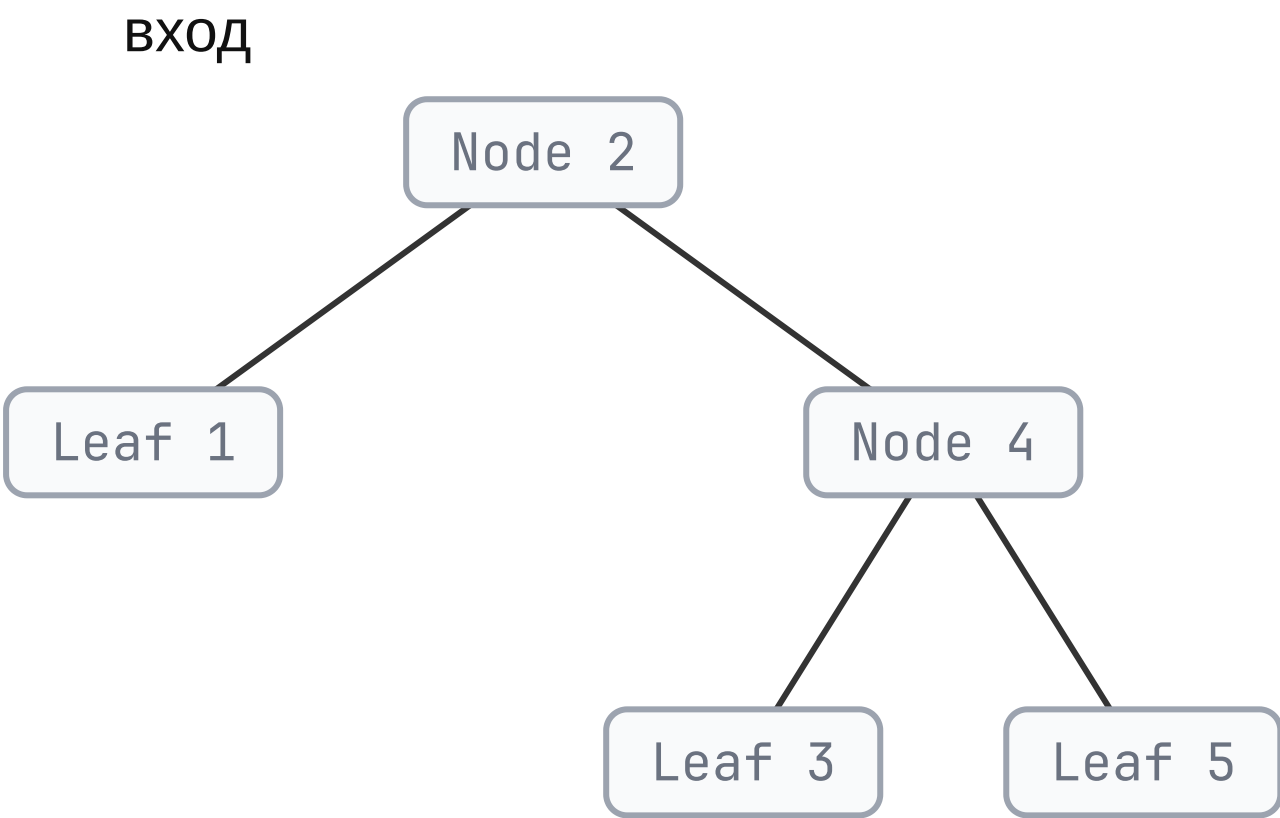


transform: собираем новое дерево

Для внутреннего значения 4 получаем 14.

```
Node
(Leaf 11)
12
(Node
  (Leaf 13)
  ((+ 10) 4)
  (transform (+ 10) (Leaf 5)))
```

- ШАГИ РЕДУКЦИИ
- transform: правило Node
 - transform: правило Node 4

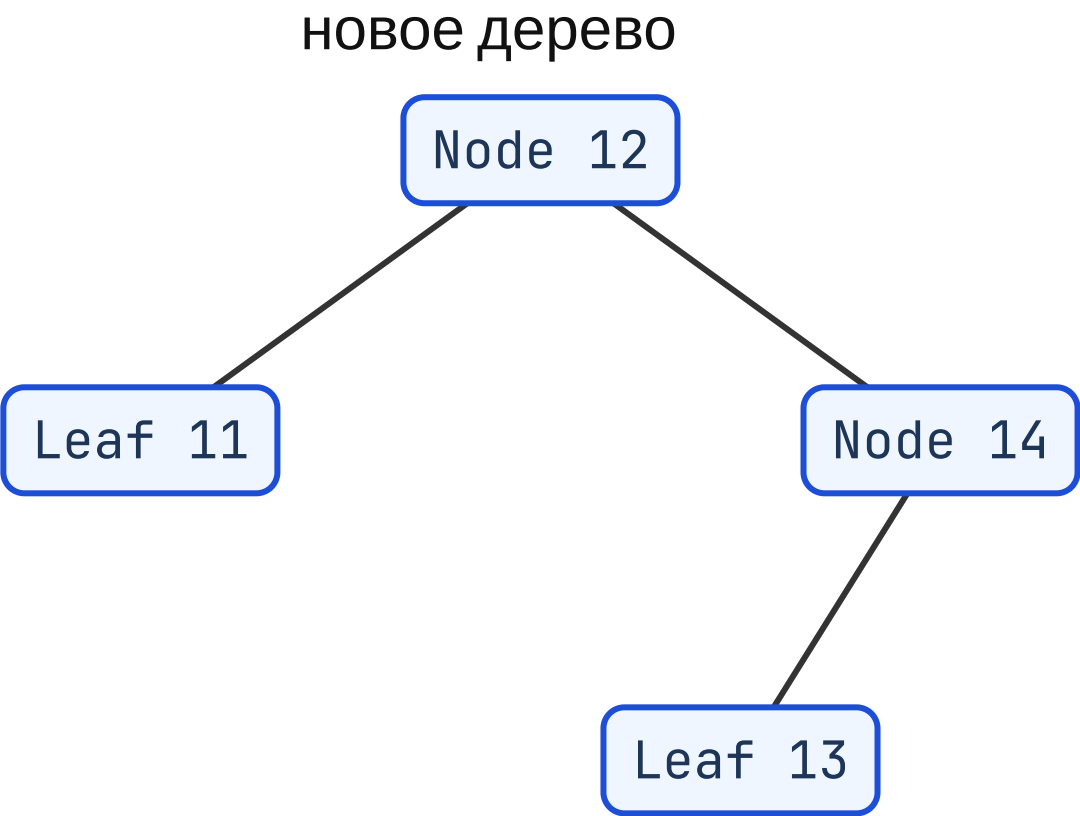
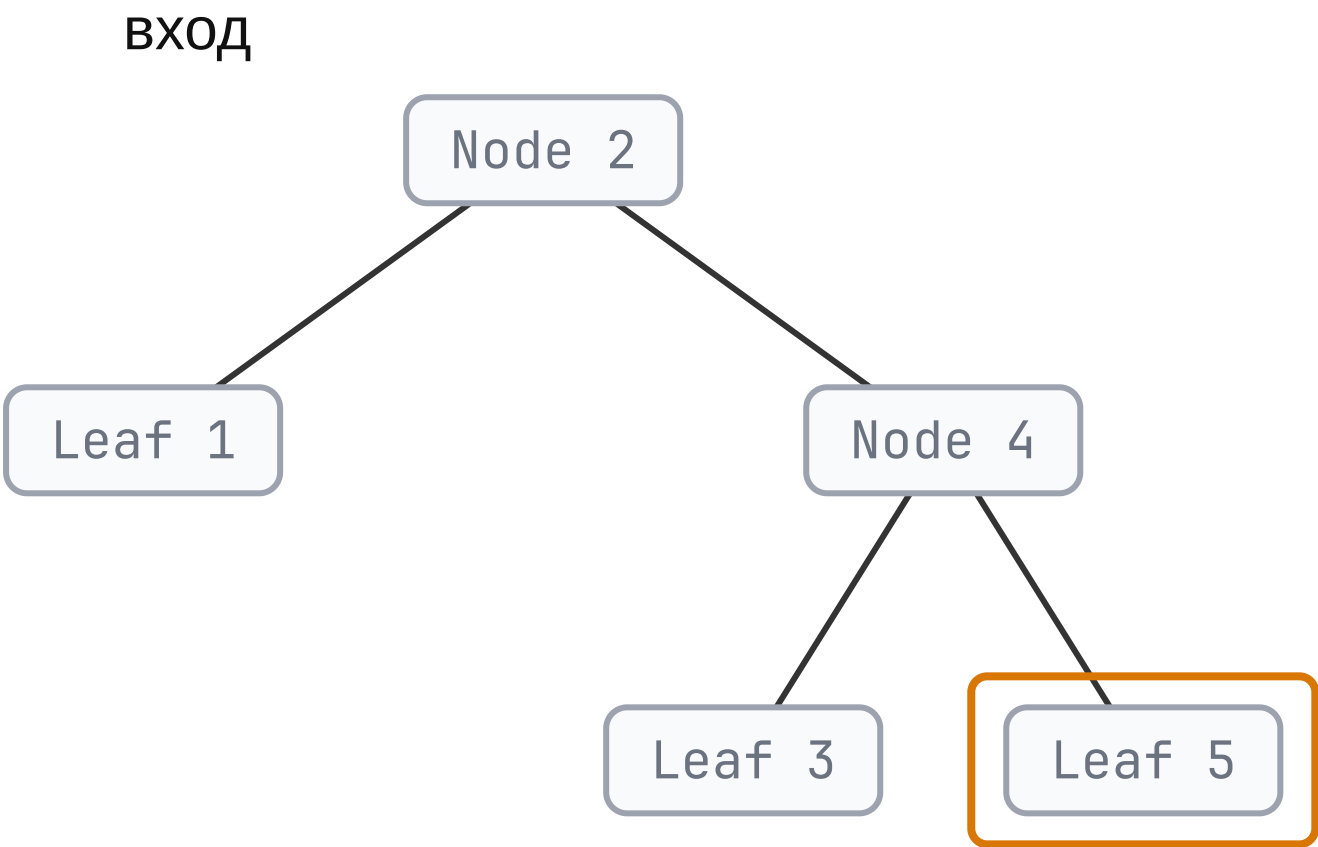


transform: собираем новое дерево

Последнее поддереву — Leaf 5.

```
Node
  (Leaf 11)
  12
  (Node
    (Leaf 13)
    14
    (transform (+ 10) (Leaf 5)))
```

- ШАГИ РЕДУКЦИИ
- transform: правило Node
 - transform: правило Node 4
 - transform: правило Leaf

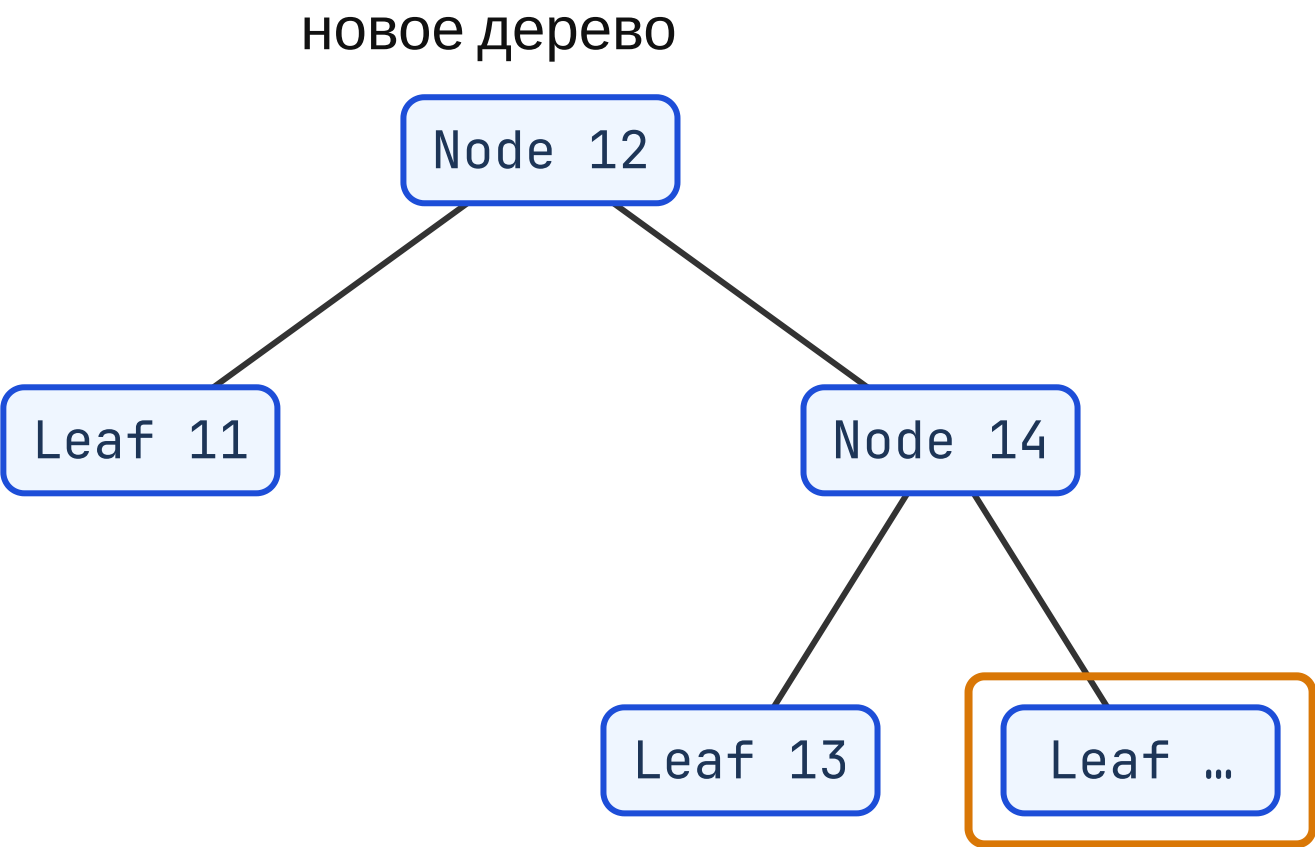
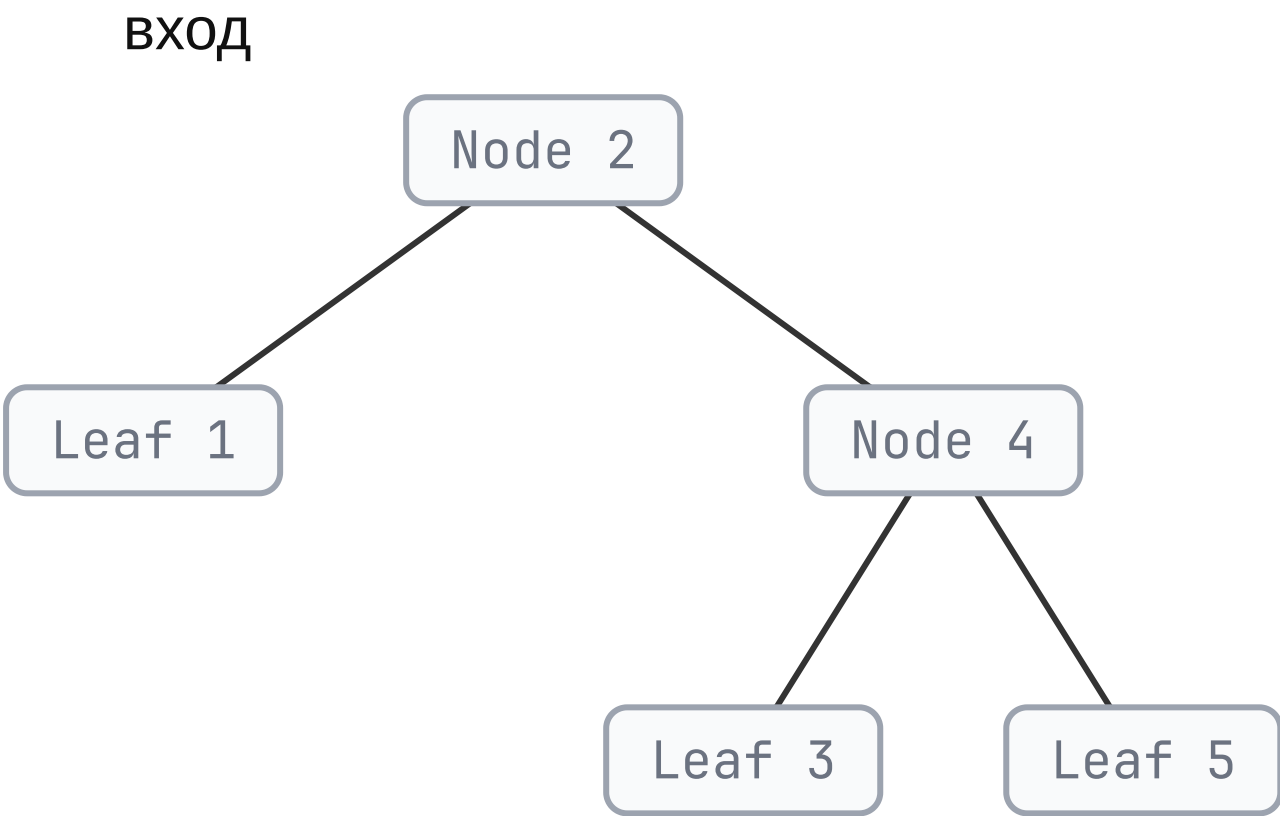


transform: собираем новое дерево

Осталось применить f к 5 и собрать последний Leaf.

```
Node
(Leaf 11)
12
(Node
(Leaf 13)
14
(Leaf ((+ 10) 5)))
```

- ШАГИ РЕДУКЦИИ
- transform: правило Node
 - transform: правило Node 4
 - transform: правило Leaf

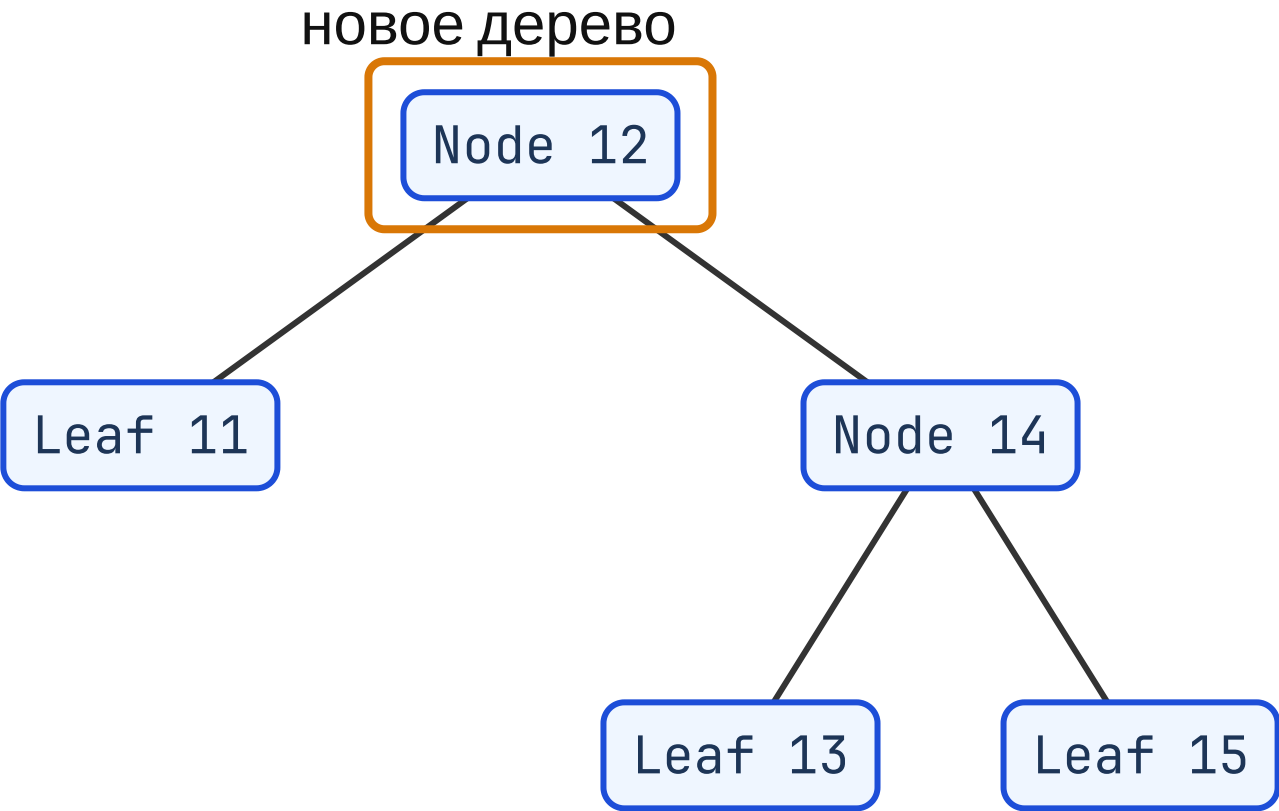
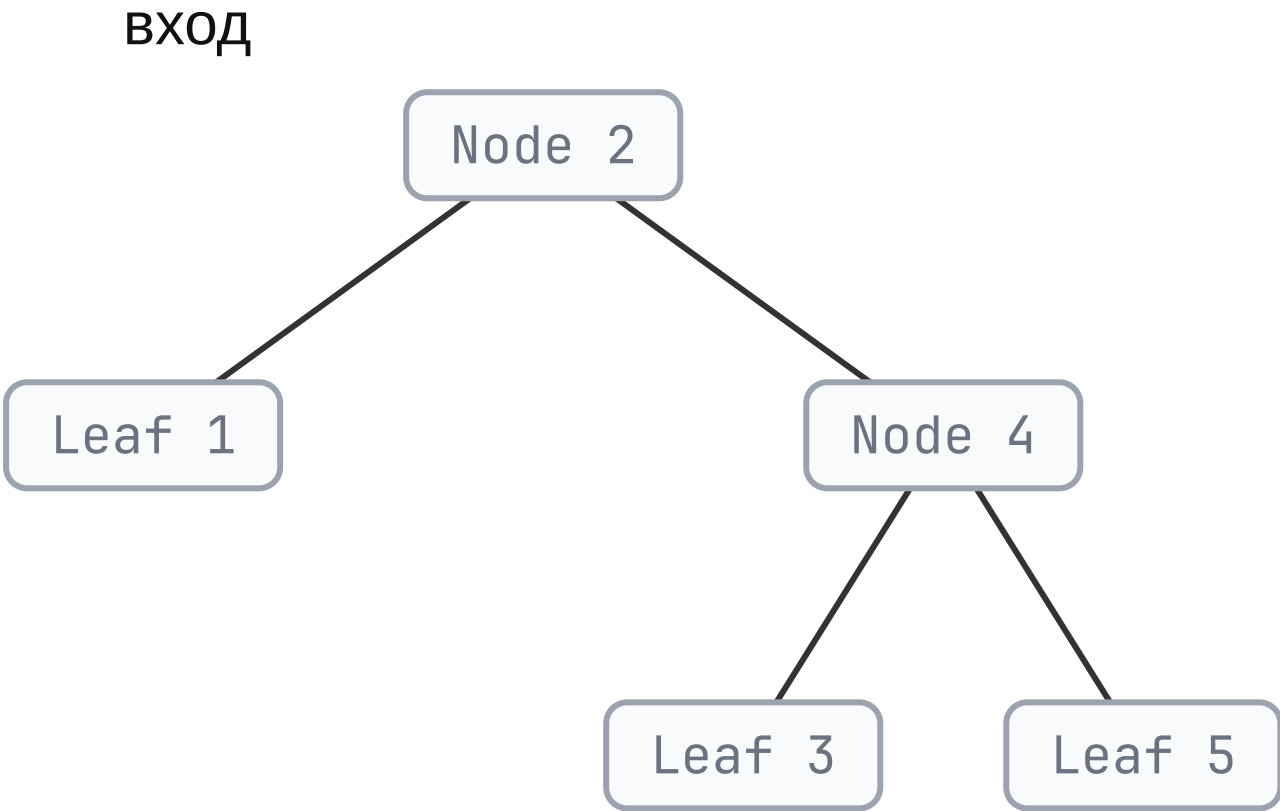


transform: собираем новое дерево

Новое дерево имеет прежнюю форму и значения 11–15.

```
Node
(Leaf 11)
12
(Node
(Leaf 13)
14
(Leaf 15))
```

- ШАГИ РЕДУКЦИИ
- transform: правило Node
 - transform: правило Node 4
 - transform: правило Leaf
 - форма сохранена



Кайнды

Терм, тип, кайнд

Терм `Just True` — выражение, которое можно вычислить.

Just True

терм

Терм, тип, кайнд

У терма `Just True` есть тип `Maybe Bool`.

`Just True`

терм

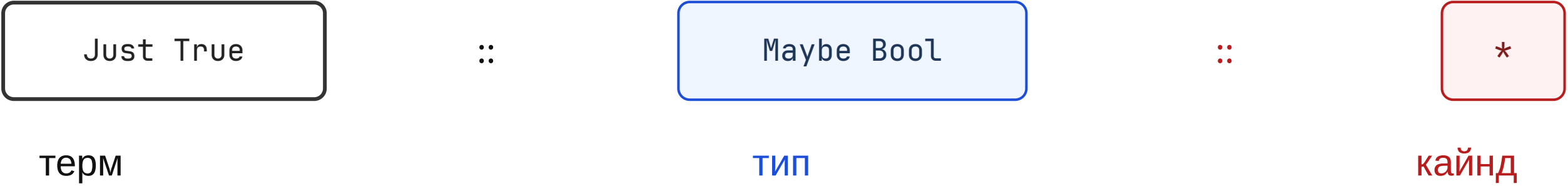
::

`Maybe Bool`

тип

Терм, тип, кайнд

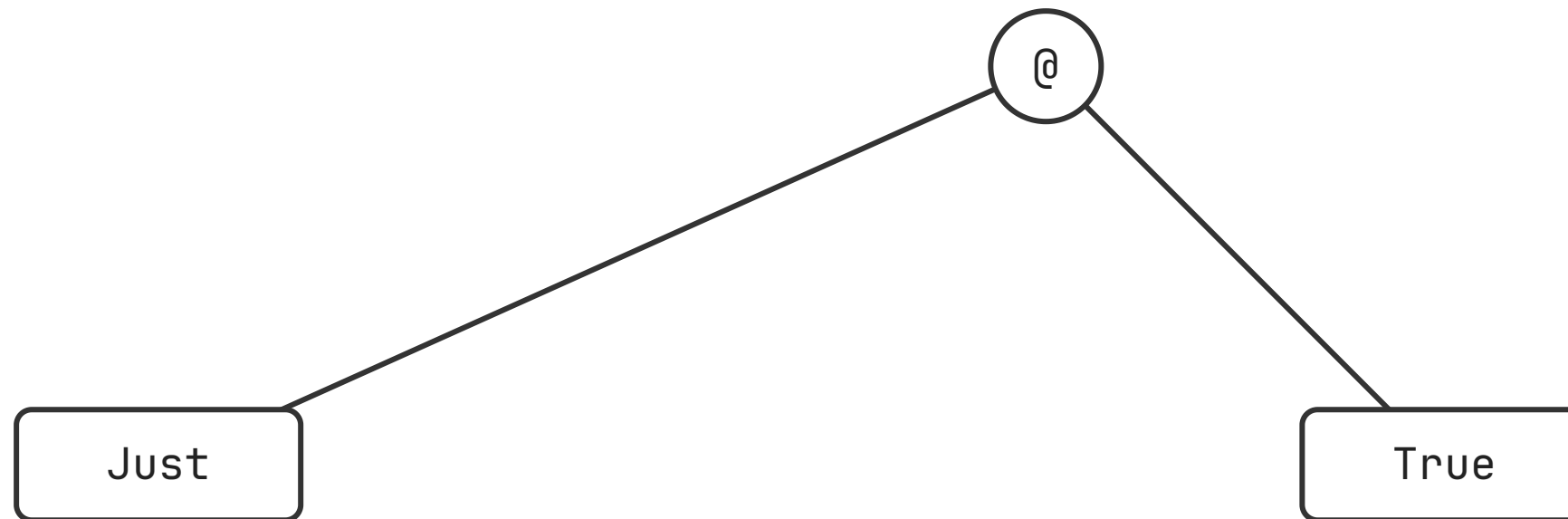
У типа `Maybe Bool` есть кайнд `*`.



Кайнды: типы для типов

Знакомое дерево терма: применение *Just* к *True*.

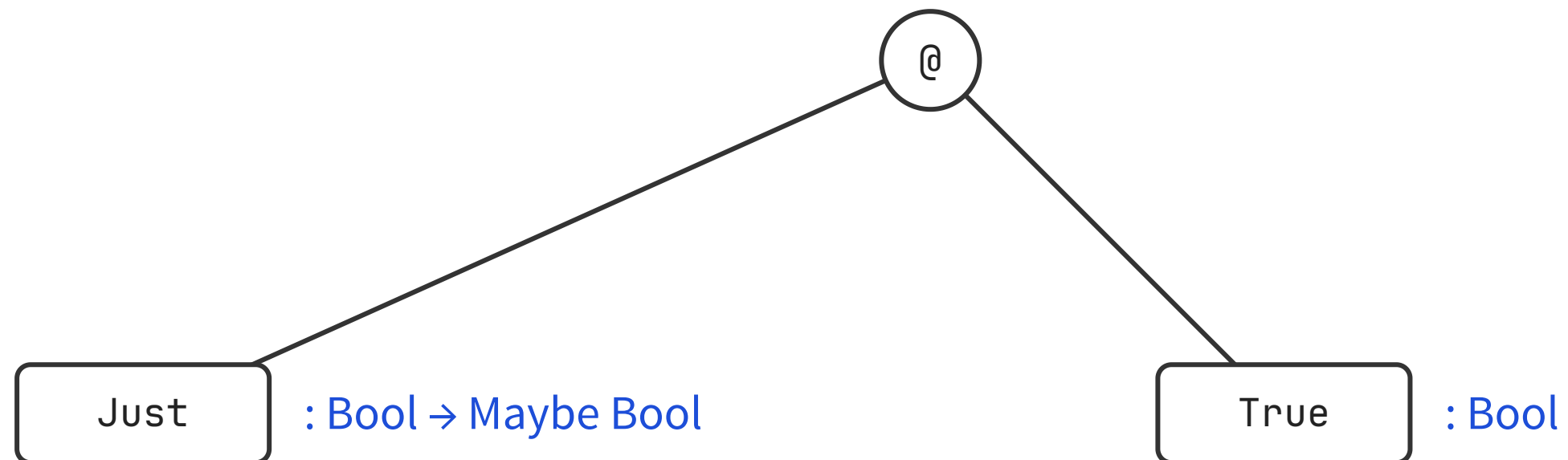
Just True



Кайнды: типы для типов

Припишем листьям типы. В этом применении у `Just` параметр типа — `Bool`.

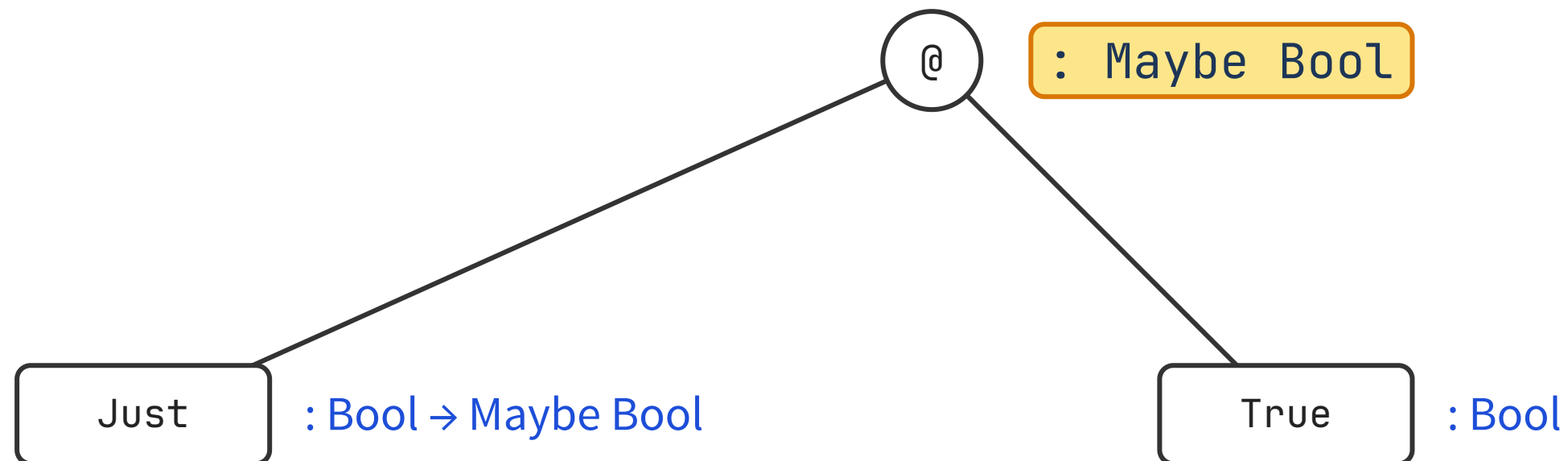
`Just True`



Кайнды: типы для типов

Аргумент имеет нужный тип `Bool`; всё применение получает тип `Maybe Bool`.

Just True

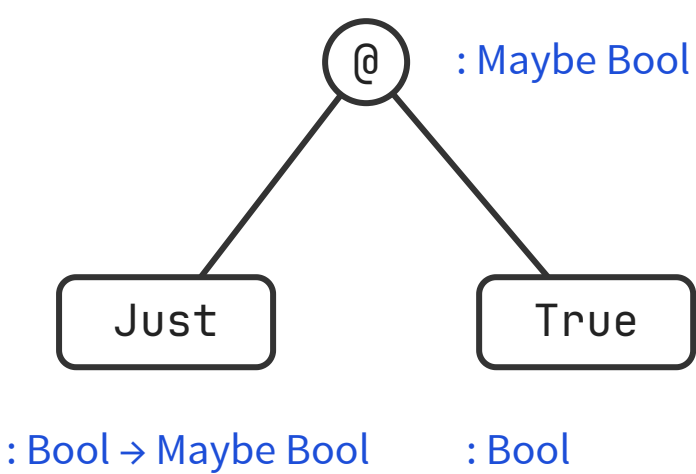


Кайнды: типы для типов

Приблизим `Maybe Bool`: тип тоже записан применением.

`Maybe Bool`

дерево терма



приближаем выбранный тип

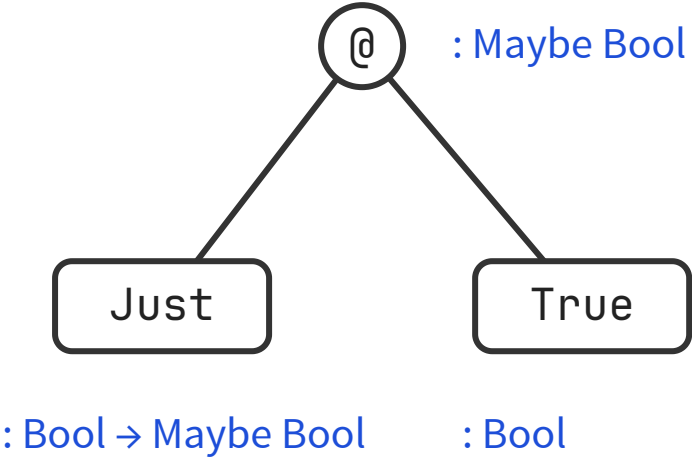
`Maybe Bool`

Кайнды: типы для типов

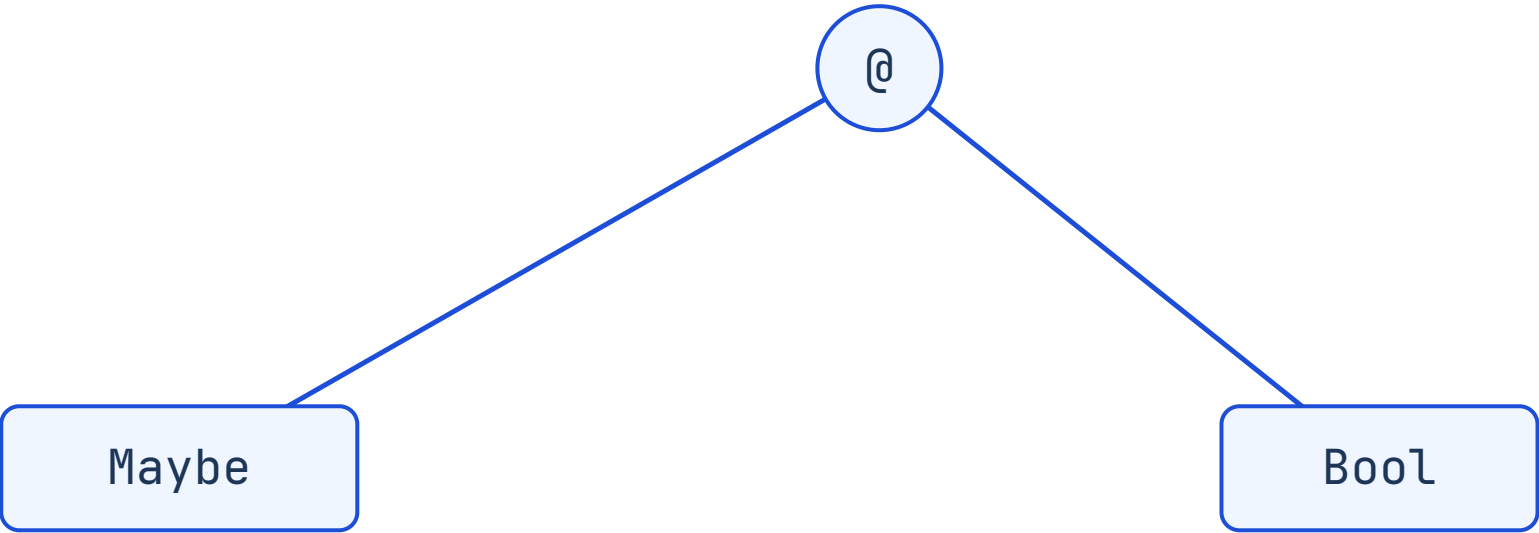
У дерева типа та же форма: применение *Maybe* к *Bool*.

Maybe Bool

дерево терма



дерево типа *Maybe Bool*

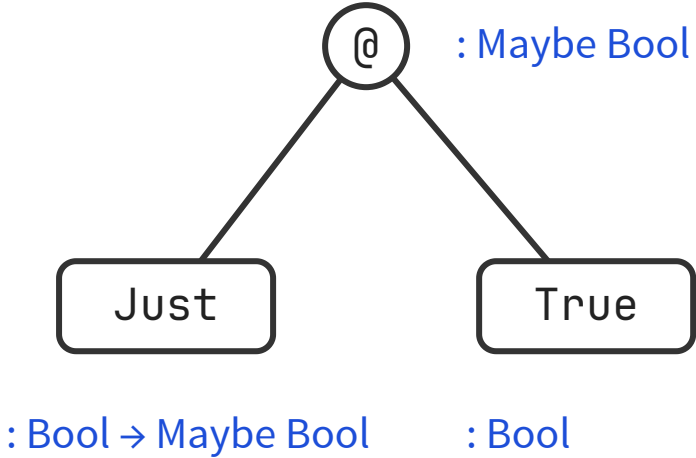


Кайнды: типы для типов

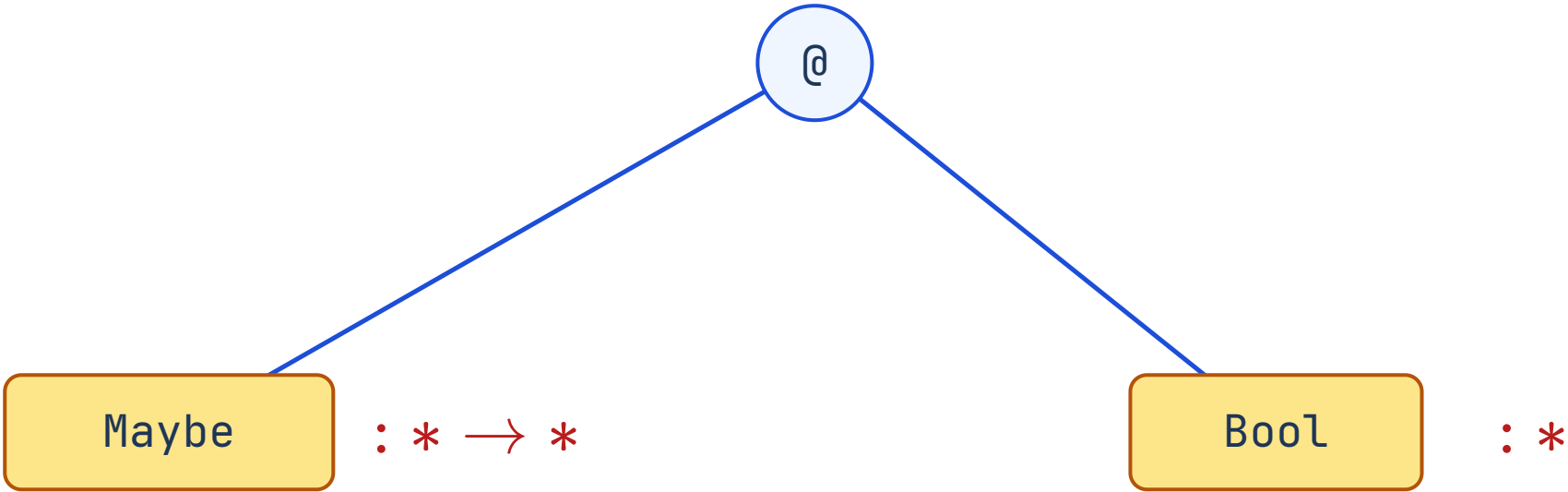
Припишем листьям кайнды: $Maybe :: * \rightarrow *$, $Bool :: *$.

Maybe Bool

дерево терма



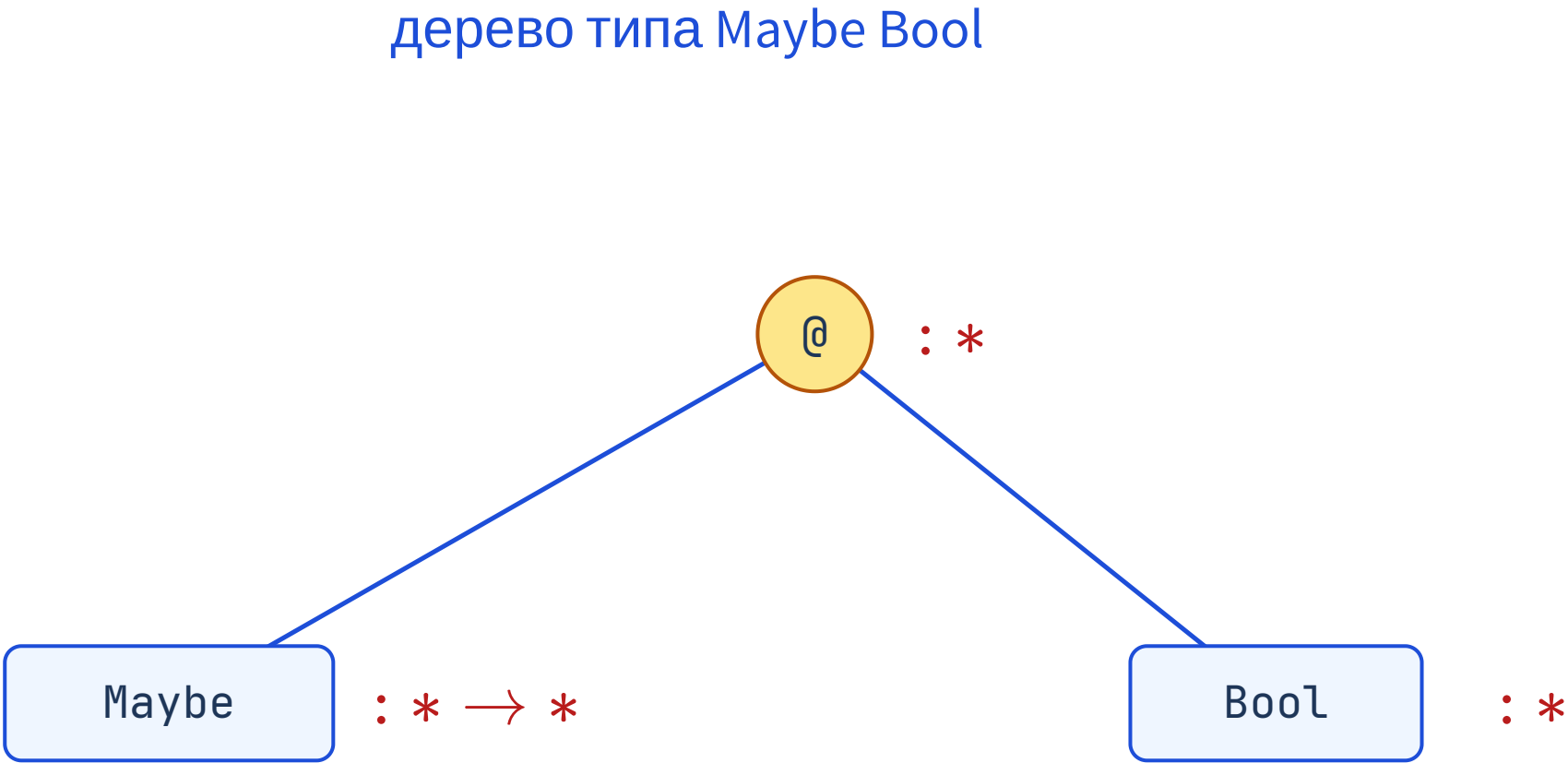
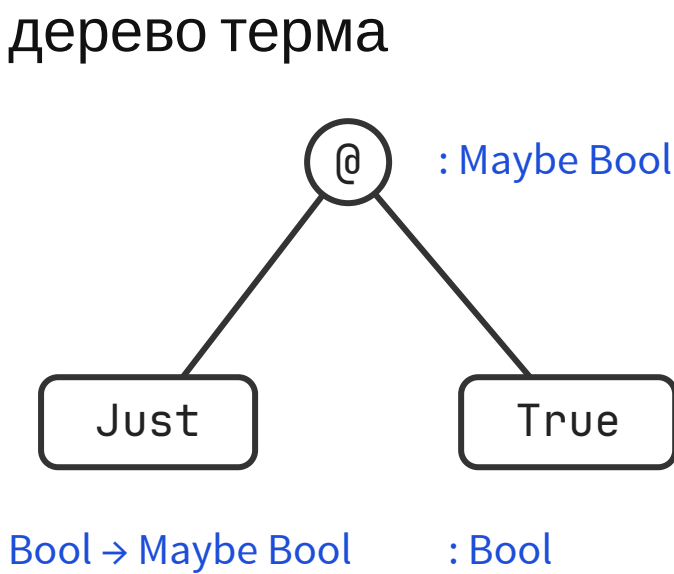
дерево типа Maybe Bool



Кайнды: типы для типов

Правило применения то же: аргумент подходит, корень получает метку результата.

Maybe Bool :: *



Терм: `Bool → Maybe Bool` и аргумент `Bool` дают тип `Maybe Bool`.

Тип: `* → *` и аргумент `*` дают кайнд `*`. Кайнды — типы для типов.

Предскажите кайнд

✎ **Устно:** Определите кайнды `Tree`, `Tree Bool`, `Either String` и `Either String Int`.

⚡ Решение.

```
ghci> :kind Tree
Tree :: * -> *
ghci> :kind Tree Bool
Tree Bool :: *
ghci> :kind Either String
Either String :: * -> *
ghci> :kind Either String Int
Either String Int :: *
```

- `:t` спрашивает тип **терма**; `:k` спрашивает кайнд **типа**
- `Node` — конструктор данных; `Tree` — конструктор типа

Списки

Список: две ветки, одно поддерево

```
-- Схематически: встроенный список устроен так
-- data List a = Nil | Cons a (List a)
[]          -- пустой список
x : xs     -- голова и хвост
```

 Напишите безопасную функцию взятия головы списка: на пустом списке возвращайте **Nothing**.

```
safeHead :: [a] -> Maybe a
safeHead xs = case xs of
  [] -> -----
  x : _ -> -----
```

 **Решение.**

```
[] -> Nothing
x : _ -> Just x
```

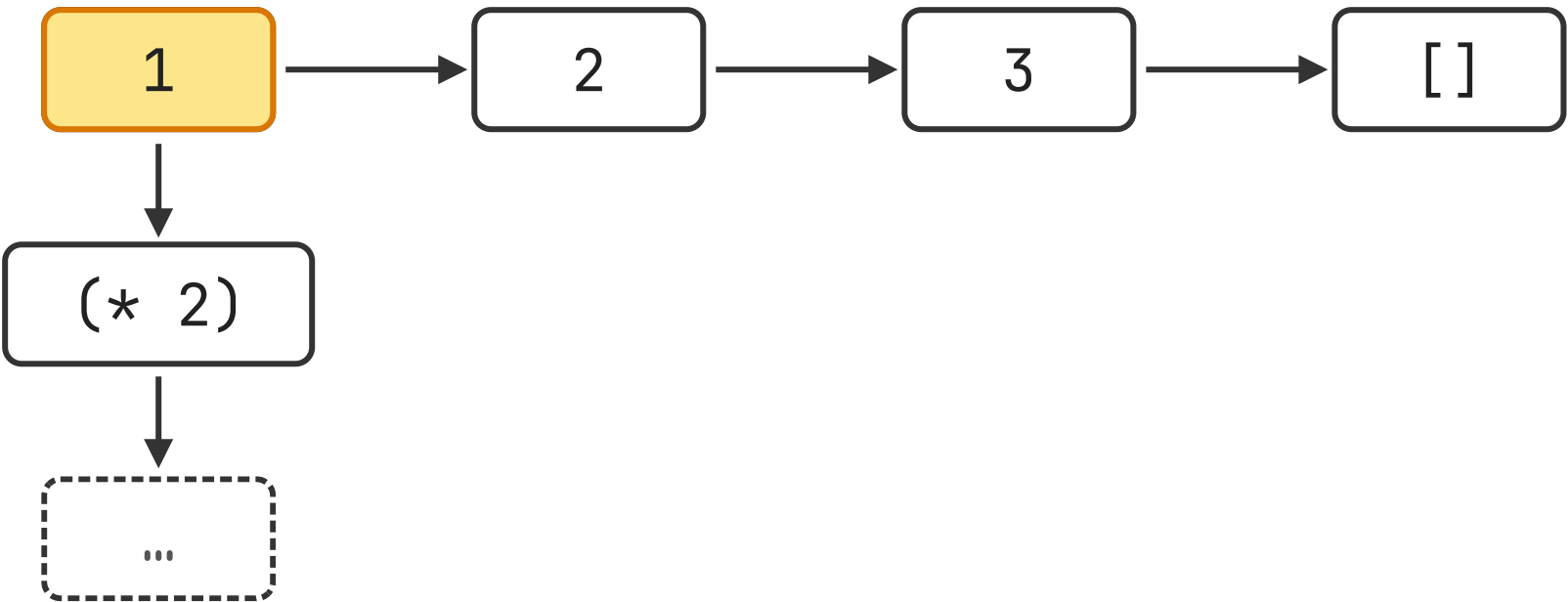
map: меняем каждый элемент

map применяет функцию к каждому элементу. Предскажите map (* 2) [1,2,3].

map (* 2) [1,2,3]

ШАГИ
map: преобразуем элементы

ВХОД



результат

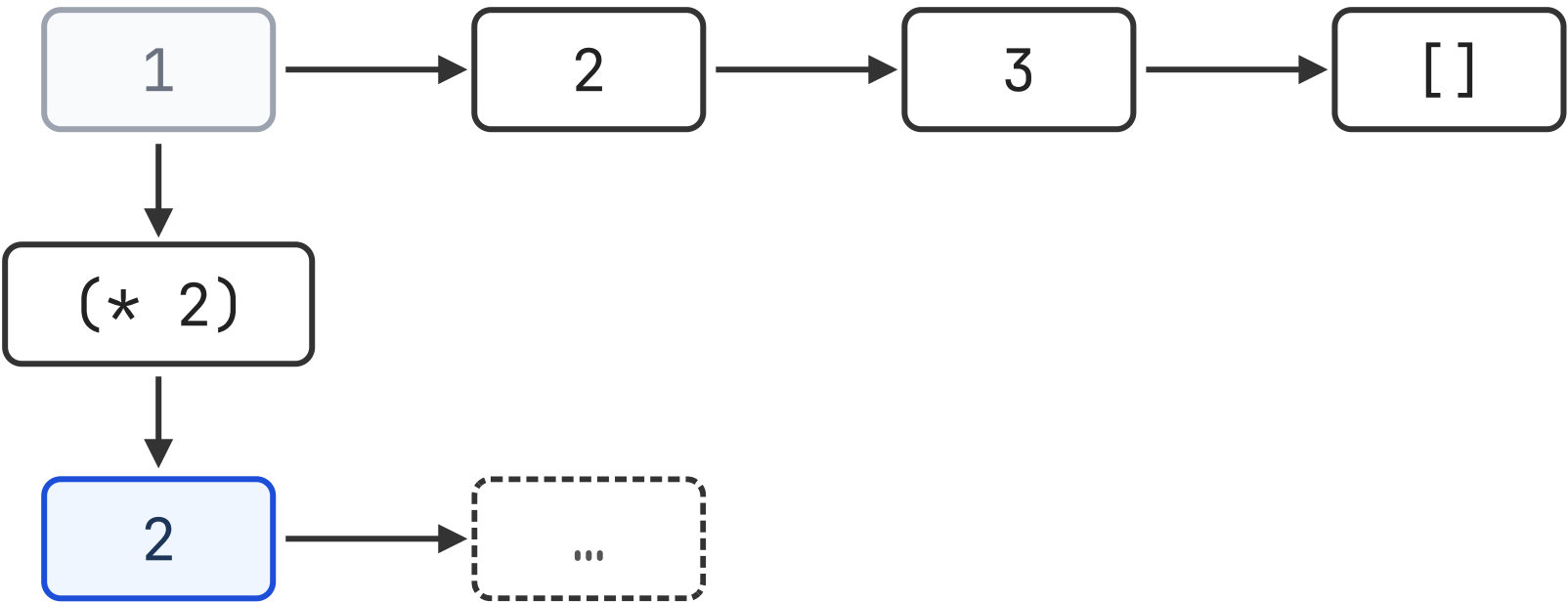
map: меняем каждый элемент

Применяем $(\ast\ 2)$ к голове; результат становится новой головой.

2 : map $(\ast\ 2)$ [2,3]

ШАГИ
map: преобразуем элементы
 $(\ast\ 2)\ 1 \rightsquigarrow 2$

ВХОД



результат

map: меняем каждый элемент

Рекурсивный вызов получает хвост [2,3]. Следующая голова — 2.

```
2 : map (* 2) [2,3]
```

ШАГИ
map: преобразуем элементы
(* 2) 1 $\rightsquigarrow$ 2

ВХОД



результат



map: меняем каждый элемент

Для 2 получаем 4. Порядок элементов сохраняется.

2 : 4 : map (* 2) [3]

ШАГИ
map: преобразуем элементы
(* 2) 1 $\rightsquigarrow$ 2
(* 2) 2 $\rightsquigarrow$ 4

ВХОД



результат



map: меняем каждый элемент

В оставшемся хвосте [3] ещё один элемент.

```
2 : 4 : map (* 2) [3]
```

ШАГИ
map: преобразуем элементы
(* 2) 1 $\rightsquigarrow$ 2
(* 2) 2 $\rightsquigarrow$ 4

ВХОД



результат



map: меняем каждый элемент

Получили 6, но хвост результата ещё задаётся вызовом на [].

```
2 : 4 : 6 : map (* 2) []
```

- ШАГИ
- map: преобразуем элементы
- (* 2) 1 $\rightsquigarrow$ 2
- (* 2) 2 $\rightsquigarrow$ 4
- (* 2) 3 $\rightsquigarrow$ 6

ВХОД



результат



map: меняем каждый элемент

`map f [] = []`: пустой вход даёт пустой хвост результата.

[2,4,6]

ВХОД



ШАГИ
map: преобразуем элементы
(`* 2`) `1` $\rightsquigarrow$ `2`
(`* 2`) `2` $\rightsquigarrow$ `4`
(`* 2`) `3` $\rightsquigarrow$ `6`
`map f []` $\rightsquigarrow$ `[]`

результат



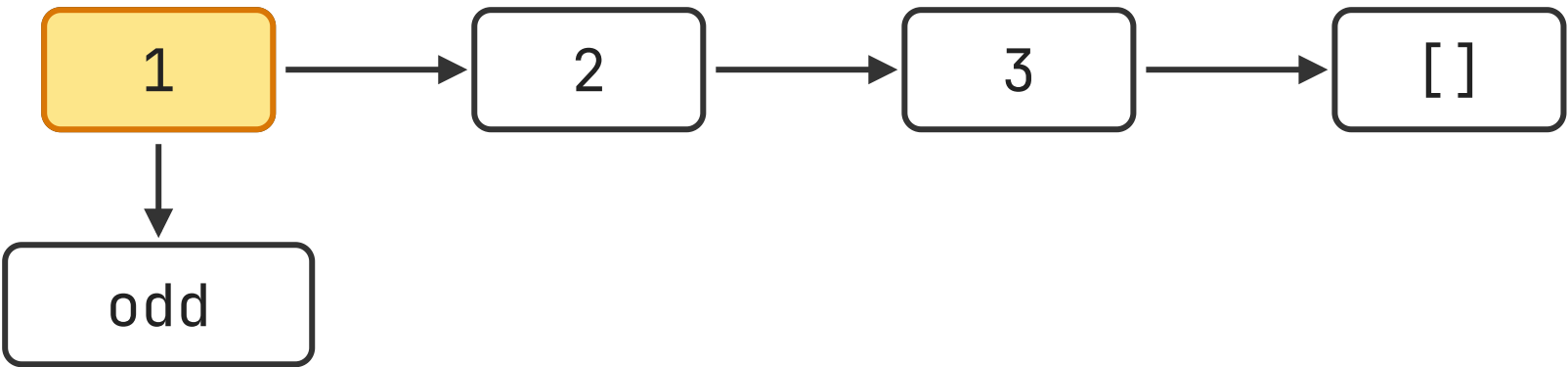
filter: оставляем подходящие

`filter` проверяет каждый элемент. Предскажите `filter odd [1,2,3]`.

`filter odd [1,2,3]`

ШАГИ
`filter: проверяем odd`

ВХОД



результат



filter: оставляем подходящие

odd 1 = True: сохраняем 1 и продолжаем с хвостом.

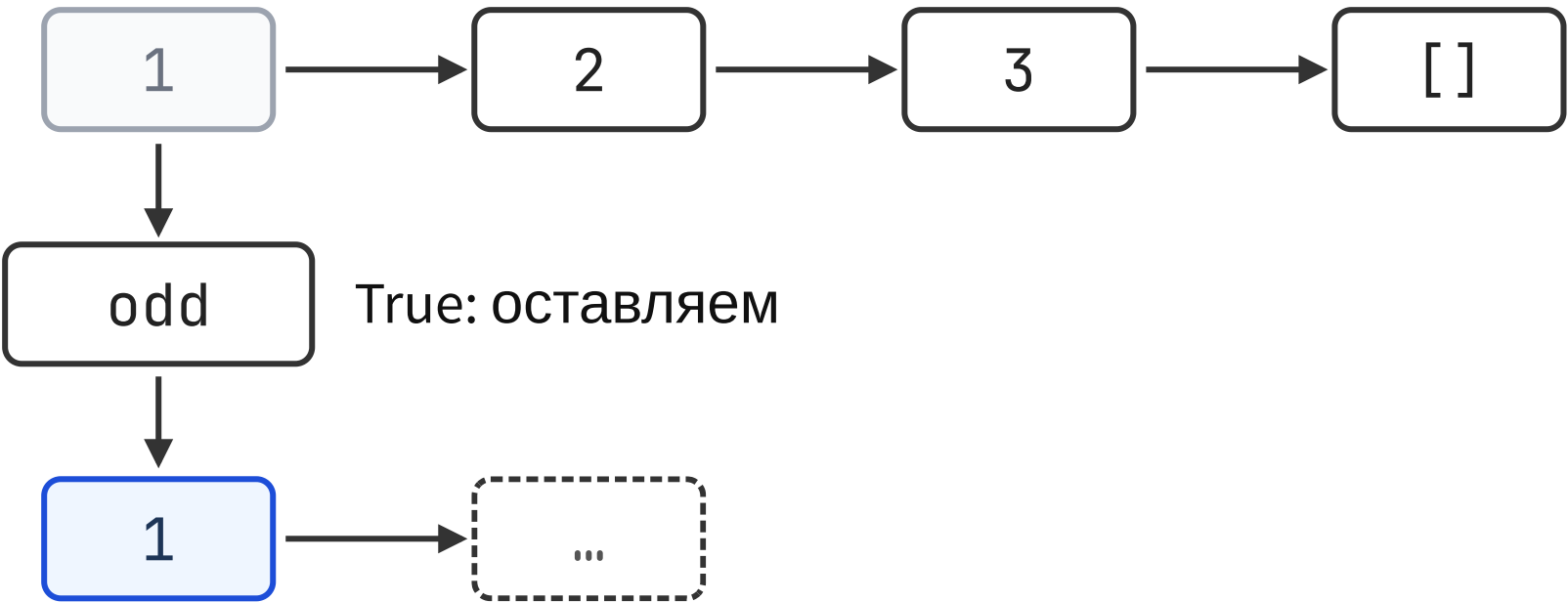
```
1 : filter odd [2,3]
```

ШАГИ

filter: проверяем odd

odd 1 ➡ True

ВХОД



результат

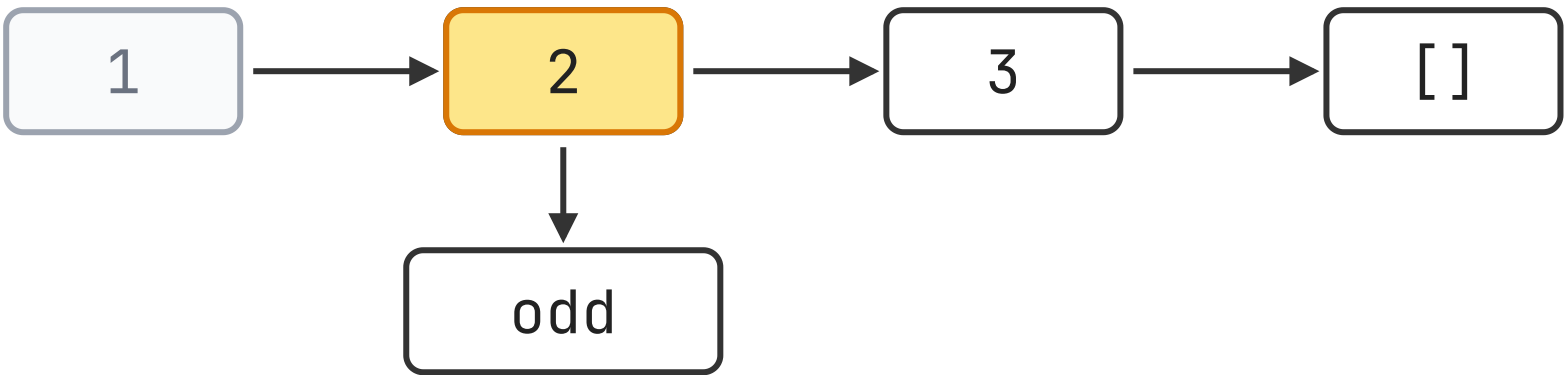
filter: оставляем подходящие

Следующая голова — 2. Проверка может дать другой результат.

```
1 : filter odd [2,3]
```

ШАГИ
filter: проверяем odd
odd 1 $\rightsquigarrow$ True

ВХОД



результат



filter: оставляем подходящие

`odd 2 = False`: пропускаем 2; новой ячейки результата нет.

```
1 : filter odd [3]
```

ШАГИ

filter: проверяем odd

odd 1 $\rightsquigarrow$ True

odd 2 $\rightsquigarrow$ False

ВХОД



результат



НЕ ВКЛЮЧАЕМ

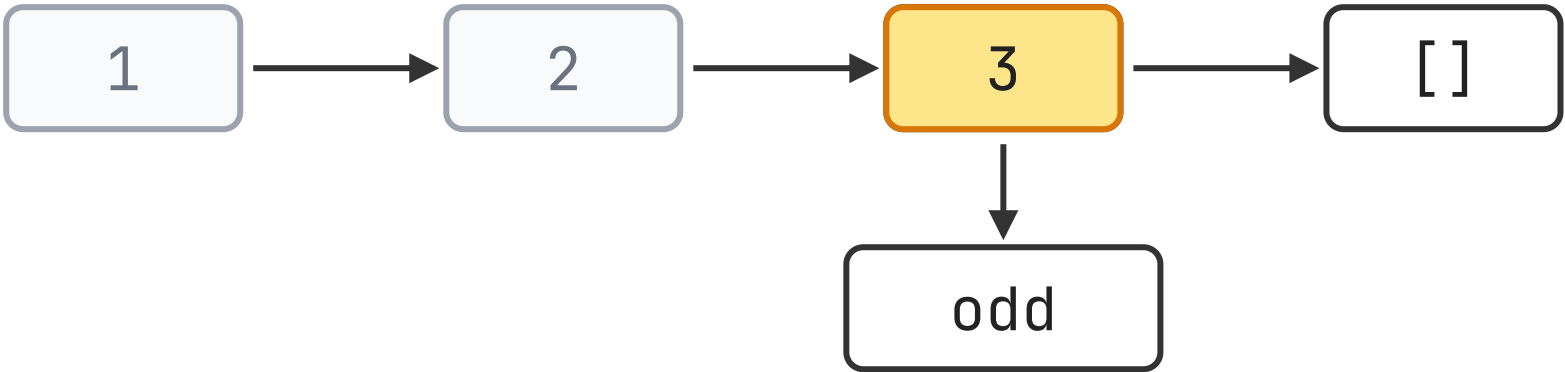
filter: оставляем подходящие

У хвоста [3] снова проверяем голову.

```
1 : filter odd [3]
```

ШАГИ
filter: проверяем odd
odd 1 $\rightsquigarrow$ True
odd 2 $\rightsquigarrow$ False

ВХОД



результат



НЕ ВКЛЮЧАЕМ

filter: оставляем подходящие

odd 3 = True: добавляем 3 после 1, сохраняя порядок.

```
1 : 3 : filter odd []
```

- ШАГИ
- filter: проверяем odd
- odd 1 ⇨ True
- odd 2 ⇨ False
- odd 3 ⇨ True

ВХОД



odd True: оставляем

результат



НЕ ВКЛЮЧАЕМ

filter: оставляем подходящие

`filter p [] = []`: конец входа завершает результат `[1,3]`.

`[1,3]`

ВХОД



ШАГИ
filter: проверяем odd
odd 1 ⇨ True
odd 2 ⇨ False
odd 3 ⇨ True
filter p [] ⇨ []

результат



НЕ ВКЛЮЧАЕМ

zip: две головы — одна пара

zip соединяет элементы попарно. Предскажите zip [1,2,3] ['a','b'].

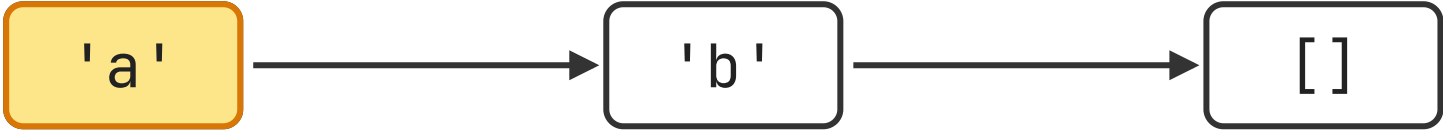
zip [1,2,3] ['a','b']

ШАГИ
zip: берём две головы

первый



второй



результат



zip: две головы — одна пара

Первые головы вместе образуют одну пару (1, 'a').

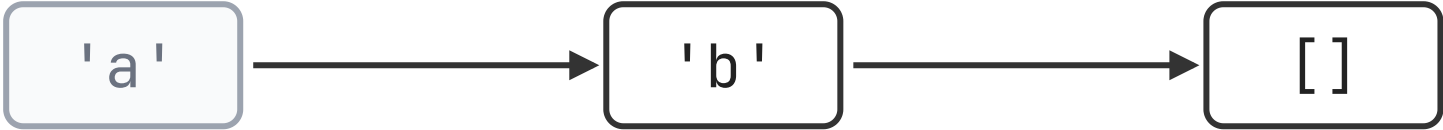
(1, 'a') : zip [2,3] ['b']

ШАГИ
zip: берём две головы
(1, 'a'): новая пара

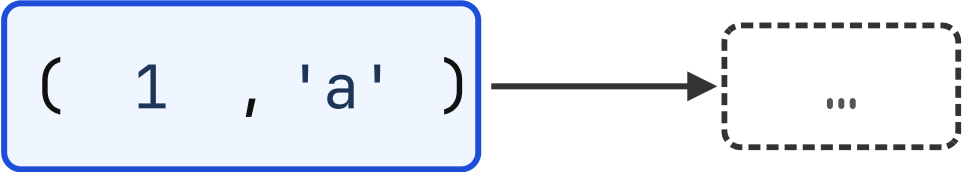
первый



второй



результат



zip: две головы — одна пара

Рекурсивный вызов получает оба хвоста: [2,3] и ['b'].

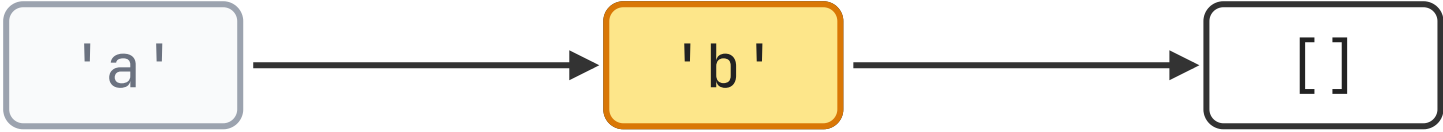
(1, 'a') : zip [2,3] ['b']

ШАГИ
zip: берём две головы
(1, 'a'): новая пара

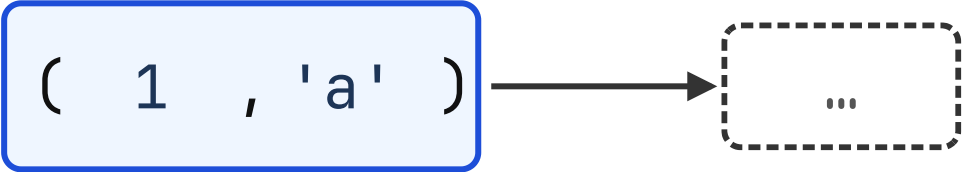
первый



второй



результат



zip: две головы — одна пара

Вторая пара — (2, 'b'). Теперь второй список пуст.

```
(1, 'a') : (2, 'b') : zip [3] []
```

ШАГИ
zip: берём две головы
(1, 'a'): новая пара
(2, 'b'): новая пара

первый



второй



результат



zip: две головы — одна пара

Один вход закончился: `zip xs [] = []`. Для 3 пары нет.

`[(1, 'a'), (2, 'b')]`

ШАГИ
zip: берём две головы
(1, 'a'): новая пара
(2, 'b'): новая пара
`zip [3] [] → []`

первый



второй



остаток [3] — без пары

результат



Дублицирование элементов списка

✎ Напишите `dupEvens :: [a] -> [a]`, дублицирующую элементы с индексами 0, 2, 4, ... Для `[10, 20, 30, 40]` результат — `[10, 10, 20, 30, 30, 40]`. Используйте `zip`, `map` и `concat`.

⚡ Решение.

```
dupEvens :: [a] -> [a]
dupEvens = concat . map f . zip [0..]
  where
    f :: (Int, a) -> [a]
    f (i, x) = if odd i then [x] else [x, x]
```

dupEvens: список после каждого этапа

Композиция применяется справа налево. Сначала добавим индексы.

```
dupEvens = concat . map f . zip [0..]  
f (i, x) = if odd i then [x] else [x, x]
```

ВХОД

[10, 20, 30, 40]

zip [0..]

map f

concat

dupEvens: список после каждого этапа

После zip у каждого элемента есть индекс. Теперь map f выберет один или два экземпляра.

```
dupEvens = concat . map f . zip [0..]  
f (i, x) = if odd i then [x] else [x, x]
```

ВХОД

[10, 20, 30, 40]

zip [0..]

[(0, 10), (1, 20), (2, 30), (3, 40)]

map f

concat

dupEvens: список после каждого этапа

После `map f` получился список коротких списков. Теперь `concat` соединит их.

```
dupEvens = concat . map f . zip [0..]  
f (i, x) = if odd i then [x] else [x, x]
```

ВХОД

[10, 20, 30, 40]

zip [0..]

[(0, 10), (1, 20), (2, 30), (3, 40)]

map f

[[10, 10], [20], [30, 30], [40]]

concat

dupEvens: список после каждого этапа

После concat получился плоский список. Индексы 0 и 2 дали по два экземпляра.

```
dupEvens = concat . map f . zip [0..]  
f (i, x) = if odd i then [x] else [x, x]
```

ВХОД

[10, 20, 30, 40]

zip [0..]

[(0, 10), (1, 20), (2, 30), (3, 40)]

map f

[[10, 10], [20], [30, 30], [40]]

concat

[10, 10, 20, 30, 30, 40]

Проектирование данных

type или newtype?

Синонимы

```
type ModuleId = Int
type CourseId = Int
```

Оба имени обозначают тот же тип `Int`.

Новые типы

```
newtype ModuleId = ModuleId Int
newtype CourseId = CourseId Int
```

Компилятор различает идентификаторы.

✎ **Устно:** Нужно запретить перестановку аргументов `publish :: ModuleId -> CourseId -> String`.
Какой вариант подходит?

⚡ **Решение.**

`newtype`: ошибку представления можно поймать по типам до запуска.

Parse, don't validate: CourseId

Проверка возвращает Bool. Тип следующей функции ничего не говорит о положительности номера.

Проверка: дальше остаётся Int

```
validateCourseId :: Int -> Bool  
validateCourseId n = n > 0
```

```
publish :: Int -> String  
publish n = "Course " ++ show n
```

Проверку нужно помнить при каждом вызове.

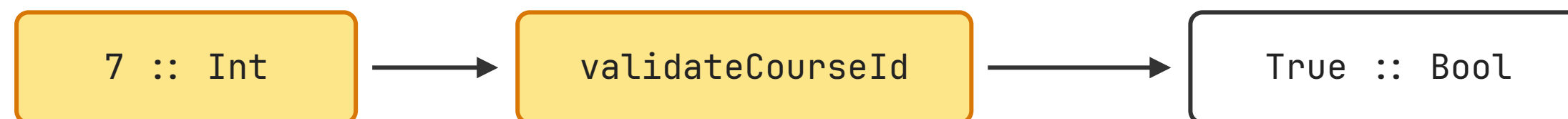
Справа конструктор CourseId скрыт: создавать такие значения может только парсер.

Парсинг: дальше нужен CourseId

```
module CourseId  
  (CourseId, parseCourseId, publish) where  
newtype CourseId = CourseId Int
```

```
parseCourseId :: Int -> Maybe CourseId  
parseCourseId n  
  | n > 0 = Just (CourseId n)  
  | otherwise = Nothing
```

```
publish :: CourseId -> String  
publish (CourseId n) = "Course " ++ show n
```



Число осталось Int; положительность в его типе не сохранена.

Parse, don't validate: CourseId

Проверку можно забыть или передать другое число. publish 0 подходит по типам.

Проверка: дальше остаётся Int

```
validateCourseId :: Int -> Bool
validateCourseId n = n > 0
```

```
publish :: Int -> String
publish n = "Course " ++ show n
```

Проверку нужно помнить при каждом вызове.
Справа конструктор CourseId скрыт: создавать такие значения может только парсер.

Парсинг: дальше нужен CourseId

```
module CourseId
  (CourseId, parseCourseId, publish) where
newtype CourseId = CourseId Int
```

```
parseCourseId :: Int -> Maybe CourseId
parseCourseId n
  | n > 0 = Just (CourseId n)
  | otherwise = Nothing
```

```
publish :: CourseId -> String
publish (CourseId n) = "Course " ++ show n
```



Bool не заставляет следующую функцию использовать проверенное число.

Parse, don't validate: CourseId

Парсер сохраняет положительный номер в CourseId. Из Just получаем значение с проверенным свойством.

Проверка: дальше остаётся Int

```
validateCourseId :: Int -> Bool  
validateCourseId n = n > 0
```

```
publish :: Int -> String  
publish n = "Course " ++ show n
```

Проверку нужно помнить при каждом вызове.

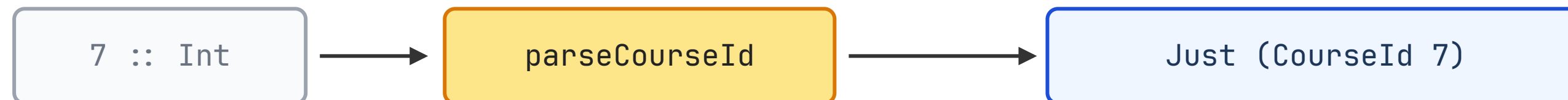
Справа конструктор CourseId скрыт: создавать такие значения может только парсер.

Парсинг: дальше нужен CourseId

```
module CourseId  
  (CourseId, parseCourseId, publish) where  
newtype CourseId = CourseId Int
```

```
parseCourseId :: Int -> Maybe CourseId  
parseCourseId n  
  | n > 0 = Just (CourseId n)  
  | otherwise = Nothing
```

```
publish :: CourseId -> String  
publish (CourseId n) = "Course " ++ show n
```



Успешный результат содержит само число и сохраняет положительность.

Parse, don't validate: CourseId

publish принимает CourseId. После Just cid передаём cid дальше без повторных проверок $n > 0$.

Проверка: дальше остаётся Int

```
validateCourseId :: Int -> Bool
validateCourseId n = n > 0
```

```
publish :: Int -> String
publish n = "Course " ++ show n
```

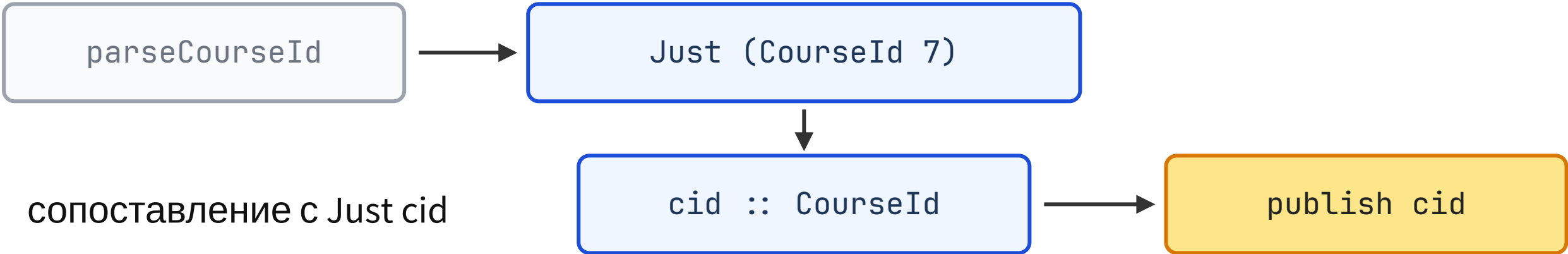
Проверку нужно помнить при каждом вызове.
Справа конструктор CourseId скрыт: создавать такие значения может только парсер.

Парсинг: дальше нужен CourseId

```
module CourseId
  (CourseId, parseCourseId, publish) where
newtype CourseId = CourseId Int
```

```
parseCourseId :: Int -> Maybe CourseId
parseCourseId n
  | n > 0 = Just (CourseId n)
  | otherwise = Nothing
```

```
publish :: CourseId -> String
publish (CourseId n) = "Course " ++ show n
```



Требование в типе функции: произвольный Int сюда не подходит.

Parse, don't validate: CourseId

Номер 0 отбрасывается на границе. Обойти парсер, передав publish обычный Int, нельзя.

Проверка: дальше остаётся Int

```
validateCourseId :: Int -> Bool  
validateCourseId n = n > 0
```

```
publish :: Int -> String  
publish n = "Course " ++ show n
```

Проверку нужно помнить при каждом вызове.

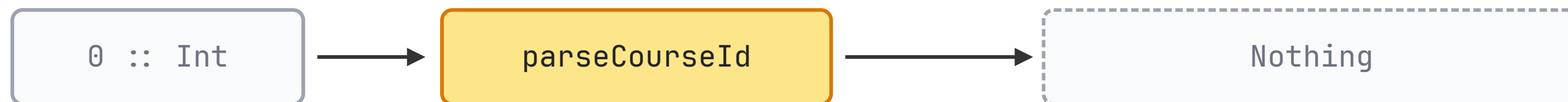
Справа конструктор CourseId скрыт: создавать такие значения может только парсер.

Парсинг: дальше нужен CourseId

```
module CourseId  
  (CourseId, parseCourseId, publish) where  
newtype CourseId = CourseId Int
```

```
parseCourseId :: Int -> Maybe CourseId  
parseCourseId n  
  | n > 0 = Just (CourseId n)  
  | otherwise = Nothing
```

```
publish :: CourseId -> String  
publish (CourseId n) = "Course " ++ show n
```



CourseId не создан: передавать следующим функциям нечего.

Проверяем один раз на входе; требование сохраняем в типах дальнейших функций.

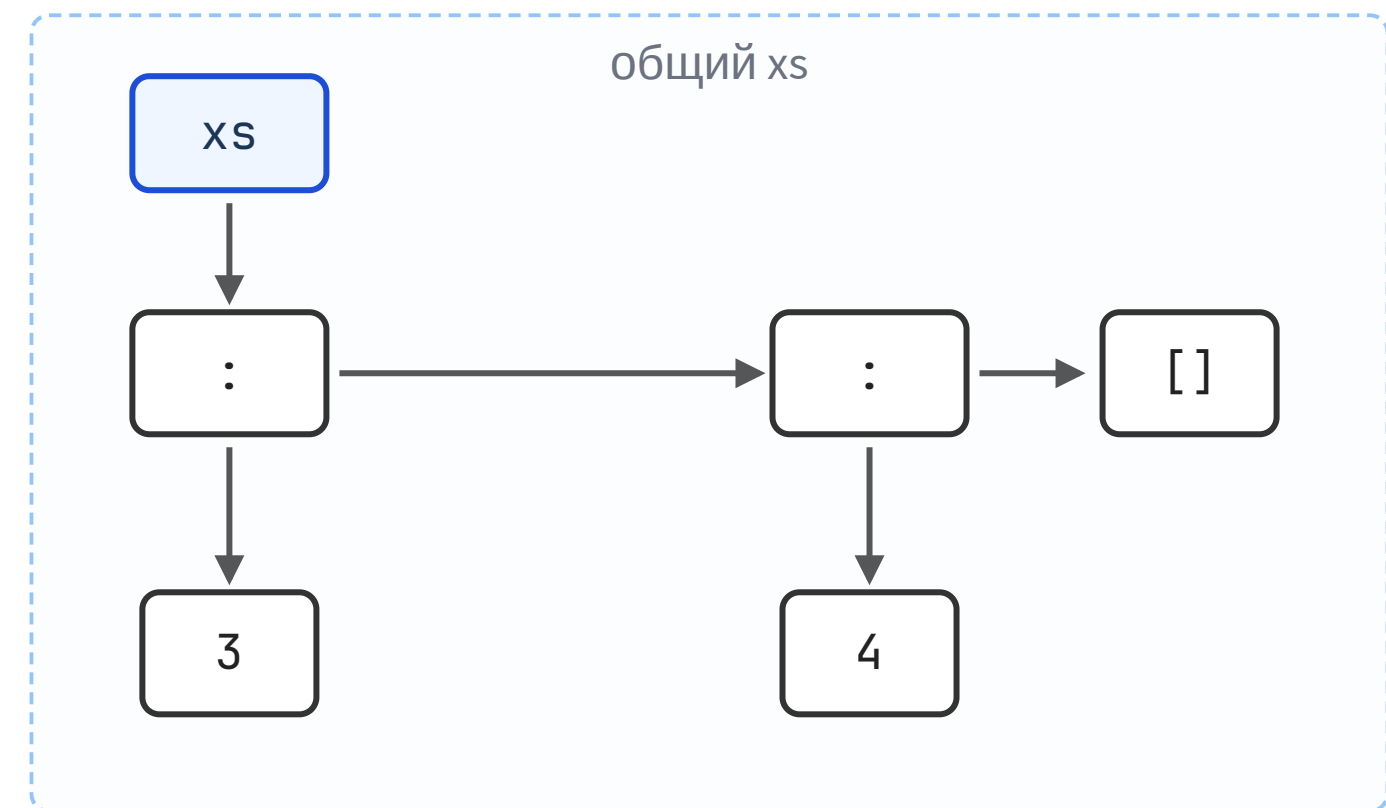
Персистентный список: новый корень, общий хвост

`xs` — исходный список `[3,4]`. У `(:)` есть голова и хвост.

`xs = [3,4]`

ЧТО ИЗМЕНИЛОСЬ

`xs`: два узла `(:)`

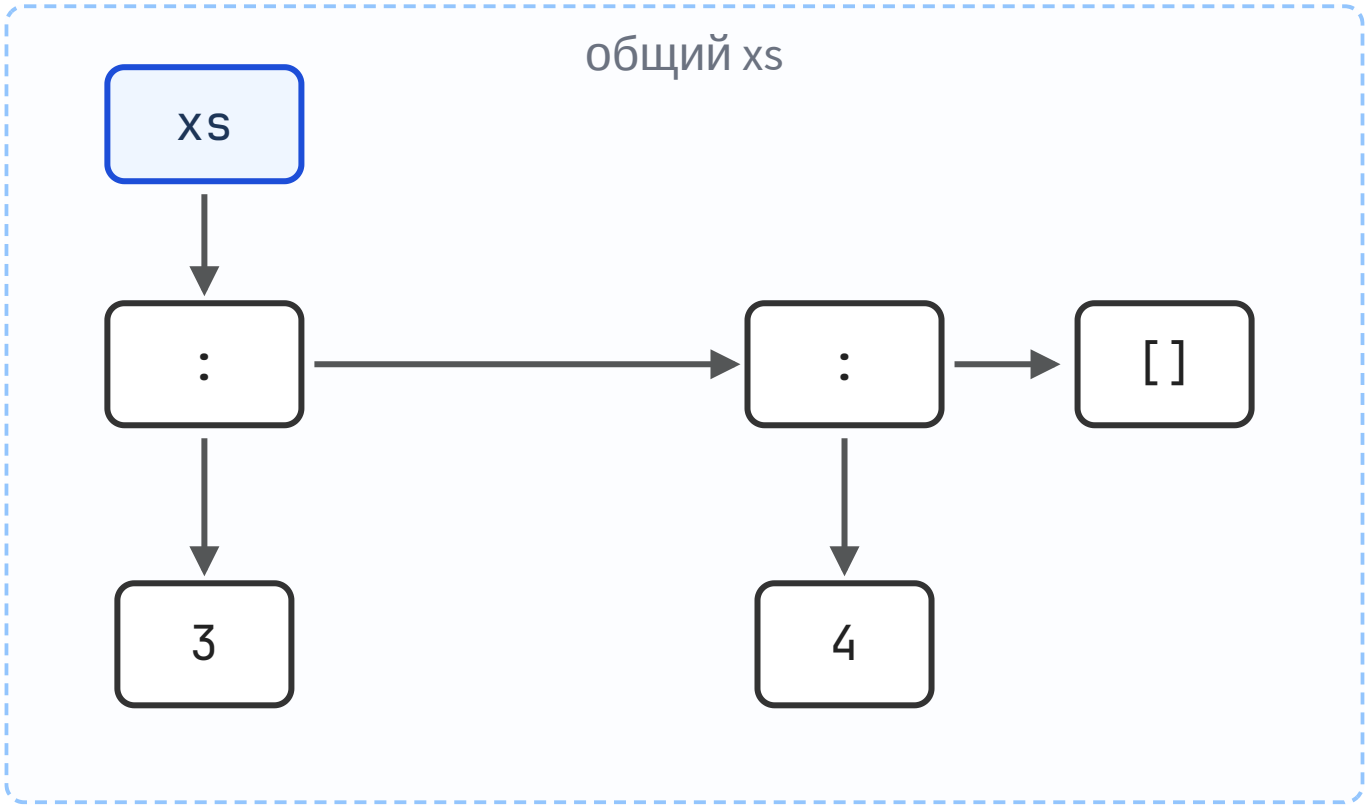
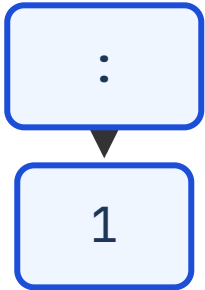


Персистентный список: новый корень, общий хвост

Для $ys = 1 : xs$ создаётся новая голова; старые узлы остаются на месте.

```
xs = [3,4]
ys = 1 : xs
```

ЧТО ИЗМЕНИЛОСЬ
xs: два узла (:)
новая голова 1

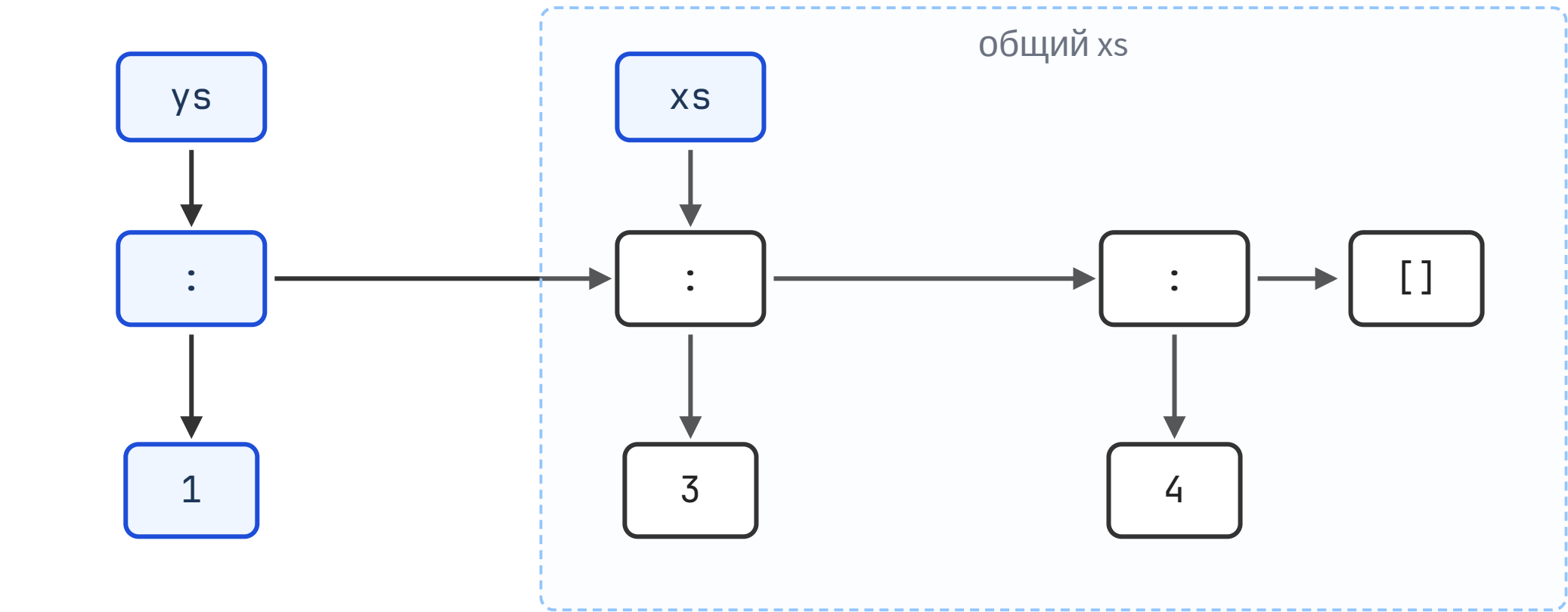


Персистентный список: новый корень, общий хвост

Хвост новой головы ссылается на уже существующий xs.

```
xs = [3,4]
ys = 1 : xs
```

ЧТО ИЗМЕНИЛОСЬ
xs: два узла (:)
новая голова 1
хвост ys — прежний xs

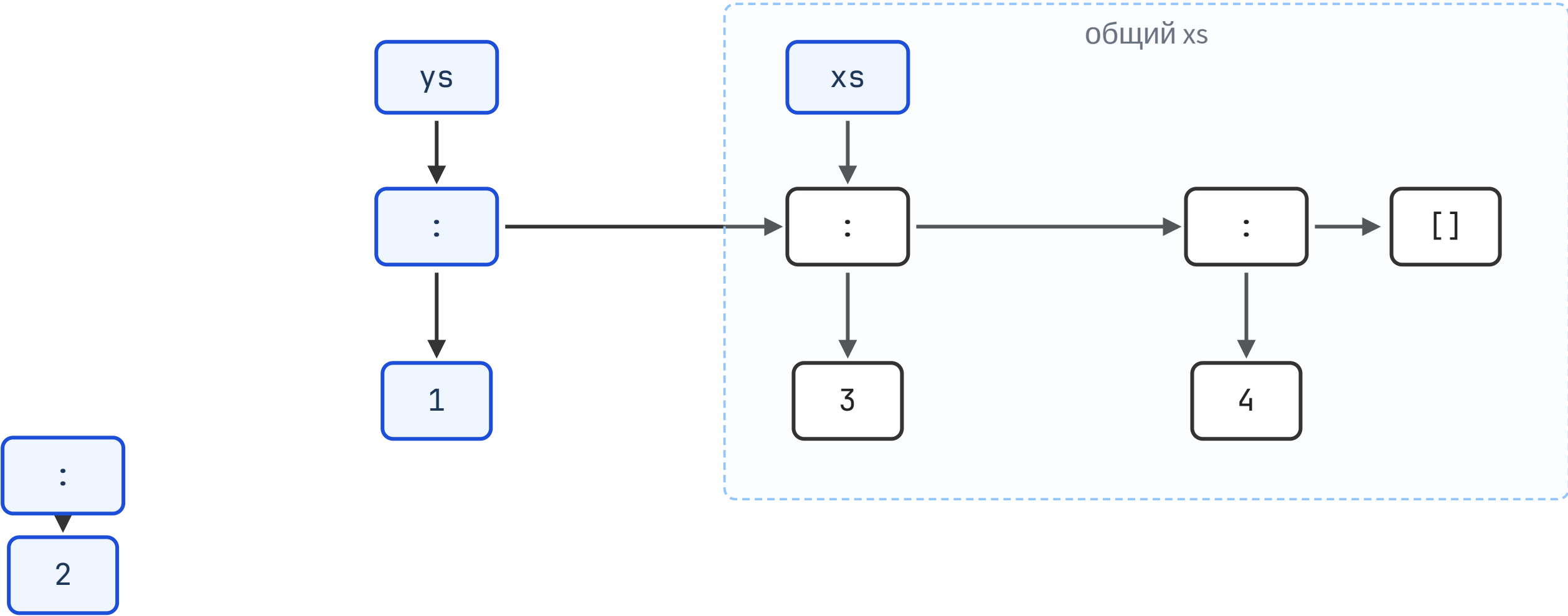


Персистентный список: новый корень, общий хвост

Для $zs = 2 : ys$ нужна ещё одна новая голова.

```
xs = [3,4]
ys = 1 : xs
zs = 2 : ys
```

ЧТО ИЗМЕНИЛОСЬ
xs: два узла (:)
новая голова 1
хвост ys — прежний xs
новая голова 2



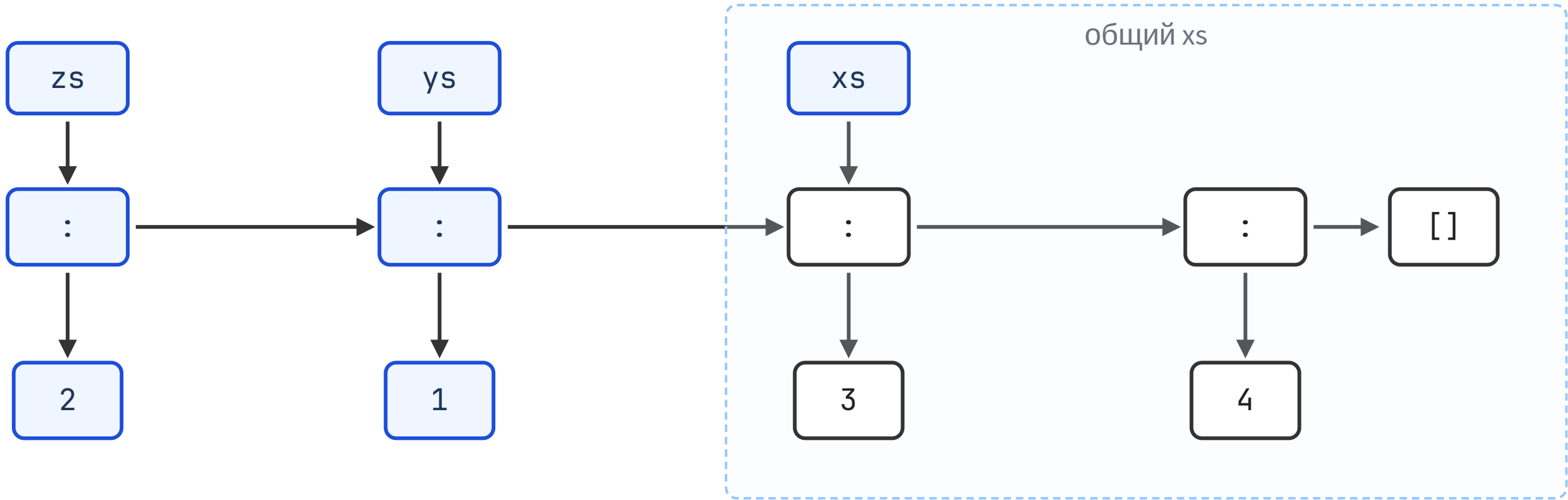
Персистентный список: новый корень, общий хвост

Все три имени доступны одновременно; xs — общий хвост ys и zs.

```
xs = [3,4]
ys = 1 : xs
zs = 2 : ys
```

ЧТО ИЗМЕНИЛОСЬ

- xs: два узла (:)
- новая голова 1
- хвост ys — прежний xs
- новая голова 2
- хвост zs — прежний ys**



Спроектируйте опции командной строки

✎ Утилита печатает справку без параметров или запускается с обязательным форматом `Plain/JSON` и выводом в `stdout` либо файл.

```
data Mode = HelpMode | RunMode
data Format = Plain | JSON
data BadOptions = BadOptions Mode (Maybe Format) (Maybe FilePath)
```

Спроектируйте `Options`, исключаящий неверные состояния. Сравните с `BadOptions` число состояний для двух путей `a.txt`, `b.txt` (`Nothing` означает `stdout`).

Разбор: сумма вариантов, произведение полей

Поля BadOptions выбираются независимо: 2 режима, 3 варианта формата и 3 варианта пути.

```
data Mode = HelpMode | RunMode
data Format = Plain | JSON
data BadOptions =
  BadOptions Mode (Maybe Format) (Maybe FilePath)
data Target = Stdout | File FilePath
data Options = Help | Run Format Target
```

СЧИТАЕМ СОСТОЯНИЯ
2 режима; 3 формата; 3 пути

Mode: 2

HelpMode | RunMode

Maybe Format: 1 + 2

нет | Plain | JSON

Maybe FilePath: 1 + 2

нет | a.txt | b.txt

2 × 3 × 3 – выбираем каждое поле

Разбор: сумма вариантов, произведение полей

Каждая комбинация полей даёт значение. Получается $2 \times 3 \times 3 = 18$ состояний.

```
data Mode = HelpMode | RunMode
data Format = Plain | JSON
data BadOptions =
  BadOptions Mode (Maybe Format) (Maybe FilePath)
data Target = Stdout | File FilePath
data Options = Help | Run Format Target
```

СЧИТАЕМ СОСТОЯНИЯ
2 режима; 3 формата; 3 пути
BadOptions: $2 \times 3 \times 3 = 18$

HelpMode

нет / нет	нет / a.txt	нет / b.txt
Plain / нет	Plain / a.txt	Plain / b.txt
JSON / нет	JSON / a.txt	JSON / b.txt

RunMode

нет / нет	нет / a.txt	нет / b.txt
Plain / нет	Plain / a.txt	Plain / b.txt
JSON / нет	JSON / a.txt	JSON / b.txt

$2 \times 3 \times 3 = 18$

Разбор: сумма вариантов, произведение полей

Для справки допустима только комбинация без формата и пути: 1 из 9.

```
data Mode = HelpMode | RunMode
data Format = Plain | JSON
data BadOptions =
  BadOptions Mode (Maybe Format) (Maybe FilePath)
data Target = Stdout | File FilePath
data Options = Help | Run Format Target
```

СЧИТАЕМ СОСТОЯНИЯ
2 режима; 3 формата; 3 пути
BadOptions: $2 \times 3 \times 3 = 18$
Help: 1 допустимое состояние

HelpMode: 1 из 9

нет / нет	нет / a.txt	нет / b.txt
Plain / нет	Plain / a.txt	Plain / b.txt
JSON / нет	JSON / a.txt	JSON / b.txt

RunMode

нет / нет	нет / a.txt	нет / b.txt
Plain / нет	Plain / a.txt	Plain / b.txt
JSON / нет	JSON / a.txt	JSON / b.txt

1 состояние справки; 8 лишних

Разбор: сумма вариантов, произведение полей

Для запуска нужен формат: $2 \times 3 = 6$ состояний. Вместе со справкой допустимы 7 из 18.

```
data Mode = HelpMode | RunMode
data Format = Plain | JSON
data BadOptions =
  BadOptions Mode (Maybe Format) (Maybe FilePath)
data Target = Stdout | File FilePath
data Options = Help | Run Format Target
```

СЧИТАЕМ СОСТОЯНИЯ

2 режима; 3 формата; 3 пути

BadOptions: $2 \times 3 \times 3 = 18$

Help: 1 допустимое состояние

Run: $2 \times 3 = 6$; всего 7

HelpMode: 1 из 9

нет / нет	нет / a.txt	нет / b.txt
Plain / нет	Plain / a.txt	Plain / b.txt
JSON / нет	JSON / a.txt	JSON / b.txt

RunMode: 6 из 9

нет / нет	нет / a.txt	нет / b.txt
Plain / нет	Plain / a.txt	Plain / b.txt
JSON / нет	JSON / a.txt	JSON / b.txt

$1 + 6 = 7$ допустимых; $18 - 7 = 11$ лишних

Разбор: сумма вариантов, произведение полей

Options хранит выбор ветки: Help либо Run. Получается $1 + 2 \times 3 = 7$ состояний.

```
data Mode = HelpMode | RunMode
data Format = Plain | JSON
data BadOptions =
  BadOptions Mode (Maybe Format) (Maybe FilePath)
data Target = Stdout | File FilePath
data Options = Help | Run Format Target
```

СЧИТАЕМ СОСТОЯНИЯ

2 режима; 3 формата; 3 пути

BadOptions: $2 \times 3 \times 3 = 18$

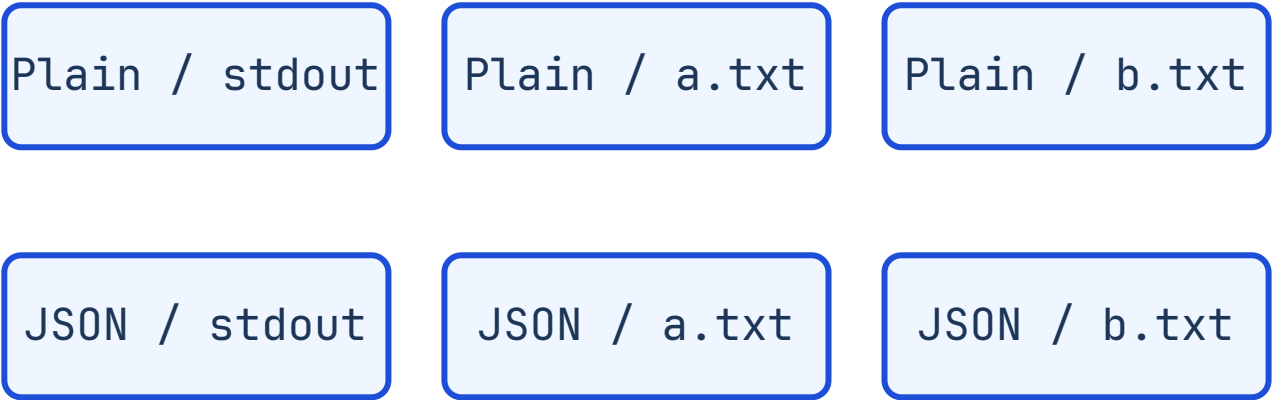
Help: 1 допустимое состояние

Run: $2 \times 3 = 6$; всего 7

Options: $1 + 2 \times 3 = 7$

Help: 1

Run: $2 \times 3 = 6$



$1 + 2 \times 3 = 7$ — все состояния допустимы

Что вы теперь умеете

- Уметь читать декларации алгебраических типов данных
- Пользоваться сопоставлением с образцом
- Писать рекурсию по дереву и списку
- Различать терм, тип и кайнд
- Выбирать `Maybe`, `type` и `newtype` по смыслу задачи
- Выражать допустимые состояния суммой и произведением типов

Материалы

Если тип применили неправильно

```
ghci> :kind Maybe Maybe
<interactive>:1:7: error: [GHC-83865]
• Expecting one more argument to ‘Maybe’
  Expected a type, but ‘Maybe’ has kind ‘* -> *’
• In the first argument of ‘Maybe’, namely ‘Maybe’
  In the type ‘Maybe Maybe’
```

`Maybe` ждёт обычный тип, а ему передали конструктор типа.

Кайнды: дополнительные примеры

```
data Free f a = Pure a | Free (f (Free f a))
-- Free :: (* -> *) -> * -> *

data HFree f a = HPure a | HFree (f (HFree f) a)
-- HFree :: ((* -> *) -> * -> *) -> * -> *
```

Полиморфные типы и система $\lambda\omega$

```
type Pair a b = forall c. (a -> b -> c) -> c
```

$\text{Pair Int Char} \equiv \text{forall } c. (\text{Int} \rightarrow \text{Char} \rightarrow c) \rightarrow c$

Pair : $* \rightarrow * \rightarrow *$

Pair = $\lambda\alpha^* \beta^*. \forall\gamma. (\alpha \rightarrow \beta \rightarrow \gamma) \rightarrow \gamma$

pair : $\forall\alpha \beta. \alpha \rightarrow \beta \rightarrow \mathbf{Pair} \alpha \beta$

pair = $\Lambda\alpha^* \beta^*. \lambda x^\alpha y^\beta. (\Lambda\gamma^*. \lambda f^{\alpha \rightarrow \beta \rightarrow \gamma}. f \ x \ y)$

fst : $\forall\alpha \beta. \mathbf{Pair} \alpha \beta \rightarrow \alpha$

fst = $\Lambda\alpha^* \beta^*. \lambda p^{\mathbf{Pair} \alpha \beta}. p \ \alpha \ (\mathbf{K} \ \alpha \ \beta)$

Records и **coerce**: подробности

- Селекторы полей могут быть частичными у типов-сумм
- Для полей используют префиксы; дополнительные варианты синтаксиса дают расширения GHC
- **newtype** позволяет разделить типы без дополнительной обёртки во время исполнения

```
import Data.Coerce (coerce)
intsToIds :: [Int] -> [CourseId]
intsToIds = coerce
```

Расширения records · Parse, don't validate

Типы-произведения в C++

```
#include <string>
struct CatT {
    std::string name;
    int age;
};
CatT createBarsik() { return CatT{"barsik", 4}; }
std::string getName(const CatT& cat) { return cat.name; }
```

Поля — компоненты произведения; конструктор создаёт значение, доступ к полю извлекает компонент.

Типы-суммы в C

```
struct Pet {
    enum { CAT, DOG, PARROT } tag;
    union {
        struct { const char *name; int age; } cat;
        struct { const char *name; int age; } dog;
    } value;
};
/* tag и value должны согласовываться */
switch (pet.tag) {
case CAT:    printf("Cat: %s", pet.value.cat.name); break;
case DOG:    printf("Dog: %s", pet.value.dog.name); break;
case PARROT: printf("Parrot"); break;
}
```

Типы-суммы в C++

```
std::variant<Cat, Dog, Parrot> pet{Cat{"barsik", 4}};  
if (std::holds_alternative<Cat>(pet)) {  
    std::cout << "Cat " << std::get<Cat>(pet).name;  
} else if (std::holds_alternative<Dog>(pet)) {  
    std::cout << "Dog " << std::get<Dog>(pet).name;  
} else if (std::holds_alternative<Parrot>(pet)) {  
    std::cout << "Parrot";  
}
```

std::variant

Типы-суммы в Python

```
class Pet: pass
class Cat(Pet):
    def __init__(self, name): self.name = name
class Dog(Pet):
    def __init__(self, name): self.name = name
class Parrot(Pet): pass

if isinstance(pet, Cat):
    print("Cat " + pet.name)
elif isinstance(pet, Dog):
    print("Dog " + pet.name)
elif isinstance(pet, Parrot):
    print("Parrot")
else:
    raise AssertionError("unreachable code") # No exhaustiveness check
```

Типы-суммы в Kotlin

```
sealed class Pet
class Cat(val name: String) : Pet()
class Dog(val name: String) : Pet()
class Parrot : Pet()

fun showPet(pet: Pet): String = when (pet) {
    is Cat -> "Cat: " + pet.name
    is Dog -> "Dog: " + pet.name
    is Parrot -> "Parrot"
}
```

Иерархия `sealed` позволяет компилятору проверять полноту выражения `when`.

Что посмотреть в транспорте

- Parse, don't validate
- (youtube) Тагир Валеев — Pattern matching в Java и его воображаемые друзья
- The algebra (and calculus!) of algebraic data types